

Vizing's Theorem in Rocq

A THESIS PRESENTED
BY
K. MILLIGAN GRINSTEAD
TO
THE DEPARTMENT OF COMPUTER SCIENCE
AND
THE DEPARTMENT OF MATHEMATICS

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE JOINT DEGREE WITH HONORS OF
BACHELOR OF ARTS

HARVARD UNIVERSITY
CAMBRIDGE, MASSACHUSETTS
MAY 2026

©2026 – K. MILLIGAN GRINSTEAD
ALL RIGHTS RESERVED.

Vizing's Theorem in Rocq

ABSTRACT

This thesis explores Vizing's theorem in the proof assistant Rocq. We employ the SSReflect language and the open-source Mathematical Components library to extend the graph theory library developed by Christian Doczkal, Damien Pous, and Daniel Severín. Specifically, we implement edge coloring and a proof of Vizing's theorem in Rocq. The corresponding code can be viewed in the appendix or here <https://github.com/milligang/vizing-thesis>. We create the necessary data structures and types to support the proof, and we discuss the representations chosen. In addition, we present a detailed written mathematical proof of the theorem which the formalized Rocq proof follows.

Contents

1	INTRODUCTION	I
1.1	A Brief History	I
1.2	Motivation	4
1.3	Related Work	6
1.4	Background and Outline	8
2	A PROOF OF VIZING’S THEOREM	10
2.1	Background	11
2.2	Auxiliary Data Structures and Lemmas	14
2.3	Main Proof	38
3	IMPLEMENTATION	42
3.1	Introduction to Rocq	44
3.2	The Graph Theory Library	47
3.3	Implementation details and decisions	48
3.4	Omitted Material	59
4	CONCLUSION	61
	REFERENCES	67
	APPENDIX A THE CODE	68
A.1	Basic Examples	68
A.2	Auxiliary Material	69
A.3	Simple Paths	74
A.4	Edge Colorings	80
A.5	Fans and Rotation	94
A.6	Kempe chains and Kempe swaps	105
A.7	Alternating Paths	115
A.8	Vizing’s Theorem	123

Acknowledgments

This thesis could not have been completed without a strong network of mentors, friends, and family, and I would like to especially thank a few of these people here.

Thank you to my advisor Philip Wood for his dedication and continued support, for his thoughtful comments to each question and draft, and for knowing exactly when to nudge me in the right direction as I navigated the web of graph coloring theory. Thank you too for fostering my appreciation of graph theory in Harvard's summer REU program of 2024.

Thank you to my advisor Nada Amin for her generous support of this project, for her availability and insightful comments, and for revealing to me the universe of formal verification in CS 1520.

Thank you to Ana Kravchuk-Kirilyuk for her constant encouragement and for always being willing to brainstorm ideas.

Thank you Dr. Pavelescu who first introduced me to the graphs that have captivated my curiosity, and thank you to all those teachers that showed me the beauty and satisfaction found in both mathematics and computer science.

Finally, thank you to McGowin, Julia, Abe, and all those I hold dear in Currier, Harvard, Alabama, California, D.C., and beyond.

Ad maiorem Dei gloriam

1

Introduction

1.1 A BRIEF HISTORY

In 1851 in Victorian England, Francis Guthrie shared a simple observation with his younger brother, challenging mathematicians for decades and laying the foundations for a new subfield in graph theory. As the story goes, Guthrie, an English lawyer, botanist, and mathematician, was coloring maps of England when he noticed the “greatest necessary number of colours to be used in colouring a map so as to avoid identity of colour in lineally contiguous districts is four” [1]. For a given map, if we associate each region with a vertex and include an edge between two vertices exactly when the two corresponding re-

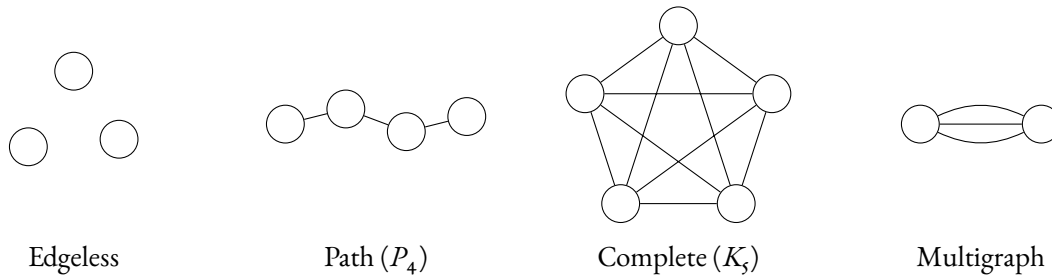


Figure 1.1: Three examples of simple graphs and one example of a multi-graph.

gions are adjacent, then we have constructed a graph. A *graph* $G = (V, E)$ refers to a set of vertices $V(G)$ and a set of edges $E(G)$ between these vertices. A few examples of graphs are displayed in Figure 1.1. The maps Guthrie discussed correspond to a special type of graphs called planar graphs. A *planar graph* is a graph that can be drawn without edge-crossings, i.e. it can be embedded in the 2-dimensional plane. Every graph shown in Figure 1.1 is planar except the complete graph K_5 . Thus, Guthrie’s observation can be stated more formally:

Theorem 1.1 (The Four Color Theorem). For any planar graph G , the chromatic number $\chi(G)$ of G is at most 4.

Here, the *chromatic number* of a graph G , denoted $\chi(G)$, is the smallest number of colors required to color the vertices of a graph such that no two adjacent vertices share a color. Vertices are considered *adjacent* if there is an edge between them.

Though the first written statement of Theorem 1.1 appeared in 1852, it went largely unnoticed beyond a few correspondences until mathematician Arthur Cayley published a short article regarding the problem in 1878 as mentioned in [2]. Before the end of the decade, a man named Sir Alfred Kempe had published a “complete” proof [3]. However, 11 years after this supposed proof had been accepted by the mathematical community, it was disproved by Percy Heawood [4]. While unable to rectify Kempe’s

proof, he readily extracted a proof of the five color theorem. Additionally, Kempe’s novel chain construction would become central to edge colorings. Perhaps the two most famous uses of these chains were Dénes Kőnig’s and Vadim Vizing’s publications bounding the chromatic indices of finite graphs [5, 6] as we will mention in Section 1.2. Meanwhile, allured by its child-like simplicity, mathematicians would spend more than a century researching the four color theorem before the first viable proofs emerged with the assistance of computers.

In 1976, Appel and Haken invested over 1000 hours of computer time to check 1,936 configurations and finally verified the four color theorem [7]. However, their proof was met with controversy and hesitation: this was the first major mathematical result to be solved with the help of a computer. Mathematicians struggled to accept a proof which could not be personally checked by peer reviewers and diverged so notably from the mainstream mathematical reasoning found in proofs [8]. Following its publication, the mathematical community rapidly began sharing revisions, each requiring fewer computer-assisted cases, desperate to solidify the result.

This solution and its subsequent simplifications, with their reliance on hand-written computer code subject to programmer error and a history scattered with false claims, demonstrate both the beauty and the necessity of formal verification. With the development of interactive theorem provers, also known as proof assistants, one needs only to rely on the system itself, and often times even the kernels of these systems have themselves been formally verified¹. In addition, the necessary detail required to formalize the proof alleviated some of the disappointment with Appel and Haken’s original proof, that computers buried mathematical insight behind ones and zeros. As Georges Gonthier remarked, “to produce a formal proof one must make explicit every single logical step of a proof; this both provides new insight in the structure of the proof, and forces one to use this insight to discover every possible

¹See, e.g., the MetaRocq project [9].

symmetry, simplification, and generalization,” [10]. That is, the author must intimately understand the proof to formalize it. In 2008, Gonthier published a fully verified proof of the four color theorem [10] as the first substantial mathematical result to be implemented in a proof assistant. Today, the four color theorem is widely accepted and Gonthier’s proof is in a well-maintained open-source library. Thus, while the impacts of this theorem continue to shape mathematics today, we conclude our short tale by encouraging the reader to glance at Gonthier’s formalization in Rocq [11].

1.2 MOTIVATION

As mathematicians labored over the four color theorem, a plethora of partial results, adjacent theorems, and new proof techniques established graph coloring as its own subfield. In addition to vertex colorings, there is also substantial research surrounding edge colorings, total colorings, balanced colorings, and fractional colorings, to name a few. This thesis will focus specifically on edge colorings. Similar to a vertex coloring, an *edge coloring* assigns colors to the edges of a graph. In fact, edge colorings can be regarded as special cases of vertex colorings. The *line graph* of a graph G , also commonly referred to as the conjugate or edge-to-vertex dual, can be constructed by associating each edge with a vertex and connecting the vertices if and only if the corresponding edges share a common vertex in G . Then an edge coloring of a graph is exactly the vertex coloring of its line graph.

The *chromatic index* of a graph G , denoted $\chi'(G)$ or simply χ' when G is inferred and defined analogously to the chromatic number, is the smallest number of colors required to color the edges of a graph such that no two adjacent edges share a color. Edges are considered *adjacent* if they share a common vertex. In 1964 and 1967, respectively, Vadim Vizing[6] and Gupta[12] independently published the most famous theorem in edge coloring:

Theorem 1.2 (Vizing’s Theorem). For any finite, simple graph G , we have $\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1$

Here, the maximum degree $\Delta(G)$ or Δ of a graph G is the maximum number of edges at any vertex in $V(G)$. Vizing’s theorem imposes a much stricter bound on the chromatic index than exists for the chromatic number of an arbitrary graph. We saw that at most four colors were required in the vertex coloring of a planar graph; for non-planar graphs, Brooks showed χ is between 2 and $\Delta + 1$ [13]. With such a range, it is all the more shocking the chromatic index is restricted to only two values. This allows us to classify graphs as Class-I if $\chi' = \Delta$ and Class-II if $\chi' = \Delta + 1$. Kőnig obtained the first major edge coloring result in 1916 when he proved *bipartite* graphs [5], that is, graphs with chromatic number 2, are always Class-I. However, despite the apparent simplicity in contrast to vertex colorings, there are many open problems on characterizing Class-II graphs [14], and classifying a graph as Class-I or Class-II is NP-hard for arbitrary graphs [15].

According to statistics collected by Crescenzi and Kann, graph coloring is a central issue in combinatorial research on NP-complete and NP-hard problems today [16]. Furthermore, graph coloring has numerous applications in both formal and applied mathematics; for a more comprehensive list, see the survey by Thakare and Bhapkar [17] containing popular results and open problems related to graph coloring as well as references to some of the field’s applications. Edge coloring in particular appears in route planning, transfer assignment in computer networks, latin squares, matrix algebra, and quantum field theory, to name a few [18, 19, 20, 21, 22].

Meanwhile, formal verification systems have a wide range of applications from pure mathematics to systems engineering and artificial intelligence. Notable examples include the odd-order theorem [23], the verified C compiler CompCert[24], and recent efforts to integrate formal reasoning into artificial intelligence systems at companies like Axiom [25] and Harmonic [26]. Gonthier’s work demonstrates the potential of integrating these proof assistants with graph theory. In fact, most of these systems to-

day already maintain graph theory libraries. For instance, Lars Noschinski wrote a library for directed graphs in Isabelle [27], the Lean Mathematical Library contains a graph theory component [28], and Rocq itself has two well-developed theoretical graph libraries [29, 30]. Edge coloring is a significant field of research, but the current scope of Rocq libraries do not yet include it. Our `edge-coloring` file defines an edge coloring function as well as basic actions and lemmas regarding recoloring and swapping colors, absent sets, and trivial colorings. Integrating this with the library would greatly benefit the Rocq community by establishing a standard definition capable of utilizing the rich set of graph results already present. Adding Kempe chains and a proof of Vizing’s theorem, from which we can conclude all graphs are either Class-I or Class-II, to the library would also support many future endeavors including the open problems we alluded to above.

1.3 RELATED WORK

Both Vizing’s and Gupta’s proofs, while correct, lacked simplicity. As such, several variations on the original works have been published in an attempt to find a more elegant approach. Bollobás sketched a proof in 1979 [31], Rao and Dijkstra developed algorithms with a proof of termination and auxiliary data structures [32], and Misra and Gries further simplified these [33]. Many expository works and introductory graph theory text books have also compiled a proof. In particular, we were inspired by [34], [35], and [36]. Even within these publications, the proof that χ' is Δ or $\Delta + 1$ remains rather involved, often dependent on fragile relationships between specialized data structures. Additionally, proofs gloss over some of the finer-grained details and omit non-trivial cases, intentionally or unintentionally. This, coupled with the wide literature on edge colorings, prompted the formalization of Vizing’s theorem in first Rodin, then Lean, and now Rocq. In the same way variations on Vizing’s original proof hold value even though the result is known, it is beneficial to formalize it multiple times for a variety of reasons.

Firstly, it strengthens the implementation when it spans multiple languages and is not reliant on a single system. For the remainder of this section, we will discuss our project in comparison to related works.

In 2011, Joachim Breitner modeled Misra and Gries’s algorithm in Event-B with verification in Rodin[37]. While sound according to the system, the individual lemmas in Rodin are write-once, read-never (WORN) and, most importantly, they rely on the soundness of Rodin itself. Although this is the case for every implementation into a formalization system, reliance on the system’s soundness is especially bold with Rodin because it has a history of errors. Breitner himself acknowledged “relying only on the Rodin platform for verifying the proof obligations would not constitute a rigorous proof.”

Undergraduates Aarbi, He, and Gatti wrote a structural template to formalize Vizing’s theorem in Lean4 with Mathlib as part of a spring 2025 research project at the University of Washington [38]. Before Bhoja’s publication in December of 2025, this was our only reference for an attempted formalization of Vizing’s theorem in a language based on the calculus of inductive constructions. Their definition of edge colorings and use of a dependent color type supported our final decision to represent edge colorings similarly. Additionally, we appreciated their inductive set-up and the creation of an explicit coloring to trivially solve the base case. Of course, their project merely outlined a potential approach and consisted mostly of comments and statements with empty proofs.

The original vision for this thesis was to construct the first complete proof of Vizing’s theorem in a verified proof assistant. A little over half-way through our project, Bhoja published a complete Lean implementation of Misra and Gries’s algorithm in December of 2025[39]. While this thesis shares similarities to Bhoja’s approach, there are several key differences. Most importantly, we wish to offer formalized edge colorings to the Rocq community. Bhoja sought to create an efficient polynomial-time algorithm and as such did not work intending to contribute to Lean’s existing graph theory libraries, instead creating specialized data structures from scratch. We, on the other hand, though valuing efficiency

in our design, acted with the goal of extending Rocq’s libraries as discussed in Section 1.2. Also, migrating the mathematical proofs into Rocq was non-trivial, even with the example in Lean, for, while Lean and Rocq have similar foundational logic, they differ in syntax as well as current results and definitions within their respective libraries, including the encouragement of entirely different graph structures. We also provide greater detail in our written proofs to more closely align with the code in Rocq, and in Chapter 3 we elaborate on implementation decisions made, especially in comparison to Bhoja’s method. Finally, we formalize the lower bound of Vizing’s theorem which, though shorter, is not included in any of the other projects we have mentioned.

1.4 BACKGROUND AND OUTLINE

As mentioned, a graph $G = (V, E)$ refers to a set of vertices $V(G)$ and a set of edges $E(G)$ between these vertices. For any two vertices x, y in $V(G)$, we will use the pair notation (x, y) to denote an edge connecting x and y . All graphs hereafter are considered finite, simple, and undirected unless otherwise specified. That is to say, the set $V(G)$ is finite, there is at most one edge between any two vertices, there are no self-loops, and for any two vertices x, y in $V(G)$ such that (x, y) is in $E(G)$, we have (y, x) in $E(G)$.

This thesis examines the edge coloring of simple graphs with formal verification. In particular, we prove Vizing’s theorem on the chromatic index of a graph in the proof assistant Rocq. *Formal verification* is the application of formalized mathematical methods to prove the correctness of a statement or system. *Proof assistants* are software systems that employ formal methods to allow us to construct proofs as computer programs and mechanically verify their correctness. The validity of a proof is migrated from humans to the correctness of the proof assistant itself. They are especially powerful for proofs involving many cases, as seen with the four color theorem. The guarantees offered by proof assistants

can be a strong foundation for applications that require a high level of certainty such as security systems, computer kernels, and hardware design. Examples of these systems include Rocq, Lean, Isabelle, Agda, Mizar, and Metamath. First released in 1989, Rocq (formerly Coq) has gained traction in the computer science community. It is constructive in nature (that is, it does not assume the axiom of the excluded middle), has well-developed libraries for large-scale mathematical proofs such as Gonthier’s four color theorem, and supports the immediate extraction of verified algorithms to runnable programs.

In the remainder of this thesis, we will describe our implementation of Vizing’s theorem in Rocq. In Chapter 2, we present our own complete proof of Vizing’s theorem and supporting results. As much as is reasonable, we try to align these proofs with those in Rocq. We also offer commentary relating these definitions and lemmas to the implementation process, though Chapter 3 explores the details and challenges of our execution in greater depth. Specifically, Chapter 3 focuses on the project’s main contributions and the most architecturally significant sections in the code. Finally, we summarize the project in the Conclusion, touching on potential directions for future work. The Rocq code is displayed in Appendix A and can also be accessed from our GitHub repository [40].

*There are no lines in nature, only areas of color, one
against another.*

Édouard Manet

2

A Proof of Vizing's Theorem

In this chapter, we present a thorough proof of Theorem 1.2 (Vizing's theorem) for simple graphs. Recall that Vizing's theorem states the chromatic index of a graph is at least the maximum degree and at most the maximum degree plus one. We begin this chapter by defining edge colorings in Section 2.1. From this definition we can already lower bound the chromatic index by the maximum degree. For the upper bound, we will use induction to create an edge coloring of G that uses $\Delta + 2$ colors. The aim will then be to construct a new edge coloring that uses one less color. For this, we will need two new structures, *fans* and *Kempe chains*. We also will define a *rotation* and an *inversion* operation for fans and Kempe chains, respectively, to modify the colors of the edges while preserving the properties of the

original edge coloring. Fans and Kempe chains rely on *absent sets*, that is, the set of colors which are not present on any edges adjacent to a given vertex. Absent sets, fans, and Kempe chains are addressed in turn in Section 2.2. For each lemma we will, where applicable, note the location of the corresponding lemma and proof in the code in Appendix A. Once these are established, we complete our proof of Vizing's theorem in Section 2.3. To decrease the number of colors on the edges, we will create a maximal fan, build a Kempe chain at the last edge in the fan, invert the vertices along the chain, rotate the fan, and finally recolor the last edge of the rotated fan with a carefully selected color such that we still have a valid coloring and have removed the old color at this edge entirely from the graph.

2.1 BACKGROUND

A graph G is defined by its vertices $V(G)$ and edges $E(G)$. Every edge is *incident* to two vertices and is said to *join* these two vertices; we only concern ourselves with loop-less graphs, so we may assume these two vertices are distinct. Two vertices are considered *adjacent vertices* if there exists some edge in $E(G)$ joining them. Two vertices x and y are said to be *connected* if one can get from x to y by only stepping through adjacent vertices in G . A non-repeating sequence of vertices connecting x to y , including x and y themselves, is called a *path*. The *neighborhood* of a vertex x is the set $N_G(x) = \{y \mid (x, y) \in E(G)\}$ of vertices adjacent to x in a graph G , often written as $N(x)$ when G can be inferred. Thus, adjacent vertices can also be called neighbors. We denote by $E_G(x) = \{(x, y) \mid (x, y) \in E(G)\} = \{(x, y) \mid y \in N(x)\}$ the set of edges incident to a vertex x . Two distinct edges in $E_G(x)$ are said to be *adjacent edges*. The degree $d_G(x) = |N_G(x)| = |E_G(x)|$ of a vertex x in G is simply the number of vertices adjacent to it, or equivalently, the number of edges which are an incident to it. When it is clear from context, $d_G(x)$ will be written as $d(x)$. The maximum degree $\Delta(G) = \max_{x \in V(G)} d(x)$ of a graph G is the maximum value of $d(x)$ over all vertices x in G .

An *edge coloring* of a graph G is a map $C : E(G) \rightarrow \{c_1, \dots, c_k\}$ from edges in $E(G)$ to colors $c_1, \dots, c_k$. We will denote by $C[\mathcal{A}]$ the image under C of $\mathcal{A}$, so $C[E(G)]$ is the range of C . The range of C is typically referred to as the *color set* of C and an item in this color set is called a *color*. The edge coloring $C[e \mapsto c_i]$ refers to the edge coloring C with some edge e recolored as some color c_i . We will also use $\text{Swap}(C, e, f)$ as shorthand for $C[e \mapsto C(f); f \mapsto C(e)]$ where e and f are any two edges. In other words, $\text{Swap}(C, e, f)$ is the edge coloring C with the colors of edges e and f swapped. Swapping edge colors does not add or remove colors from the color set. Note that there may be colors in the color set of C which are not assigned to any edges because C is not necessarily surjective. If this is case, the color set of C would be a proper subset of the co-domain. Thus, we can have unused colors, recolor edges with colors not already present in the graph, and be explicit about when a color must specifically be in the color set as opposed to generally in the co-domain (especially regarding Kempe swaps in Section 2.2.3 which potentially remove colors from the color set).

An edge coloring is a *proper edge coloring* if, for all vertices x in $V(G)$, if there are two edges e_1, e_2 in $E_G(x)$ such that $C(e_1) = C(e_2)$, then $e_1 = e_2$. Before continuing, we would like to point out three key distinctions among our definition and the variations across the literature. First, note that we distinguish between an edge coloring and a proper edge coloring while other papers have inconsistent terminology¹ and often only refer to what we call a proper edge coloring. This is likely in part because the chromatic index and Vizing's theorem itself are defined with respect to proper edge colorings. However, not all supporting definitions and lemmas rely on this property, so we chose to first define (and later implement) a more general edge coloring definition. After writing the code in Rocq, this also allowed us to better understand when the properness is explicitly required and when this implicit assumption

¹It is referred to as an 'edge coloring' by Diestel [35], Stiebitz et al. [36], and Green [34]; as a 'complete proper edge coloring' by Bhoja [39]; and as a 'legal edge coloring' by Barenboim and Elkin [41].

is unnecessary. The second distinction we would like to point out is our actual definition of a proper edge coloring. This definition more commonly adheres to the following structure: for all edges e_1, e_2 in $E(G)$, if there exists a vertex x such that e_1 and e_2 are in $E_G(x)$ (i.e. they are adjacent edges) and $e_1 \neq e_2$ then $C(e_1) \neq C(e_2)$. We instead utilize the contrapositive of this statement, that is, $C(e_1) = C(e_2)$ implies $e_1 = e_2$, because this naturally implies C is injective when constrained to the edges $E_G(x)$ incident to a vertex x ; we are then able to directly apply this in our proof of Vizing’s lower bound. In natural language with typical mathematical assumptions, the proof of the lower bound is nearly immediate and this conclusion is not necessary, but in the Rocq formalization where each step must be justified, this proved rather useful. Finally, we do not include the notion of a partial coloring,² so every edge in the domain of C is mapped to a color in the co-domain. We found we could instead generate new colors to be replaced later as needed, so supporting a partial coloring was unnecessary. In fact, we mimic the properties of a partial edge coloring with our `extendedColType` in the code, elaborated upon in the next chapter.

Now we shall continue with our preliminary definitions. A *k-edge coloring* C_k is a proper edge coloring that uses exactly k colors, so the cardinality of $C_k[E(G)]$ is k . Then the *chromatic index* of a graph G , denoted $\chi'(G)$, is the smallest such k for which a k -edge coloring of G exists. Observe that a k -edge coloring is exactly a proper edge coloring except that we explicitly state the cardinality of the proper edge coloring’s image as the natural number k . That is, we can, for any natural number k , write “ k -edge coloring” instead of the wordier “proper edge coloring C with $C[E(G)] = k$.” We can immediately upper bound $\chi'(G)$ by the total number of edges in a graph G because we can provide an edge coloring which maps each edge to a distinct color. At this point, we have sufficient information to lower bound $\chi'(G)$ by the maximum degree, though we save this for Section 2.3. However, to prove the

²See Misra and Gries[33], Stiebitz et al [36], and even Bhoja’s Lean implementation [39].

upper bound of $\chi'(G)$ as proposed by Vizing, we must define a few auxiliary concepts.

2.2 AUXILIARY DATA STRUCTURES AND LEMMAS

2.2.1 THE ABSENT SET

Given some vertex x in the vertex set $V(G)$ of a graph G and some edge coloring C of this graph, the absent set at x , notated $\mathcal{A}_C(x)$, is the set of colors in color set that are not present on the edges adjacent to vertex x . In other words, we have

$$\mathcal{A}_C(x) := C[E(G)] - C[E_G(x)].$$

where subtraction here refers to set difference. We say a color in $\mathcal{A}_C(x)$ is absent at x . Naturally, for any edge e with a vertex x as an endpoint, the color of edge e cannot be absent at the vertex x . If there are more colors present in the color set than edges at any given vertex, every vertex will have at least one color in its absent set. This follows directly from cardinality arguments, and to explicitly match our implementation in Rocq, we briefly present a lemma with this proof.

Lemma 2.1 (Appendix A.4, lines 280-287). If C_k is a k -edge coloring of a graph G such that $\Delta + 1 \leq k$, then for all vertices x in $V(G)$ there exists some color $c \in C_k[E(G)]$ such that c is absent at x .

Proof. Let G be any graph with a k -edge coloring C_k such that $\Delta + 1 \leq k$ and let x be any vertex in this graph. By definition of the maximum degree, we have $|C_k[E_G(x)]| \leq |C_k[E(G)]| \leq \Delta(G)$. Therefore, we arrive at the inequality

$$\Delta(G) + 1 - \Delta(G) \leq |C_k[E(G)]| - |C_k[E_G(x)]|.$$

Substituting the definition of an absent set, this tell us $1 \leq |\mathcal{A}_{C_k}(x)|$, so there is at least one color absent at x and we are done. $\square$

Observe that no edge can be given some color that is also in the absent set of one of its endpoints. While this is generally straightforward, we present a proof below because it became useful for our implementation (it is used six times) and served as a check on our definition of the absent set.

Lemma 2.2 (A.4, lines 241-250). Let x be any vertex in a graph G and let C be an edge coloring of this graph. For any edge $e \in E_G(x)$ and any color c in $\mathcal{A}_C(x)$ we have $c \neq C(e)$.

Proof. Let e be any edge in $E_G(x)$ and c be any color in $\mathcal{A}_C(x)$. Applying the definition of an absent set, we know $c \in C[E(G)] - C[E_G(x)]$ which implies c must not be in $C[E_G(x)]$. However, we have $e \in E_G(x)$ so $C(e) \in C[E_G(x)]$. Therefore, c cannot equal $C(e)$. $\square$

If we recolor an edge with some color absent at its endpoints, then recoloring preserves the properness of a proper edge coloring. Again, this is an intuitive statement. However, it is key to the final step of Vizing's proof when we remove an extraneous color in the graph while preserving the properness of the coloring, it serves as a stepping stone for more complicated proofs regarding the properness of edge colorings, and the proof grew quickly in the code due to casework involved. Thus, we believe there is value in sharing the reasoning here.

Lemma 2.3 (A.4, lines 402-435). Given any two vertices x, y of some graph G , a proper edge coloring C , and a color c_1 in the color set of C , if c_1 is in the absent set of both x and y then the edge coloring $C[(x, y) \mapsto c_1]$ is proper.

Proof. Let e_1 and e_2 be any two adjacent edges in G . To show $D = C[(x, y) \mapsto c_1]$ is proper, we must show that if $D(e_1) = D(e_2)$ then $e_1 = e_2$. We assume C is a proper edge coloring, so if $C(e_1) = C(e_2)$, then $e_1 = e_2$. We proceed by casework on if e_1 or e_2 is the edge (x, y) .

If neither e_1 nor e_2 are the edge (x, y) , then $D(e_1) = C(e_1)$ and $D(e_2) = C(e_2)$, so we have $D(e_1) = D(e_2)$ only if $C(e_1) = C(e_2)$ and we are done. Therefore, we can assume, without loss of generality, that $e_1 = (x, y)$. Now $D(e_1) = c_1$ and $D(e_1) = D(e_2)$ only if $D(e_2) = c_1$. Clearly, if $e_2 = (x, y) = e_1$ we are done, so we can also assume $e_2 \neq (x, y)$. However, this implies $D(e_2) = C(e_2)$ and we know $C(e_2) \neq c_1$ because c_1 was absent at x and y before the recoloring and e_2 is in $E_G(x)$ or $E_G(y)$. Thus, $D(e_1) \neq D(e_2)$ if $e_1 = (x, y)$ and $e_2 \neq (x, y)$. In all cases, the properness of C is preserved under the recoloring so $D = C[(x, y) \mapsto c_1]$ is a proper edge coloring. $\square$

Now that we have established absent sets and some basic properties, we can define a structure dependent on these sets and critical to reasoning about edge colorings: Fans.

2.2.2 FANS AND ROTATION

Given a vertex x and an edge coloring C , a fan $F_C(x) = \langle w_1, \dots, w_k \rangle$ starting at w_1 is a sequence of vertices satisfying the following:

1. $\langle w_1, \dots, w_k \rangle$ is a non-empty sequence of distinct neighbors of x .
2. $C(x, w_1)$ is not in the set $C[E(G) - \{(x, w_1)\}]$. That is, edge (x, w_1) is the only edge with color $C(x, w_1)$ in all of G .
3. For all $1 \leq i < k$, we have $C(x, w_{i+1}) \in \mathcal{A}_C(w_i)$.

When partial edge colorings are allowed, property 2 states that edge (x, w_1) is uncolored, though this naturally has the same implications as edge (x, w_1) being a distinct color. Note that property 2 implies $C(x, w_1) \in \mathcal{A}_C(y)$ for all vertices y other than x and w_1 . An example fan is shown in Figure 2.1

If there exists some w_{k+1} in the neighborhood of x such that w_{k+1} is not in $F_C(x)$ and $C(x, w_{k+1})$ is absent at w_k , then we can extend $F_C(x)$ with w_{k+1} because $\langle w_1, \dots, w_{k+1} \rangle$ is also a fan. A *maximal fan* is

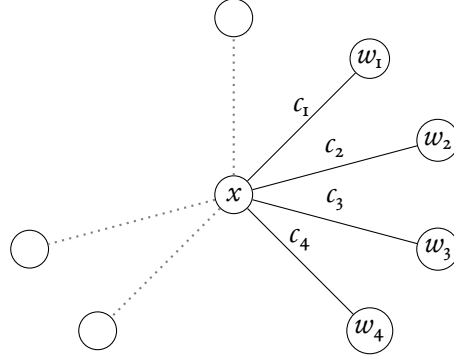


Figure 2.1: A fan $F_C(x) = \langle w_1, w_2, w_3, w_4 \rangle$ centered at x with edge colors labeled c_1, c_2 , etc. Edges connected to x not in $F_C(x)$ are shown in gray (dotted).

a fan that cannot be extended in such a way. As a corollary of this definition, if $F_C(x) = \langle w_1, \dots, w_k \rangle$ is a maximal fan and there exists some vertex w_i in the neighborhood of x such that $C(x, w_i)$ is absent at w_k , then w_i must already be in the fan. Otherwise, we could have extended $F_C(x)$ with w_i , contradicting the assumption that $F_C(x)$ was maximal. We present this result below, and we will later use this in Section 2.3 to split our fan at this vertex w_i .

Lemma 2.4 (A.5, lines 398-413). Let $F_C(x) = \langle w_1, \dots, w_k \rangle$ be a maximal fan centered at a vertex x in some graph G where C is an edge coloring. If there exists some color c such that c is absent at w_k but is not absent at x , then there exists some vertex w_i with $1 \leq i < k$ in the fan such that $C(x, w_i) = c$.

Proof. By assumption, we know $c \in C[E(G)]$ because $c \in \mathcal{A}_C(w_k)$. Since $c \notin \mathcal{A}_C(x) = C[E(G)] - C[E_G(x)]$, it must be the case that $c \in C[E_G(x)]$ and there exists some edge $(x, w_i) \in E_G(x)$ with $C(x, w_i) = c$. Now assume for the sake of contradiction vertex w_i is not in the fan. Then vertex w_i is in the neighborhood of x , vertex w_i is not in $F_C(x)$, and $C(x, w_i) \in \mathcal{A}_C(w_k)$, so we can extend $F_C(x)$ with w_i . However, this contradicts the assumption $F_C(x)$ is a maximal fan. Therefore, the vertex w_i must be in $F_C(x)$. Clearly w_i is distinct from w_k because $C(x, w_i) \in \mathcal{A}_C(w_k)$, so w_i is a vertex in $F_C(x)$ with $1 \leq i < k$ such that $C(x, w_i) = c$. □

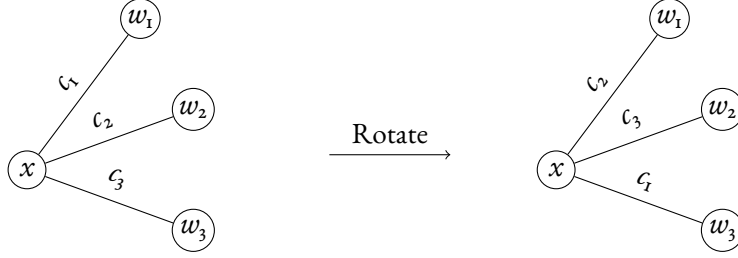


Figure 2.2: A fan $F_C(x) = \langle w_1, w_2, w_3 \rangle$ centered at x with edge colors labeled c_1, c_2, c_3 before and after rotation.

The unique properties of a fan allow us to define a rotation operation which will be fundamental to the proof of Vizing's theorem. When we rotate a fan $F_C(x)$, we create a new edge coloring R such that

$$R(e) = \begin{cases} C(e) & e \notin E_G(x) \\ C(x, w_{i+1}) & e = (x, w_i) \text{ and } 1 \leq i < k \\ C(x, w_1) & e = (x, w_k) \end{cases}$$

In other words, the colors of the edges $(x, w_1), (x, w_2), \dots, (x, w_k)$ are rotated counterclockwise as demonstrated in Figure 2.2.

While the definition of rotation is generally clear enough for standard mathematical proofs, the implementation into computer code requires an explicit algorithm. Bhoja's verified Lean implementation of Vizing's theorem in 2025 [39] also includes an algorithm `rotateFan` which recursively iterates through the fan's vertices, replacing the edge between this vertex and the central fan vertex with the original color of the previously recolored edge. The algorithm begins at the last vertex of the fan and takes in some new color α with which to color the edge in this first iteration. For our Rocq implementation, in order to perform the rotation of a $F_C(x)$ of vertices $\langle w_1, \dots, w_k \rangle$, we constructed a recursive algorithm `Rotate` which iterates over the vertices of this sequence, swapping the colors of (x, w_i) and

(x, w_{i+1}) for $1 \leq i < k$. We found our definition easier to reason about inductively, avoiding the helper color α altogether. We could prove properties about the simpler operation of swapping edge colors in an edge coloring and then directly apply this reasoning to our rotation algorithm. This did require us to reverse the sequence of fan vertices in the code before rotating the colors to ensure the colors were rotated clockwise as shown in Figure 2.2 and not counterclockwise. Algorithm 1 displays the pseudo-code for the rotation operation.

Algorithm 1 Rotation

```

1: function ROTATE( $C, \langle w_1, w_{i+1}, \dots, w_k \rangle, x$ )
2:   if  $i < k$  then
3:      $D \leftarrow \text{SWAP}(C, (x, w_i), (x, w_{i+1}))$ 
4:     return ROTATE( $D, \langle w_{i+1}, \dots, w_k \rangle, x$ )
5:   else
6:     return  $C$ 
7:   end if
8: end function

```

We say the *rotated edge coloring* of a fan $F_C(x) = \langle w_1, \dots, w_k \rangle$, written as $R(F_C(x))$, is the edge coloring returned by $\text{Rotate}(C, \langle w_1, \dots, w_k \rangle, x)$. Note that the rotation algorithm itself does not take in a fan as input; this is meant to reflect our Rocq implementation and is defined in this way to allow recursive calls without requiring the input to be a fan after the first call. In fact, the input will still be a fan, but we must prove this statement before we can use it.

We now make a few helpful observations related to rotation. Most of these will be building up towards Proposition 2.13 at the end of this section. Since the rotation algorithm is repeated swapping edge colors, we find it helpful to first prove a statement about swapping edge colors before proving a similar statement about rotation with induction. In addition to maintaining the readability of our code and making for a cleaner written inductive proof, extracting these color swapping statements as their own

lemmas also supports future applications of the swap operation should one wish to apply it outside of a rotation context. This is particularly applicable if our code becomes integrated with a larger graph theory library.

First, we show that if an edge satisfies property 2 of fans then swapping color with another edge will also swap which edge property 2 applies to.

Lemma 2.5. Let C be an edge coloring of a graph G , and let e_1 and e_2 be two distinct edges in this graph such that edge e_1 is the only edge with color $C(e_1)$ in all of G . If $S = \text{Swap}(C, e_1, e_2)$, then $C(e_1) = S(e_2)$ and edge e_2 is the only edge with color $S(e_2)$ in all of G .

Proof. Our assumption about $C(e_1)$ can be expressed as:

$$C(e_1) \notin C[E(G) - \{e_1\}].$$

Note that, since e_1 and e_2 are distinct, this also implies $C(e_1) \neq C(e_2)$. To reach our conclusion, we must show

$$S(e_2) \notin S[E(G) - \{e_2\}].$$

Only the two edges e_1 and e_2 are affected by the swap, so

$$C[E(G) - \{e_1, e_2\}] = S[E(G) - \{e_1, e_2\}].$$

By definition, we have $C(e_1) = S(e_2)$ and $C(e_2) = S(e_1)$. Putting this together, we get the following set

of equivalent statements.

$$S(e_2) \notin S[E(G) - \{e_2\}]$$

$$S(e_2) \notin S[E(G) - \{e_1, e_2\}] \cup S(e_1)$$

$$C(e_1) \notin C[E(G) - \{e_1, e_2\}] \cup C(e_2)$$

$$C(e_1) \notin C[E(G) - \{e_1, e_2\}] \text{ and } C(e_1) \neq C(e_2)$$

We know both $(e_1) \notin C[E(G) - \{e_1, e_2\}]$ and $C(e_1) \neq C(e_2)$ are true by our assumptions, for $C[E(G) - \{e_1, e_2\}]$ is a subset of $C[E(G) - \{e_1\}]$. Therefore, we have shown $C(e_1) = S(e_2)$ and edge e_2 is the only edge in G with color $S(e_2)$. $\square$

With this result, we may now show that the sequence given as input at each recursive call in `Rotate` is indeed a fan. Again, we generalize away from the rotation algorithm itself, but the result directly applies.

Lemma 2.6. Let C be an edge coloring and let $F_C(x) = \langle w_1, w_2, \dots, w_k \rangle$ be a fan centered at a vertex x in some graph G containing at least two vertices. If $S = \text{Swap}(C, (x, w_1), (x, w_2))$ then $F_S(x) = \langle w_2, w_3, \dots, w_k \rangle$ is a fan.

Proof. To prove $F_S(x)$ is a fan, we will show each of the three properties of fans holds for $F_S(x)$.

(1) No edges were added or removed from G when we swapped the colors of two edges, all vertices in $\langle w_2, w_3, \dots, w_k \rangle$ are in the neighborhood of x . Additionally, the fan $F_C(x)$ contained at least two vertices so the sequence $\langle w_2, w_3, \dots, w_k \rangle$ is non-empty.

(2) Here, we had property 2 from the fan $F_C(x)$, so we can directly apply Lemma 2.5.

(3) The swap does not affect edges aside from (x, w_1) and (x, w_2) , so property 3 is preserved from $F_C(x)$ for vertices $w_3, w_4, \dots, w_k$. Additionally, from $\mathcal{A}_C(w_2)$ to $\mathcal{A}_S(w_2)$, the only color removed was $S(x, w_2) = S(x, w_1)$. Property 2 of fan $F_C(x)$ tells us color $C(x, w_1)$ cannot possibly be the same as $C(x, w_2)$, so we still have $C(x, w_3) = S(x, w_3) \in \mathcal{A}_S(x_2)$. Therefore, property 3 holds for the sequence $\langle w_2, w_3, \dots, w_k \rangle$.

All properties required for a fan apply, so $F_S(x) = \langle w_2, w_3, \dots, w_k \rangle$ is a fan. □

As mentioned, a corollary of the lemma above is that, assuming the initial sequence was a fan, the sequence given as input to `Rotate` will always be a valid fan. If we follow this reasoning to the last call of `Rotate` before the function returns a new edge coloring, this implies the singleton sequence containing the last vertex of the inputted fan is itself a fan in the returned edge coloring. In other words, property 2 applies to this vertex in the resulting edge coloring.

Corollary 2.7 (A.5, lines 283-302). Let C be an edge coloring and let $F_C(x) = \langle w_1, \dots, w_k \rangle$ be a fan centered at a vertex x in some graph G . If $R = R(F_C(x))$ is the rotated edge color of $F_C(x)$, then $C(x, w_1) = R(x, w_k)$ and property 2 of fans holds for edge (x, w_k) under R .

We now would like to show that rotation preserves the properness of a proper edge coloring. The brunt of this work is handed over to supporting lemmas about edge coloring swapping. Most importantly, if the initial edge coloring was proper, then swapping the colors for the edges of two consecutive fan vertices still leaves a proper edge coloring.

Lemma 2.8 (A.4, lines 512-587). Let C be a proper edge coloring, and let $F_C(x) = \langle w_1, \dots, w_k \rangle$ be a fan centered at some vertex x in a graph G . Additionally, let i be some index with $1 \leq i < k$ such that vertices w_i and w_{i+1} are in the fan $F_C(x)$. If $C(x, w_i)$ is absent at w_{i+1} and $C(x, w_{i+1})$ is absent at w_i , then the edge coloring $\text{Swap}(C, (x, w_i), (x, w_{i+1}))$ is proper.

Proof. Let S be the edge coloring $\text{Swap}(C, (x, w_i), (x, w_{i+1}))$. To show S is proper, we must show $S(e_1) = S(e_2)$ implies $e_1 = e_2$ for any adjacent edges e_1 and e_2 in the graph G . The only edges affected by the swap are those in the edge sets $E_G(w_i)$, $E_G(x)$, and $E_G(w_{i+1})$, so we may assume S is proper if e_1 and e_2 are not together in one of these edge sets. Also, at least one of e_1 and e_2 must be edge (x, w_i) or (x, w_{i+1}) because otherwise again neither edge has been affected. Edges e_1 and e_2 as well as vertices w_i and w_{i+1} are symmetrical in our assumptions, so without loss of generality we may assume $e_1 = (x, w_i)$. Now we will do case work on whether $e_2 \in E_G(x)$ or $e_2 \in E_G(w_i)$. For all cases except case 2, we can simply apply the assumption C is a proper edge coloring. Let y represent an arbitrary vertex in $V(G)$.

Case 1. $e_2 \in E_G(x)$

Case 1.1. $e_2 = (x, w_{i+1})$

The edge coloring S is proper in this case:

$$S(x, w_i) = S(x, w_{i+1}) \implies C(x, w_{i+1}) = C(x, w_i) \implies (x, w_{i+1}) = (x, w_i)$$

Case 1.2. $e_2 = (x, y)$

The edge coloring S is proper in this case:

$$S(x, w_i) = S(x, y) \implies C(x, w_{i+1}) = C(x, y) \implies (x, w_{i+1}) = (x, y)$$

Case 2. $e_2 \in E_G(w_i)$

Case 2.1. $e_2 = (w_i, w_{i+1})$

The edge coloring S is proper in this case:

$$S(x, w_i) = S(w_i, w_{i+1}) \implies C(x, w_{i+1}) = C(w_i, w_{i+1}) \implies (x, w_{i+1}) = (w_i, w_{i+1})$$

Case 2.2. $e_2 = (w_i, y)$

Applying the definition of S , we conclude if $S(x, w_i) = S(w_i, y)$ then it must be the case $C(x, w_{i+1}) = C(w_i, y)$. However, edge (w_i, y) is in $E_G(w_i)$ and we know by assumption $C(x, w_{i+1})$ is absent at w_i . According to Lemma 2.2, the color $C(x, w_{i+1})$ of edge (x, w_{i+1}) cannot equal $C(w_i, y)$. As such, we have a contradiction and $S(x, w_i) \neq S(w_i, y)$. Then the requirements of a proper edge coloring do not apply to these two edges, so S is proper in this case.

We have shown that S is a proper edge coloring in all possible cases, so we may conclude that $S = \text{Swap}(C, (x, w_i), (x, w_{i+1}))$ is a proper edge coloring given the lemma's assumptions. $\square$

With this helper lemma done, we may now show rotation preserves the properness of a proper edge coloring. Within this proof, we have a few additional asides related to the swap operation. These are short and naturally fit within the written proof here, but in Rocq we also pull them out as their own lemmas.

Proposition 2.9 (A.5, lines 342-362). Let C be an edge coloring and let $F_C(x) = \langle w_1, \dots, w_k \rangle$ be a fan centered at a vertex x in some graph G . If C is proper, then the rotated edge coloring of $F_C(x)$ is also proper.

Proof. We will proceed via strong induction on k . If $k = 1$ then $F_C(x)$ contains only one vertex, so $R(F_C(x)) = C$ and this case is trivial. If $k = 2$, then $R(F_C(x)) = \text{Swap}(C, (x, w_1), (x, w_2))$. We know

$C(x, w_1) \in \mathcal{A}_C(w_2)$ and $C(x, w_2) \in \mathcal{A}_C(w_1)$ by property 2 and property 3 of fans, respectively. Thus, Lemma 2.8 tells us the rotated edge coloring $R(F_C(x))$ of $F_C(x)$ is proper.

Let k be an arbitrary natural number. For the inductive hypothesis, assume that for all proper edge colorings C , vertices x , and sequences $fs = \langle w_1, \dots, w_k \rangle$ such that $F_C(x) = fs$ is a fan, the rotated edge coloring $R(F_C(x))$ is proper. Then we need to show that for all proper edge colorings D , vertices y , and sequences $gs = \langle u_1, \dots, u_k, u_{k+1} \rangle$ such that $F_D(y) = gs$ is a fan, the edge coloring $R(F_D(y))$ is proper.

Since $1 < 2 < k + 1$, we know we are within the if-statement during the rotation function's first iteration, so we can rewrite $R(F_D(y)) = \text{Rotate}(D', gs', y)$ where $D' = \text{Swap}(D, (y, u_1), (y, u_2))$ and $gs' = \langle u_2, \dots, u_{k+1} \rangle$. By Lemma 2.6, we know $F_{D'}(y) = gs'$ is a fan. If we can show D' is a proper edge coloring, then we can apply our inductive hypothesis.

According to property 2 and property 3 of fan $F_D(y) = gs$, respectively, we know $D(y, u_1)$ is absent at u_2 and $D(y, u_2)$ is absent at u_1 . Thus, by Lemma 2.8, the edge coloring $D' = \text{Swap}(D, (y, u_1), (y, u_2))$ must be proper. Now we may use the inductive hypothesis to assert that $R(F_{D'}(y)) = R(F_D(y))$ is a proper edge coloring.

By induction, we can conclude for all proper edge colorings C , vertices x , and fans $F_C(x)$, the rotated edge coloring of $F_C(x)$ is proper. □

Naturally, we are not adding or removing colors throughout the rotation process, so the color set is invariant under rotation. Because we chose not to include the auxiliary color as in Bhoja's algorithm, this result is almost immediate. Originally, we did not even have this as a lemma. Yet once we began writing our code, we realized we would need a more substantial argument to support the claim (as opposed to a singular sentence). The proof required induction, even if it was short. As such, we present the reasoning here.

Lemma 2.10 (A.5, lines 234-244). Let C be an edge coloring and let $F_C(x)$ be a fan centered at a vertex x in some graph G . Then the color set of C equals the color set of the rotated edge coloring of $F_C(x)$.

Proof. At each step in the `Rotate` algorithm, we modify the given edge coloring by swapping the colors of two edges. Since swapping edge colors does not add or remove colors from the range of an edge coloring, the range of the edge coloring is invariant at each iteration of the rotation algorithm. By induction on the size of our fan, the color set of C is invariant under rotation and thus is equivalent to the color set of $R(F_C(x))$. $\square$

We proved Lemma 2.11 in `Rocq` with similar logic. It was required because it did not immediately follow Lemma 2.10; in fact, it would be a false statement for any vertex w which is contained in the rotated fan when this fan includes at least two vertices. For instance, in Figure 2.2, we have $c_1 \in C[E_G(w_1)]$ before the rotation, and this is not the case after rotation.

Lemma 2.11 (A.5, lines 220-231). Let C be an edge coloring and let $F_C(x)$ be a fan centered at a vertex x in some graph G . Let $R = R(F_C(x))$ be the rotated edge coloring at $F_C(x)$. Then we have $R[E_G(x)] = C[E_G(x)]$.

Proof. The set of colors at edges adjacent to x is invariant at each iteration of the `Rotate` algorithm. For an arbitrary step in the recursive calls, the edge coloring is only modifying by swapping the colors of two edges adjacent to x . These two colors are present on edges adjacent to x both before and after the swap, so the set of colors at edges adjacent to x is unaltered. By induction on the size of our fan, the set of colors at edges adjacent to x is invariant under rotation and $R[E_G(x)] = C[E_G(x)]$. $\square$

From these two lemmas on the color set of R , we can conclude $\mathcal{A}_C(x) = \mathcal{A}_R(x)$. This next lemma extends this to derive information about the absent set of vertices in the fan. The Mathematical Compo-

nents library in Rocq [29] includes a well-developed section on finite sets, so this proof was reasonably straightforward to formalize once we familiarized ourselves with it.

Lemma 2.12 (A.5, lines 314-340). Let C be an edge coloring and let $F_C(x)$ be a fan centered at a vertex x in some graph G . Let $R = R(F_C(x))$ be the rotated edge coloring at $F_C(x)$. Suppose w is any vertex in $F_C(x)$. If there exists some color c_1 such that $c_1 \in \mathcal{A}_C(x) \cap \mathcal{A}_C(w)$ then $c_1 \in \mathcal{A}_R(x) \cap \mathcal{A}_R(w)$.

Proof. Expanding the definition of an absent set and applying Lemmas 2.10 and Lemma 2.11, we have

$$\begin{aligned}\mathcal{A}_C(x) &= (C[E(G)] - C[E_G(x)]) \\ &= R[E(G)] - R[E_G(x)] \\ &= \mathcal{A}_R(x)\end{aligned}$$

This in turn tells us $c_1 \in \mathcal{A}_R(x) \cap \mathcal{A}_R(w)$ only if $c_1 \in \mathcal{A}_R(w)$. Since again we know $C[E(G)] = R[E(G)]$, it is sufficient to show $c_1 \notin R[E_G(w)]$. The only edge in $E_G(w)$ which changes colors between edge colorings C and R is the edge (x, w) . In other words, we can say the set $R[E_G(w) - \{(x, w)\}]$ with edge (x, w) removed is equivalent to $C[E_G(w) - \{(x, w)\}]$. Our assumption $c_1 \in \mathcal{A}_C(w)$ implies c_1 is not in $C[E_G(w)]$. Merging these two statements, we know $c_1 \notin R[E_G(w) - \{(x, w)\}]$ and as such $c_1 \in R[E_G(w)]$ only if $c_1 = R(x, w)$. However, we have already shown c_1 is in $\mathcal{A}_R(x)$. Clearly, edge (x, w) is in $E_G(x)$, so by Lemma 2.2 we know c_1 cannot be $R(x, w)$. As such, the color c_1 cannot be contained in $R[E_G(w)]$ and we are done. $\square$

Finally, we merge the concept of fans, recoloring, and absent sets to produce a $(\Delta(G) + 1)$ -edge coloring of a graph G . When we do case work in our proof of Vizing's theorem, this lemma will allow us to conclude the two main branches. In essence, if we have a fan $F_C(x) = \langle w_1, \dots, w_k \rangle$ and there is

some color c_j absent at both x and w_k , then we can rotate the fan and recolor the last edge (which is a unique color according to Lemma 2.7) with c_j to decrease the total number of colors in the color set. We use the subscript j for the last vertex of our fan rather than k to emphasize that this fan may be a sub-fan of some other fan; it may itself not be a maximal fan.

Proposition 2.13 (A.8, lines 13-47). Let G be any graph such that there exists a $(\Delta(G) + 2)$ -edge coloring C of G and a fan $F_C(x) = \langle w_1, \dots, w_j \rangle$. If there exists some $c_j \in C[E(G)]$ such that $c_j \in (\mathcal{A}_C(x) \cap \mathcal{A}_C(w_j))$ then we can produce a $(\Delta(G) + 1)$ -edge coloring of G .

Proof. Observe that (x, w_j) is an edge in $E(G)$ because, by property 1 of fans, vertex w_j is in the neighborhood of x . We will create an edge coloring R' by rotating the fan and then recoloring the last edge (x, w_j) . To show R' is a $(\Delta(G) + 1)$ -edge coloring of G , we must prove it is proper and it uses exactly $(\Delta(G) + 1)$ colors.

Let $R = R(F_C(x))$ be the rotated edge coloring of $F_C(x)$. By Lemma 2.9, we know R is a proper edge coloring. From Lemma 2.10 we have $|R[E(G)]| = |C[E(G)]| = \Delta(G) + 2$. Therefore, R is a $(\Delta(G) + 2)$ -edge coloring of G . Now, let $R' = R[(x, w_j) \mapsto c_j]$ be the coloring R with edge (x, w_j) recolored as c_j . From our assumptions, we have $c_j \in (\mathcal{A}_C(x) \cap \mathcal{A}_C(w_j))$. By Lemma 2.12 and Lemma 2.3, this implies $c_j \in (\mathcal{A}_R(x) \cap \mathcal{A}_R(w_j))$ and R' is a proper edge coloring. Recall that by property 2, color $C(x, w_1)$ was distinct in G under C . After the fan is rotated, we have $R(x, w_j) = C(x, w_1)$ and (x, w_j) is the only edge with color $R(x, w_j)$ in all of G by Lemma 2.7. With the edge coloring R' , we replaced the only appearance of color $C(x, w_1) = R(x, w_j)$ with c_j , thus removing it from the color set. Clearly, we already have $c_j \in R'[E(G) - (x, w_j)]$, for c_j , which we assumed existed in $C[E(G)]$, was not present on the edges adjacent to x in the fan, and as such was not affected by the rotation or recoloring operation. Hence, by recoloring edge (x, w_j) , we have removed one color without using any new colors, so $|R'[E(G)]| =$

$|R[E(G)]| - 1 = \Delta(G) + 1$ and we can conclude R' is a $(\Delta(G) + 1)$ -edge coloring of G . □

2.2.3 KEMPE CHAINS

We considered two main viable definitions of a Kempe chain. The first was a maximal path that alternates in color, and Bhoja's implementation in Lean followed this approach. After considering the strengths of Rocq's graph theory library, we decided to pursue alternative definitions and came across a subgraph definition in Scheide et al. [36]. Inspired, this served as our main point of reference as we implemented this new representation. Since we had already explored the former definition, our final proof in Section 2.3 still shares many ideas with Misra and Grieg's algorithm.

Suppose C is an edge coloring of a graph G and c_1 and c_2 are any two colors. Then let us consider the subgraph H of G including exactly the edges colored c_1 or c_2 that is $V(H) = V(G)$ and $E(H) = \{e \in E(G) \mid C(e) = c_1 \text{ or } C(e) = c_2\}$. A *Kempe chain* is a connected component of this subgraph H containing some specified vertex v and is denoted $K_C(v, c_1, c_2)$. When it is clear from context, we will shorten $K_C(v, c_1, c_2)$ to K_C . An example Kempe chain is shown in Figure 2.3. When C is a proper edge coloring,

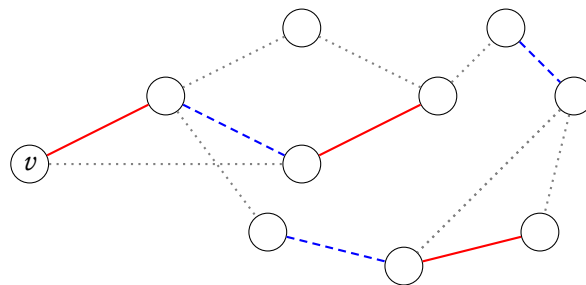


Figure 2.3: The subgraph H induced by edges colored red (solid) and blue (dashed). Each vertex is in a component; two vertices connected by red and blue edges are in the same component. The component containing v is the Kempe chain $K_C(v, \text{red}, \text{blue})$. Edges shown in gray (dotted) are in $E(G)$ but not in $E(H)$.

every vertex in G is adjacent to at most one edge colored c_1 and one edge colored c_2 , so every vertex in H is at most degree two and every Kempe chain is either a cycle or a path. Scheide et al. [36] state this as

fact. However, this core idea is non-trivial, especially the jump to the implicit assumption that a chain can only have two endpoints. There were many potential approaches to support this proposition in Rocq. Due to the minimal number of lemmas relating cycles, paths, and vertex degree in the existing library, we decided to tailor the final Proposition 2.18 to our specific needs rather than be more general in hopes of limiting the proof length and amount of auxiliary lemmas. We do prove Lemma 2.14, Proposition 2.15, and a few other results in `spath.v` that could be used outside of a Kempe graph context, and these could be also be written in terms of paths and cycles, trees, or $K_{1,3}$ subdivisions and extending this portion of the library could be a direction for future work.

If we have some path $P = \langle x_1, x_2, \dots, x_n \rangle$ from some vertex x_1 to some vertex x_n , then we refer to $\langle x_2, \dots, x_{n-1} \rangle$ as the *interior* of the path. Note that if x_1 and x_n are adjacent then it is possible the interior of P is empty. Lemma 2.14 says that interior vertices must have degree at least two, and Proposition 2.15 then concludes there must exist some interior vertex with degree at least three if we have two connected paths with shared interior vertices.

Lemma 2.14 (A.3, lines 36-46). Let P be a path from some vertex x_1 to some vertex x_n in a graph G . If y is some vertex in G such that $d(y) \leq 1$, then y is not in the interior of P .

Proof. Consider any vertex x_i in the interior of P . Then there exists some vertices x_{i-1} and x_{i+1} in $\langle x_1, x_2, \dots, x_{n-1}, x_n \rangle$. By definition of a path, there also exists an edge joining x_{i-1} and x_i in G and a different edge joining x_i and x_{i+1} in G . Then $2 \leq |E_G(x_i)| = d(x_i)$. Since we assumed $d(y) \leq 1$, we must have $y \neq x_i$ for any vertex x_i in the interior of P . Therefore, if y is a vertex with $d(y) \leq 1$ then y cannot be in the interior of a path. □

To assist in understanding the proof of Proposition 2.15, Figure 2.4 displays four possible ways two paths with the same starting vertex and different ending vertices could interact. Of course, there are

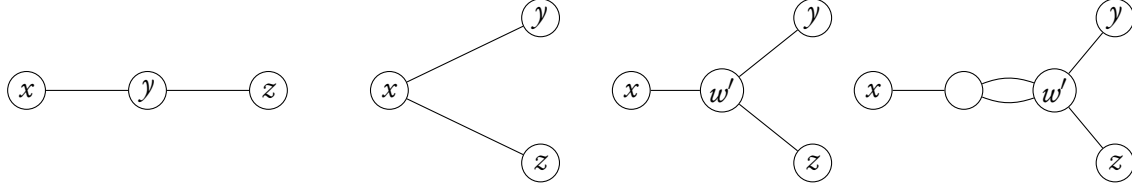


Figure 2.4: Four possible ways two paths with the same starting vertex could appear in a graph.

other possibilities not shown here. For instance, the last example shows the paths diverging and reuniting once, but this could happen any number of times. However, considering just these four cases is enough to prove the proposition, for the logic generalizes to all other cases. For instance, the same logic applies for paths diverging once or twice or thrice, etc., because we only rely on the last vertex w' of divergence. Thus, Proposition 2.15 will proceed with these four possibilities in mind. We will first show that neither path is contained within the other, as shown in the first example, by applying Lemma 2.14. We then argue there must exist at least one shared internal vertex, eliminating the second possibility. Finally, we recognize that all other possibilities include a vertex of degree three to finish the proof.

Proposition 2.15 (A.3, lines 50-134). Let P be a path from x to y , and let Q be a path from x to z where $x \neq y \neq z$. If $d(x) = d(y) = d(z) = 1$, then there exists some vertex with degree greater than 3.

Proof. Lemma 2.14 implies vertex y cannot be in the interior of path Q and vertex z cannot be in the interior of path P . That is, P is not entirely contained within Q and vice versa. Since $d(x) = 1$, there is exactly one edge (x, w) at x where w is some vertex in $V(G)$. This vertex w must be distinct from y and z , else we would have $y = z$ or one of P and Q would be contained within the other. In order for x to be connected to y and z through paths P and Q , both paths must include edge (x, w) . Thus, we can split these paths at w such that P' is the path from w to y and Q' is the path from w to z . Again, the paths P' and Q' are not equal nor contained within each other, so there must exist some vertex w' in both P' and Q' where they diverge. That is, there is a path from R from x to w' (containing w) and there

are two disjoint paths P'' from w' to y and Q'' from w' to z . Note it is possible P' and Q' diverge and reconnect multiple times. However, there must exist some last divergent vertex at which the paths do not reconnect (since $y \neq z$); we let w' be this last vertex. It is also possible $w = w'$, but our argument still applies if this is the case.

We wish to show w' has degree at least 3. To do so, we prove there is at least one edge in each of R , P'' , and Q'' . By construction, these three paths are non-overlapping, so their edges are distinct. By construction, vertex w' is shared by P'' and Q'' , so it must be distinct from vertices y and z . Then in order for P'' and Q'' to be connected, there must exist at least one edge in each path. Vertex w' is connected to each of these edges since it is in both paths; thus, we have two distinct edges with an endpoint at w' . Additionally, we know that at least one edge (x, w) is in our path R , and w' must be connected to this edge. R is disjoint from P'' and Q'' , so this gives us a third unique edge with an endpoint at w' . Hence, there are at least three edges in $E_G(w')$ and we conclude w' had degree at least 3. $\square$

Before we apply Proposition 2.15 to our Kempe chains, we must state some natural conclusions about the degree of vertices in the Kempe chains. First, we show that if there are two vertices in $V(K_C)$ then these must have positive degree because they are connected.

Lemma 2.16 (A.6, lines 201-217). Let $K_C(v, c_1, c_2)$ be to a Kempe chain in a graph G where C is an edge coloring, vertex v is in G , and c_1 and c_2 are colors. If x and y are two distinct vertices in $V(K_C)$, then both $d(x)$ and $d(y)$ are greater than zero.

Proof. Components are connected subgraphs, so we know K_C is connected and as such there exists some path P from x to y in K_C . Since $x \neq y$, we can split P as P_1 and P_2 where P_1 is the singleton path containing x . Paths P_1 and P_2 must be connected via some edge in $E(K_C)$ in order for P itself to be a connected path. This edge has an endpoint in P_1 , so it must have an endpoint at x , the only vertex in P_1 .

Therefore, there is at least one edge at x and $0 < d_{K_C}(x)$. We could have instead split the path on the right with P_2 as the singleton path containing y , so by symmetry $0 < d_{K_C}(y)$. $\square$

Vertices adjacent to only one edge colored c_1 or c_2 must be endpoints in their respective Kempe chains; this will later allow us to apply Proposition 2.15 if there are three ‘endpoint’ vertices in a single chain.

Lemma 2.17 (A.6, lines 271-288). Let $K_C(v, c_1, c_2)$ be to a Kempe chain in a graph G where C is a proper edge coloring, vertex v is in G , and c_1 and c_2 are colors. Let x be a vertex in K_C . If either c_1 or c_2 is in $\mathcal{A}_C(x)$, then $d_{K_C}(x) \leq 1$.

Proof. Since C is by definition injective when constrained to the domain of $E_G(x)$, it is also injective when constrained to the domain of $E_{K_C}(x)$ because $E_{K_C}(x)$ is a subset of $E_G(x)$. Now, by definition $d_{K_C}(x) = |E_{K_C}(x)|$ and by injectivity $|E_{K_C}(x)| = |C[E_{K_C}(x)]|$. The edges of K_C are exactly those colored c_1 and c_2 in G under C , so we have $C[E_{K_C}(x)] \subseteq \{c_1, c_2\}$.

We know by assumption either c_1 or c_2 are in $\mathcal{A}_C(x)$. Without loss of generality, suppose c_1 is in $\mathcal{A}_C(x)$. Lemma 2.2 tells us $c_1 \neq C(e)$ for all $e \in E_{K_C}(x) \subseteq E_G(x)$. Thus, the color c_1 is not in $C[E_{K_C}(x)]$ and we conclude

$$d_{K_C}(x) = |E_{K_C}(x)| = |C[E_{K_C}(x)]| \leq |\{c_2\}| = 1.$$

$\square$

With these assisting lemmas, the proof of Proposition 2.18 can be relatively short.

Proposition 2.18 (A.6, lines 290-317). Let $K_C(v, c_1, c_2)$ be to a Kempe chain in a graph G where C is a proper edge coloring, vertex v is in G , and c_1 and c_2 are colors. Suppose x, y , and z are three distinct

vertices such that at each vertex x, y and z , either c_1 or c_2 is absent at the vertex. Then at most two of x, y , and z are in $V(K_C)$.

Proof. Suppose for the sake of contradiction that x, y , and z are in $K_C(v, c_1, c_2)$. Our goal will be to show Proposition 2.15 forces us to arrive at a contradiction. Sandwiched by Lemma 2.16 and Lemma 2.17, we have $d_{K_C}(x) = d_{K_C}(y) = d_{K_C}(z) = 1$. Because Kempe chains are connected components, there exists some path P connecting x to y in K_C and some path Q connecting x to z in K_C . Therefore, there exists some vertex $w \in V(K_C)$ with $3 \leq d_{K_C}(w)$ by Proposition 2.15. However, we know

$$d_{K_C}(w) = |E_{K_C}(w)| = |C[E_{K_C}(w)]| \leq |\{c_1, c_2\}| = 2$$

by the same reasoning as in the previous lemma. Now we have $3 \leq d_{K_C}(w) \leq 2$, a contradiction; hence, it is impossible for all three of x, y , and z to be in $V(K_C)$. $\square$

Like rotation with the fans, we define a recoloring operation on the Kempe chain structure that will preserve the properness of a proper edge coloring. In this case, that operation is a Kempe swap. A *Kempe swap* consists in swapping the two colors along a given Kempe chain. That is, given a Kempe chain $K_C(v, c_1, c_2)$, we will invert the edge coloring for edges within this component, recoloring the edges c_1 with c_2 and vice versa. To not be conflated with the *Swap* edge coloring, we will refer to this new coloring I as the *inverted edge coloring* of $K_C(v, c_1, c_2)$. It is defined as follows:

$$I(e) = \begin{cases} C(e) & e \notin E(K_C) \\ c_2 & e \in E(K_C) \text{ and } C(e) = c_1 \\ c_1 & e \in E(K_C) \text{ and } C(e) = c_2 \end{cases}$$

In our proof of Vizing's theorem, we will construct a Kempe chain starting at the central vertex of a fan. Thus, we show that, given some assumptions, there is still a fan present at this central vertex after we perform a Kempe swap.

Proposition 2.19 (A.6, lines 325-411). Let C be a proper edge coloring, and let I be the inverted edge coloring of a Kempe chain $K_C(v, c_1, c_2)$ where v is some vertex and c_1 and c_2 are colors. Let $F_C(x) = \langle w_1, \dots, w_k \rangle$ be a fan centered at a vertex x . Note that x and v may or may not be the same vertex. If c_2 is in $\mathcal{A}_C(x)$ and there exists some $i \in [2, k]$ such that $C(x, w_i) = c_1$, then $F_I(x) = \langle w_1, \dots, w_{i-1} \rangle$ is a fan.

Proof. We will show the three properties of a fan hold for $F_I(x) = \langle w_1, \dots, w_{i-1} \rangle$.

(1) Clearly, property 1 holds for $F_I(x)$ because all vertices in $F_I(x)$ are also in $F_C(x)$ and we did not add or remove edges of our graph G .

(2) Since $F_C(x)$ is a fan, property 2 tells us edge (x, w_1) is the only edge in the graph G with color $C(x, w_1)$. We will now show property 2 of fans holds after a Kempe swap of chain K_C . Under coloring C , we know the color c_2 is absent at x , so edge (x, w_1) is not colored c_2 . Additionally, C is a proper edge coloring and edge (x, w_i) is already colored c_1 under C where $1 < i$, so we know (x, w_1) cannot be colored c_1 . The Kempe chain only includes edges colored c_1 and c_2 , and the inversion only swaps these two colors. Therefore, edge (x, w_1) is still the only edge in G with color $I(x, w_1) = C(x, w_1)$ after the Kempe swap and property 2 of our fan is preserved.

(3) Finally, we need to show $I(x, w_{j+1}) \in \mathcal{A}_I(w_j)$ for all $1 \leq j < i - 1$. Aside from colors c_1 and c_2 , the colors absent at any given vertex are invariant under a Kempe swap. We know that edge (x, w_i) is the only edge at x with the color c_2 because C is a proper edge coloring, and we know no edge at x is colored c_2 under C because $c_2 \in \mathcal{A}_C(x)$. Hence, we have $C(x, w_{j+1}) = I(x, w_{j+1})$ and $I(x, w_{j+1}) \in \mathcal{A}_I(w_j)$ for all $1 \leq j < i - 1$. This is what we wanted to show, so property 3 is preserved on this sub-fan $F_I(x)$.

We have now proven all properties of a fan hold for $F_I(x)$. Therefore, we conclude $F_I(x) = \langle w_1, \dots, w_{i-1} \rangle$ is a fan. □

Finally, we will later perform a Kempe swap on a graph with a k -edge coloring, so we will need to show that after the inversion there still exists proper edge coloring of the graph that uses at most k colors. Thus, we first show properness is preserved by Kempe swaps.

Lemma 2.20 (A.6, lines 219-262). Let C be an edge coloring, and let I be the inverted edge coloring of a Kempe chain $K_C(v, c_1, c_2)$ where v is some vertex and c_1 and c_2 are colors. If C is proper, then the inverted edge coloring I is also proper.

Proof. Let e_1 and e_2 be any two adjacent edges in $E(G)$. To show I is proper in G , we must show that if $I(e_1) = I(e_2)$ then $e_1 = e_2$. We assumed C is a proper edge coloring so if $C(e_1) = C(e_2)$ then $e_1 = e_2$. Now we may proceed by casework, inspired by the definition of I , on whether e_1 and e_2 are in the subgraph K_C .

Case 1. $e_1 \notin E(K_C)$ and $e_2 \notin E(K_C)$

This is a rather simple case because $I(e_1) = C(e_1)$ and $I(e_2) = C(e_2)$. If $I(e_1) = I(e_2)$ then $C(e_1) = C(e_2)$, so $e_1 = e_2$ and we are done.

Case 2. $e_1 \notin E(K_C)$ and $e_2 \in E(K_C)$

We know that all edges e in $E(G)$ colored c_1 or c_2 under C are in K_C . Therefore, we know that $I(e_1) = C(e_1)$ is not c_1 or c_2 . Meanwhile, $I(e_2)$ by definition can only be colored c_1 or c_2 since e_2 is in $E(K_C)$. This implies $I(e_1)$ cannot possibly equal $I(e_2)$, and we are done.

Case 3. $e_1 \in E(K_C)$ and $e_2 \notin E(K_C)$

This is symmetric to Case 2.

Case 4. $e_1 \in E(K_C)$ and $e_2 \in E(K_C)$

All edges in $E(K_C)$ must be colored either c_1 or c_2 under C by construction. If edges e_1 and e_2 are the same color under C , then $I(e_1) = I(e_2)$ and the properness of C implies $e_1 = e_2$ so we are done. Thus, we may assume that $C(e_1) \neq C(e_2)$ and $c_1 \neq c_2$. By symmetry, we can assume $C(e_1) = c_1$ and $C(e_2) = c_2$. Then $I(e_1) = c_2$ and $I(e_2) = c_1$. Since $I(e_1) \neq I(e_2)$, we do not need to continue.

In all cases, we have shown that $I(e_1) = I(e_2)$ implies $e_1 = e_2$ when C is a proper edge coloring, so I is a proper edge coloring of G when C is a proper edge coloring of G . $\square$

We conclude this section with a short proof that the color set after the Kempe swap is a subset of the color set before the inversion. The algorithm presented by Misra and Gries [33] does not address the possibility that either c_1 or c_2 may have been removed from the color set altogether. The aim of both our proof of Vizing's theorem and theirs is to demonstrate we can decrease the total number of colors used given any proper edge coloring with more than $\Delta + 1$ colors in its color set. We both recognize that inversion will not add any colors since c_1 and c_2 were already present in the color set, so their result is certainly still valid. We separate the proof into cases based on whether or not the size of the color set has decreased or remained the same to assist our implementation. The constructive nature of Rocq encourages the explicit construction of a k -edge coloring to show a graph is at worst $(\Delta + 1)$ -edge colorable, and this means we must specify k . Thus, we prefer split the inequality and work with an equality instead (this also lets us directly use the `rewrite` tactic). Luckily, when the inversion decreases the number of colors in the color set, we can already conclude our proof and this case is quickly resolved.

Lemma 2.21 (A.6, lines 105-118). Let C be an edge coloring, and let I be the inverted edge coloring of a Kempe chain $K_C(v, c_1, c_2)$ where v is some vertex and c_1 and c_2 are colors. If c_1 and c_2 were present in the color set of C , then the color set of I is a (not necessarily strict) subset of the color set of C .

Proof. It is possible the color set $I[E(G)]$ of I is smaller than the color set $C[E(G)]$ of C if we inverted the only edge colored c_1 or c_2 under C . However, the only colors we are reassigning to edges during the inversion are c_1 and c_2 . Since these were already present in the color set of C , there are no colors in $I[E(G)]$ that are not already in $C[E(G)]$. Thus, we conclude $I[E(G)] \subseteq C[E(G)]$. $\square$

2.3 MAIN PROOF

Many proofs³ tend to focus on the upper bound in Vizing's theorem, omitting discussion of the lower bound as it is rather straightforward. However, this is not trivially solved in Rocq, i.e. it has non-trivial steps and assumptions, so we include an explicit proof of both the lower and upper bounds here. For ease, we replicate the statement of Theorem 1.2 here.

Theorem 2.22 (A.8, lines 49-55 and 175-267). For any finite, simple graph G , we have $\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1$

We first prove the lower bound $\Delta(G) \leq \chi'(G)$. If the chromatic index is some natural number $\chi'(G)$, there must exist at least one $\chi'(G)$ -edge coloring of G . We will refer to this coloring as C . Observe that, for all x in $V(G)$, we know that $\chi'(G) = |C[E(G)]|$ must be greater than or equal to $|C[E_G(x)]|$ because $E_G(x)$ is a subset of $E(G)$. Additionally, C is by definition injective when constrained to the domain $E_G(x)$, so we must use $E_G(x)$ distinct colors on the edges at x in order for C to be a proper edge coloring. Then, letting y be a vertex of G such that $d(y) = \Delta(G)$, we can conclude

$$\Delta(G) = d(y) = |E_G(y)| = |C[E_G(y)]| \leq |C[E_G(G)]| = \chi'(G).$$

We will proceed to prove the upper bound via induction on the number of edges m of G . For the

³See, for instance, the proofs by Diestel [35] and Misra and Gries [33]

base case when $m = 0$, the maximum degree of G is trivially 0 and any proper edge coloring of G requires 0 colors so

$$\chi'(G) = 0 < \Delta(G) + 1.$$

For the inductive step, assume the hypothesis holds for $m = k$ where k is some natural number. Then we need to show it holds for $k + 1$.

Let G be any graph with $k + 1$ edges. Define G' to be the graph G with some arbitrary edge (x, w_1) removed. Then the graph G' has k edges, so by our inductive hypothesis there exists some $\chi'(G')$ -edge coloring of G' such that $\chi'(G') \leq \Delta(G') + 1$. Now let us add back edge (x, w_1) .

If we select some new color with which to color (x, w_1) , then this naturally extends to a $(\chi'(G') + 1)$ -edge coloring of G ; we will refer to this coloring as C . Note that we have only removed edges and as such the maximum degree of G' is less than or equal to the maximum degree of G . Therefore, by transitivity we have $\chi'(G') \leq \Delta(G') + 1 \leq \Delta(G) + 1$. Furthermore, if $\chi'(G')$ is strictly less than $\Delta(G) + 1$ then our coloring C shows that $\chi'(G) \leq \Delta(G) + 1$ and we are done. Hence, going forward we assume $\chi'(G')$ is exactly equal to $\Delta(G) + 1$ and C is a $(\Delta(G) + 2)$ -edge coloring. Note that Lemma 2.1 now tells us there is at least one color absent at each vertex in G' . These colors remain absent at their respective vertices even when we add back edge (x, w_1) because we do not color the new edge with any colors already present in the color set. Additionally, only the edge (x, w_1) has the color $C(x, w_1)$ by construction, so let $F_C(x) = \langle w_1, \dots, w_k \rangle$ be a maximal fan starting at w_1 . As mentioned, some color c_k , which is not the new color $C(x, w_1)$, is absent at w_k . We may do casework on whether or not c_k is also absent at x , the central vertex of our fan.

Case 1. $c_k \in A_C(x)$

In this case, we can directly apply Proposition 2.13 because $c_k \in A_C(x) \cap A_C(w_k)$, so there exists a

$(\Delta(G) + 1)$ -edge coloring of G . This implies $\chi'(G) \leq \Delta(G) + 1$ and we are done.

Case 2. $c_k \notin \mathcal{A}_C(x)$

For this case, we will need to create a Kempe chain $K_C(x, c_x, c_k)$ where c_x is a color absent at x . Let I be the inverted edge coloring of K_C . We know I is a proper edge coloring by Lemma 2.20, and we know c_x and c_k were in the color set $C[E(G)]$ so the color set $I[E(G)]$ is a subset of $C[E(G)]$ by Lemma 2.21. If $I[E(G)]$ is a proper subset of $C[E(G)]$, then $|I[E(G)]|$ is strictly less than $|C[E(G)]|$, so we have

$$\chi'(G) \leq |I[E(G)]| \leq \Delta(G) + 1 < \Delta(G) + 2 = |C[E(G)]|$$

and we are done. Thus, we may assume $I[E(G)] = C[E(G)]$.

Lemma 2.4 tells us there exists some vertex w_i in $F_C(x)$ such that $C(x, w_i) = c_k$. Edge (x, w_i) is contained within our chain K_C because this edge is connected to x and colored c_k . Recall that the color $C(x, w_1)$ is not c_k , implying $w_i \neq w_1$ and w_{i-1} is a valid vertex in the fan $F_C(x)$ with $c_k \in \mathcal{A}_C(w_{i-1})$ from property 3 of fans. According to Proposition 2.19, we know $F_I(x) = \langle w_1, \dots, w_{i-1} \rangle$ is a fan. Do casework on whether or not w_{i-1} is in K_C .

Case 2.1. $w_{i-1} \in K_C(x, c_x, c_k)$

Recall c_x is in $\mathcal{A}_C(x)$ and c_k is in both $\mathcal{A}_C(w_{i-1})$ and $\mathcal{A}_C(w_k)$. Proposition 2.18 tells us at most two of these vertices can be in the Kempe chain, and we know both x and w_{i-1} are. Hence, we know w_k cannot be in K_C . The colors c_x and c_k are only inverted at edges with endpoints in the chain. As such, we know $c_k \in \mathcal{A}_I(x)$ and $c_k \in \mathcal{A}_I(w_k) = \mathcal{A}_C(w_k)$.

We would now like to apply Proposition 2.13. Since we know $c_k \in \mathcal{A}_I(w_k)$, it suffices to show $F_I(x)' = \langle w_1, \dots, w_{i-1}, w_i, \dots, w_k \rangle$ is a fan.

(1) The vertices of $F_I(x)'$ are equivalent to those in the fan $F_C(x)$, so property 1 applies.

(2) Fan $F_I(x)$ is contained in the head of $F_I(x)'$, so the first vertex is the same and property 2 extends to fan $F_I(x)'$.

(3) Again, the existence of fan $F_C(x)$ tells us property 3 applies to the first few vertices in $F_I(x)'$. In order to show this property holds for the entire sequence, we must show $I(x, w_{j+1}) \in \mathcal{A}(w_j)$ for all $i - 1 \leq j < k$. We can apply the same reasoning as in Proposition 2.19 to conclude $I(x, w_{j+1}) \in \mathcal{A}(w_j)$ for all $i \leq j < k$. Hence, we are left with the case $j = i - 1$. The color c_k was in $\mathcal{A}_C(w_{i-1})$ and vertex w_{i-1} was contained in K_C , so after the inversion $c_x \in \mathcal{A}_I(w_{i-1})$. Additionally, edge (x, w_i) is in K_C and $C(x, w_i) = c_k$, so $I(x, w_i) = c_x$. Together, these imply $I(x, w_i) \in \mathcal{A}_I(w_{i-1})$ and property 3 holds for $F_I(x)'$.

We have successfully shown $F_I(x)'$ is a fan, so we are done with this case.

Case 2.2. $w_{i-1} \notin K_C(x, c_x, c_k)$

Similar to the previous case, we know $c_k \in \mathcal{A}_I(x)$. We also have $c_k \in \mathcal{A}_I(w_{i-1})$ because the edges adjacent to w_{i-1} were not affected by the inversion. Then we can apply Proposition 2.13 with $F_I(x)$ to produce a $(\Delta(G) + 1)$ -edge coloring of G and conclude $\chi'(G) \leq \Delta(G) + 1$.

In every case, we were able to demonstrate the existence of a $(\Delta(G) + 1)$ -edge coloring of G , so $\chi'(G)$ is at most $\Delta(G) + 1$. Putting this together with our lower bound, we can conclude

$$\Delta(G) \leq \chi'(G) \leq \Delta(G) + 1.$$

[A formal program proof] is not merely a method to make absolutely sure we have not made a mistake in a proof, but also a tool that shows us and compels us to understand why a proof works.

Georges Gonthier

3

Implementation

Vizing's theorem relies heavily on a number of auxiliary data structures, including graphs, edge colorings, edge sets, fans, and Kempe chains. For each of these, we needed to provide a well-suited definition, state basic properties, and reason about how it interacts with other structures and functions under particular circumstances. Ideally, the properties should be readily extractable from the chosen representation, and we should minimize the complexity required to properly interact with the other structures. Thus, finding the right representations in our code was crucial to the success of this project. We divided our code into, roughly, one file per core data structure or type as well as a file called `vizing.v` which contains the final proof of Vizing's theorem, pulling from these other files. The list of files and related

statistics can be viewed in Table 3.1. Aside from `vizing.v.`, each file listed centers around new defini-

File	Lines	Lemmas	Definitions	Admits
<code>alternate_path</code>	248	25	11	(15)
<code>aux</code>	169	24	4	0
<code>edge_coloring</code>	486	34	17	1
<code>fans</code>	330	37	12	2
<code>kempe</code>	343	22	6	3
<code>spath</code>	195	10	2	0
<code>vizing</code>	214	4	0	0
Total	1985	156	52	6

Table 3.1: Per-file breakdown of the VizingThesis library. The lines of code are excluding blank/commented lines. The lemma count includes admitted proofs, lemmas, corollaries, propositions, and theorems. The proofs in `alternate_path` are intentionally admitted, so we do not include them in our total.

tions (e.g., a Kempe chain, fan, etc.) and functions to modify the structures (e.g., swap colors, rotate a fan). The majority of each file is devoted to lemmas about properties which are derived directly from the definition (e.g., all k -edge colorings are proper edge colorings), change with functional operations (e.g., the effect of swapping colors on an absent set), or describe interactions with objects in other files (e.g., performing a Kempe swap with a fan). The purpose of this chapter is to elaborate on some of the representations we chose, why these were chosen, and how these impacted the project.

The remaining four sections are organized as follows: Section 3.1 provides background on the Rocq proof assistant before we dive into more technical details. Then Section 3.2 shares how graphs are seen in Rocq and what supporting material is already available within Rocq’s libraries. Section 3.3 is the heart of this chapter; we split this into four subsections, each roughly focused on one of the code files from Table 3.1. Finally, we conclude with Section 3.4 by addressing the admitted proofs alluded to in Table 3.1.

3.1 INTRODUCTION TO ROCQ

Proof assistants and formal verification systems can exist because of the *Curry-Howard correspondence* [42] which equates formal proof systems and type systems. Mathematical formulas and proofs can be interpreted as types and programs for models of computation, and vice versa. The formal system underlying the Rocq proof assistant utilizes the *calculus of inductive constructions* (CIC), an extension of the Curry-Howard correspondence. With this mindset, Rocq can be understood as both a dependently typed functional programming language and as a formal verification system where formulas are types and proofs are programs. Just as every program has a type, every proof term is associated with a formula or proposition. We write $x : T$ to mean “ x is of type T .” This can also be read as “ x belongs to T ” if we think of types as sets; in this sense, the term x is said to *inhabit* T . A type is *inhabited* if it contains at least one term. For most programming languages, we can interpret $x : T$ as “ x is a program of type T .” In Rocq, this could also be interpreted to mean “ x is a proof term for the proposition T .” Determining if some term x is indeed a proof for a proposition T is equivalent to deciding if $x : T$ is a well-typed expression. Rocq’s kernel is a type checker, and it follows the set of typing rules outlined by the CIC (see [43]) to verify programs and proofs.

As an introduction to how one might formalize a mathematical proof in Rocq, we will walk through an implementation of the following lemma.

Lemma 3.1. Suppose A and B are two finite sets containing elements of some type X . For any element a of type X , if we have $a \in A$ and $a \notin A \setminus B$, then $a \in B$.

This Lemma is named `notin_setD` in Rocq and is the first proof in the `aux` file. It is meant to assist with arguments about sets in later files. In Rocq, we type $(X : \text{finType})$ to mean X is any finite type, $(A, B : \{\text{set } X\})$ to mean variables A and B are any sets containing elements of type X , and $(a :$

X) to mean a is any element element of type X . Set membership or exclusion is denoted with “ $\backslash\text{in}$ ” and “ $\backslash\text{notin}$,” and the characters “ $:\backslash:$ ” refer to set difference. Then we can write the lemma as

Lemma notin_setD

```
{X : finType} (A B : {set X}) (a : X) (hA : a \in A) (hAB : a \notin A :\ B)
: a \in B.
```

where hA and hAB are arbitrary names we have assigned to our hypotheses. Thus, this lemma is of the form $x : T$ where x is our list of variables and hypotheses and T is the conclusion $a \in B$. Just as with programs, we can write this in curried form:

```
Lemma notin_setD {X : finType} (A B : {set X}) (a : X) :
a \in A -> a \notin A :\ B -> a \in B.
```

This has the same meaning as above, though it now better resembles the corresponding mathematical statement: $a \in A$ implies $a \notin A \setminus B$ implies $a \in B$. Note that this is logically equivalent to $a \in A$ and $a \notin A \setminus B$ implies $a \in B$.

To verify the lemma, we must enter a *proof state* by stepping to this location in the code and construct the proof term to show our hypotheses have type $a \in B$. A proof state consists of the context and the goal or goals. The *context* displays local hypotheses or assumptions (e.g. $a : X$), and the *goal* displays the current proof term we need to construct. Proof terms quickly become cumbersome and difficult to interpret. Like most proof assistants, Rocq supports *tactics* to assist the programmer in building a proof term with standard logical reasoning techniques. Our code is written in the small-scale reflection language (SSReflect). This is a formal proof language based on Rocq and functions as an extension of Rocq’s traditional tactics with the intent of shortening proofs and generalizing traditional tactics for a wider range of applications. Rocq’s Mathematical Components library, graph theory library, and

proof of the four color theorem are all written in SSReflect. An example of a tactic in SSReflect is the `rewrite` tactic. It supports rewriting two items given a proof that they are equivalent (e.g. rewriting an occurrence of A with B given $A = B$), similar to Rocq’s traditional `rewrite` tactic. Additionally, it incorporates definition expansion and partial evaluation. Another distinction of SSReflect are the tacticals “:” and “=>” which can be attached to tactics and move hypotheses from the context to goal or vice versa, respectively, without requiring an additional `intros` tactic or similar. Every goal is encouraged to end with a terminating tactic, that is, a tactic that generates an error if it does not close the goal. In SSReflect, this is commonly done by preceding the last line of a proof with the “by” tactical. Not only does this convert any subsequent tactics into terminating tactics; it also solves any trivial goals remaining after the tactic is applied.

Given this background, we close this section with the proof of our example Lemma 3.1. We present, side-by-side, the proof in Rocq and a traditional mathematical proof of the lemma. The exact library results we are rewriting with (`in_setD`, `andbT`, and `negbK`) are not as important here as the general structure of a proof in Rocq, but they essentially align with the steps taken in the mathematical proof.

Rocq proof

```
rewrite in_setD.
move=> hA. rewrite hA.
by rewrite andbT negbK.
```

Mathematical Proof

By definition of set difference, the assumption $a \notin A \setminus B$ is equivalent to $\neg(a \notin B \wedge a \in A)$.

Since $a \in A$, this is equivalent to $\neg(a \notin B \wedge \text{true})$.

Therefore, we have $\neg(a \notin B) = a \in B$. This is what we wished to show, so we are done.

3.2 THE GRAPH THEORY LIBRARY

Rocq’s main library for formalized mathematics is the Mathematical Components library, or MathComp. MathComp is written in SSReflect, and it was used in Gonthier’s proof of the four color theorem [11] as well as a proof of the odd order theorem [23]. While we did reference this library, especially for finite sets, sequences, and natural number arithmetic, the finite graphs in MathComp were built with planar graphs in mind and did not meet our needs. In 2019, Christian Doczkal and Damien Pous released a more general graph library that can now be installed as a Rocq package [44, 30], and we build our project off this. The library already includes Hall’s Marriage theorem, Menger’s theorem, and results related to vertex colorings, matchings, treewidth, connectivity, and a few other areas. They begin with a directed graph, called an `digraph`, as a structure consisting of a finite type G and a relation $R : G \rightarrow G \rightarrow \text{bool}$ where `bool` is an inductive data type that can either be true or false. The vertices of the graph are the inhabitants of G and the relation R denotes edges. For two vertices $x, y : G$, if the relation on x and y is true, then there is an edge between x and y in the graph, and this is denoted as $x \dashv\vdash y$. A simple graph, called an `sgraph`, is a `digraph` with the a symmetric and irreflexive restriction imposed on the edges. That is, a simple graph is a directed graph in which we have an edge $x \dashv\vdash y$ if and only if we have an edge $y \dashv\vdash x$ and there can be no self-loops ($x \neq y$). Sometimes we will refer to edges with this notation, and sometimes we will refer to them as sets with elements of type G , e.g. $[x; y]$. The latter does not depend on the relation $x \dashv\vdash y$ to be true, for we can easily write $[x; y]$ for any two vertices in the graph, even if there is no edge between x and y . However, sometimes the existence of an edge is unnecessary or it is implied by other hypotheses; in these cases, it is sometimes easier to work with the set notation rather than the relation.

3.3 IMPLEMENTATION DETAILS AND DECISIONS

3.3.1 EDGE COLORINGS

The first hurdle in our implementation was defining edge colorings. This foundational step was non-trivial and deserved sufficient deliberation. The main contender was rooted in set theory. In the library, vertex colorings are seen as partitions of the vertices into disjoint sets. Matching this form, we considered ‘coloring’ the edges by partitioning them into disjoint ‘color’ sets themselves. While this promotes the relationship between edge colorings and matchings, we ultimately rejected it for a number of reasons. Firstly, we would likely want to adjust the definition’s structure because the default approach introduced a cumbersome nested set notation. Additionally, it does not as cleanly lead to an extraction of the ‘color’ assigned to a given edge nor to reasoning about non-proper edge colorings. Thus, we rejected this viable alternative, though it could be interesting to compare our product to such an implementation.

Before migrating to a functional approach, we briefly considered constructing a new `CGraph` type which, similar to the library’s coercion between simple and directed graphs, would extend the `SGraph` with a symmetric coloring relation over the edges. Another option would be to modify a vertex coloring to assign *two* distinct colors to each vertex and require at least one color be shared between any two vertices with an edge relation. While these and many others were enjoyable and, in some cases, enlightening to ponder, most proved too inconvenient or bulky to be practically worthwhile.

Ultimately, we converged on the simple mapping

$$C_{G,Colors} : E(G) \rightarrow Colors$$

which sends an edge in some graph G to a color in some set $Colors$ of valid colors. In Rocq, this dependent mapping is defined as follows:

```
Definition edgeColoringType (G : sgraph) (ColorType : finType) : Type :=
  {set G} -> ColorType.
```

Thus, `edgeColoringType` is the type for a map sending an edge (represented as set of vertices) to a ‘color’ where the color is some object of type `ColorType`, and it is dependent on a simple graph G and finite type `ColorType`. We intentionally granted flexibility in the `ColorType` rather than avoid the additional argument but limit ourselves to something less general such as natural numbers. In fact, we almost never need to actually specify a finite type to be the `ColorType`. We only explicitly state our `ColorType` when we create an injective coloring to prove Vizing’s lower bound, matching each edge to a unique color. In this case, the `ColorType` is the type of the edges themselves (`{set G}`) which is valid because G itself is a finite graph.

Once we had established edge colorings, we added methods to recolor edges, swap edge colors, and extend an edge coloring to incorporate new edges. Swaps can be thought of as recoloring twice, so they were relatively similar to the recoloring implementation. We checked our definition of recoloring by proving, in one line each, that the recolored edge maps to its new color and for all other edges, the color of that edge remains the same. In other words, if we let $D = C[e \mapsto c_o]$ be the edge coloring with some edge e recolored as some color c_o , then we show $D(e) = c_o$ and $D(f) = C(f)$ for all edges $f \neq e$ as expected. Recoloring itself receives little direct attention in mathematical literature, in part because it is straightforward in nature and its applications to other more complicated structures or operations are of greater interest. However, a couple of recoloring propositions fundamental to Vizing’s theorem required some level of casework. This casework is often hidden within a succinct mathematical justifica-

tion, but proof assistants ensure no case is accidentally missed. The last statement of Misra and Grieg’s algorithm is

“Rotation leaves [the color] d free at [the vertex] w , since the only edge incident on w that has its color changed by the rotation is [the edge] (x, w) and this edge is uncolored by the rotation. Since d is also free at x , and since the rotation uncolors (x, w) , giving (x, w) the color d increases the number of colored edges and leaves the graph valid,” [33].

The lemma `recolor_proper` corresponding to the last claim alone (the recoloring “leaves the graph valid”) accounts for 30 lines of code excluding the assumptions and proof statement itself, for the Rocq kernel is well-suited to handle casework but must be told which cases to consider. It cannot trivialize a logical assertion at the same level of abstraction as Misra and Grieg can expect from their readers. As was the case throughout the implementation process, we had to intimately understand *why* each statement was true in order to validate it in Rocq.

To extend an edge coloring C when an edge was added to a graph G , we built a *de facto* partial coloring C' from C :

$$C'(e) = \begin{cases} \text{Some}(C(e)) & e \in E(G) \\ \text{None} & e \notin E(G) \end{cases}$$

Instead of creating an uncolored edge, we assigned the color ‘None’ to the new edge. All other colors were wrapped inside a ‘Some’ to ensure there were no conflicts (an edge in $E(G)$ may have already had color ‘None’). We then proved extending an edge coloring preserved the properness of a coloring and added exactly one new color to the color set. Although these seem like natural conclusions of the definition, the edge coloring C and the extended edge coloring C' have different types that became challenging to manage, especially because we already were dealing the graph G and the graph G with an edge added, which also have different types. In particular, we struggled to build fans starting at the newly

colored edge for both extended edge colorings and extended k -edge colorings. Eventually, we decided to have two separate methods of constructing fans. Vizing's theorem only relied on the fan constructed from an extended k -edge coloring, so this was sufficient. However, this may indicate we may be missing a feature of Rocq or there is limitation in our representation, and investigating this could be a direction for future work.

Proper edge colorings, referred to as `properEdgeColoringType`, and k -edge colorings, referred to as `kEdgeColoringType`, followed directly from the `edgeColoringType`. As noted in Chapter 2, these are technically the same by our mathematical definitions. In the code, we utilize the `kEdgeColoringType` when we wish to hide the `ColorType` and emphasize the cardinality of the color set. This occurs when we define the chromatic index because it is not be dependent on the choice of `ColorType` and 'chi' or 'k' is more concise than, for instance, typing $\#|c[E(G)]|$ to mean the cardinality of the image of an edge coloring c . As such, unlike the `edgeColoringType` and `properEdgeColoringType`, the `kEdgeColoringType` is dependent on a graph G and a natural number k but not some finite type `ColorType`. We included several results regarding lower and upper bounds of $\#|c[E(G)]|$ and created an injective coloring for any graph to assign every edge exactly one color, though we omit discussion of these here for brevity.

We then say a graph G is k -edge colorable if the type $(kEdgeColoringType\ G\ k)$ is inhabited given some graph G and natural number k . Recall from Section 3.1 that a type is inhabited if it contains at least one term. Determining if a type is inhabited is undecidable in general, and to convince Rocq a type is inhabited we must explicitly construct a term of that type. In this case, a graph G is k -edge colorable if we can show we have a term of type $(kEdgeColoringType\ G\ k)$. A natural number χ' is the chromatic index of a graph if the graph is χ' -edge colorable and for all k less than χ' the graph is not k -edge colorable. Hence, once we move to proving the upper bound of Vizing's Theorem, our goal

is to construct a term of type $(\text{kEdgeColoringType } G \ (\Delta(G) + 1))$. To do this, we required the assistance of Kempe chains, terminating functions to construct maximal-length structures, and a few new propositions about the edges of graphs.

3.3.2 KEMPE CHAINS

For the Kempe chain, the literature suggested two main representations: a recursively extended alternating path and a connected subgraph containing a given vertex. We focus on the latter, though we also explore the former as a means of comparison and deeper understanding. The former begins with a length-one path, adding edges of alternating color until it is no longer possible. Following Doczkal et al.’s packaged paths detailed in [44], we could quickly prove statements about extending and merging paths. Similar to fans, the construction of a maximal alternating path required a proof of termination. However, termination here relies on the properties of a proper edge coloring to avoid returning to a vertex already in the path as we extend it. We can push this more complicated proof away from the recursive function itself (and allow the function to be applicable even for improper edge colorings) by requiring the vertices to be unique. That is, the function terminates if no edges adjacent to the last vertex have the correct color or if such an edge exists but is adjacent to a vertex already in the path. The proof of termination also now nearly matched the maximal fan case, discussed in the next section. We would then later need to prove that the second case is unreachable when the relevant edge coloring is proper. The theorem `Vizings_altpath` in the file `vizing.v` proves Vizing’s theorem with Kempe chains implemented as these alternating paths, and they are defined in the file `alternating_path.v`. Most of the supporting lemmas as a proof-of-concept are stated but admitted. Bhoja follows this representation in Lean and the curious reader can reference his paper [39] and code [45] for completed proofs. We also added an edge membership predicate for paths to support the alternating paths. This is

fully implemented in `spath.v`, and Section 3.3.4 discusses it in greater depth.

Meanwhile, our final product supports Kempe chains interpreted as connected components of tailored subgraphs. Two main reasons ultimately led us to pursue this representation. Firstly, subgraphs and graph components already existed in the library. Modeled especially after the induced and complement graph types, we began by characterizing the edges of our Kempe subgraph with a graph G , an edge coloring C , and two colors c_1 and c_2 . If we let H be the Kempe subgraph, then the edges of H were defined as a subset of the edges of G :

$$E(H) = \{e \mid e \in E(G) \wedge (C(e) = c_1 \vee C(e) = c_2)\}$$

This translated to the following in Rocq:

```
Definition Kempe_rel :=
  [rel x y : G | x -- y && ((C [set x; y] == c1) || (C [set x; y] == c2))].
```

That is, there is an edge relation between any two vertices x and y if there is an edge relation between x and y in G and the edge (x, y) is colored either c_1 or c_2 under C . We could then define the subgraph H of G with the vertices $V(H) = V(G)$ and edges $E(H)$ from the symmetric, irreflexive edge relation `Kempe_rel`. We were required to prove basic properties of the edge relation and resulting subgraph in order to properly define the subgraph as a simple graph and be able to manipulate it later on. Yet, we could closely mirror the proofs for other subgraphs in the library, and, though it demanded some time, was not particularly challenging. We also verified that our implementation acted as expected. The most direct lemma in this process, labeled `mem_kempe`, stated

$$e \in E(H) \iff e \in E(G) \wedge (C(e) = c_1 \vee C(e) = c_2)$$

and turned out to be quite useful – we used it with the rewrite tactic 14 times.

Graph components were already in the graph library, so a Kempe chain was defined as the component of the Kempe subgraph H containing some specified vertex v . This leads us to the second benefit of the subgraph representation: we avoided the tricky termination proof associated with the maximal Kempe chain. A component of a graph is immediately maximal; it by definition contains all edges connected to the specified vertex. Thus, we could reason directly with the component and degrees of its vertices. We still had to show that a Kempe chain was an acyclic path in order to restrain the number of distinct single-degree vertices. Originally, we expected our proof to apply the definitions of paths and cycles to disprove the existence of disconnected components and branching. Eventually though, we settled on Proposition 2.18 and related results due to complications and limitations discussed in Section 2.2.3 of Chapter 2.

The Kempe chains themselves were the more challenging half of this section. Kempe swaps were akin to the recolor and swap operations, so much of the high-level logic could be applied. Some proofs were slightly longer because we also had to reason about edges in or not in the chain. The lemmas `not_kempe_edge` and `is_kempe_edge` assisted here and served to check assumptions about how the inversion should behave. They stated the color of an edge in the inverted edge coloring given its initial color and whether it was in the chain. Most of the other results on Kempe swaps are stated as lemmas in Section 2.2.3.

3.3.3 RECURSION AND TERMINATION

All functions in Rocq must terminate. By default, the kernel checks a *guard condition*, ensuring recursive calls are made on structurally smaller arguments [46]. The author can explicitly label the decreasing argument when necessary. However, sometimes this is not enough, as was the case with maximal fans

and maximal alternating paths. The claim that, given some object, there exists a maximal form of this object can be easily overlooked, taken as a fact rather than an assumption in handwritten proofs. The beauty of proof assistants is that no assumption can be unintentionally overlooked.

`Program Fixpoints` and `Equations` exist to assist in proving more complicated cases of dependent pattern matching and structural or well-founded recursion. As we discovered when attempting to practice with the `Program` mechanism, `Program` relies on an additional axiom – `JMeq`, or John Major’s Equality. The details on this axiom are out of the scope of this thesis, but we generally leaned against assuming additional axioms if not necessary. Luckily, the `Equations` plugin has no additional axioms, generates a functional elimination principle for inductive reasoning, and has, in our opinion, cleaner syntax.

To actually define terminating recursive functions for the fans and Kempe chains, we found a lower-bounded value that decreased at each recursive call. The vertices within a fan must be neighbors of the central vertex and must be unique, so each time a new vertex is added to the fan, the number of neighbors of the central vertex that are not already in the fan decreases. The vertices of a Kempe chain must also be unique, so each time a new vertex pair is added, the number of edges in the graph that are not already crossed by the chain decreases. Note that these values were bounded and finite because we only considered finite graphs.

These functions also required constructors to build new fans and paths. Each additional vertex created a longer sequence, but we needed to create a longer fan or a longer path. Fans and paths are sequences with additional constraints, so we wrote tailored sequence cons functions that would return not just a sequence but a fan or path, respectively, if the corresponding requirements were met. This allowed the recursive maximal fan and path functions to return objects with the expected type. Finally, referring to Bhoja [39], we formalized the notions of maximality for our fans and paths and proved that

the fans and paths returned by the recursive functions were indeed maximal.

3.3.4 EDGE OPERATIONS

As we converted our written proofs into code, we sometimes discovered basic concepts (especially related to edges) such as the edge set of a vertex had not yet been integrated into the library. Thus, we created files `aux.v` and `spath.v` to contain broadly applicable material that would be a valuable addition to the graph theory library independent of Vizing's theorem. This material includes, but is not limited to, the edge set of a vertex, edge membership in paths, additional results about the interior of a path, the maximum degree of a graph, and a special case of the lemma `bigmax_leq_pointwise` in the preliminaries of the graph library when exact equality is known. For the sake of brevity, we will only discuss the most interesting of these here: edge sets and path membership.

The (open) neighborhood $N_G(x)$ of a vertex x in some graph G is the set of vertices y in G such that vertices x and y are adjacent in G . On the other hand, the edge set $E_G(x)$ of a vertex x in some graph G is the set of edges (x, y) in G such that vertex x is adjacent to vertex y in G . Using set notation, we can express these formally:

$$N_G(x) = \{y \mid (x, y) \in E(G)\}$$

$$E_G(x) = \{(x, y) \mid y \in N_G(x)\}$$

The graph library already had established neighborhoods, so we only needed to define edge sets of a vertex. We present both definitions here, adjusting variable names for comparison:

```
open_neigh (G : diGraph) (x : G) := [set y | x -- y].
```

```
edge_neigh (G : sgraph) (x : G) := [set [set x; y] | y in (open_neigh G x)].
```

Both of these definitions use Rocq’s set notation and can be read analogously to the mathematical definitions we provided above. One may notice that G is type `diGraph` (a directed graph) for the open neighborhood and type `sgraph` (a simple graph) for the edge set. The scope of our project did not include directed graphs, so we did not concern ourselves with functionality beyond simple graphs, though, notably, simple graphs can be considered a special case of directed graphs. In certain scenarios, such as this definition, the two are essentially interchangeable, though not in all, such as when we rely on the symmetry of a simple graph. In any case, expanding our code to directed graphs where applicable should not be incredibly complicated and is a useful direction for future work. The main distinction we would like to point out is the use of the relation “ $x \dashv\dashv y$ ” versus the set “[set x ; y].” In this library, the notation $x \dashv\dashv y$ denotes the edge relation between two vertices x and y . The set notation [set x ; y] ignores the relation, only storing the two vertices themselves. Yet, no information is lost because we know that a pair of vertices x and y are in $E_G(x)$ only if the vertex y is in $N_G(x)$ and from this we can extract the relation. In fact, we prove

```
Lemma edgesSetP {G : sgraph} (x : G) (e : {set G}) :
  reflect (exists y, e = [set x; y] /\ x -- y) (e \in (edge_neigh G x)).
```

to explicitly use this fact when necessary. For the majority of cases, the set notation was easier to reason about, especially given its alignment with the library’s definition for $E(G)$, the set of all edges in the graph. This definition and a number of helpful lemmas can be found in the `aux` file of [40].

We also heavily reference definition, lemmas, and the general structure of the packaged paths designed for the graph theory library. We observed no consistent definition for edge membership of these paths. Kempe chains as alternating paths necessitate a definition of edge membership, so we took this as an opportunity to contribute a potential implementation. To determine if a path crossed some edge,

we needed to check if the edge’s two endpoint vertices appeared consecutively in the path. We found the best way to go about this was to mimic the definition of the path itself from MathComp. MathComp paths consist of a sequence of elements and a boolean predicate asserting some progression of the sequence. It also includes the view `pathP` directly relating a path to a proposition. We immediately derived our edge predicate from `pathP` by replacing the “forall” with an “exists” and dropping the boolean relation. Thus, given a path P of a sequence of vertices $x_1, \dots, x_n$ and an edge (u, v) , we say the edge (u, v) is in P if the following logical statement is true:

$$\exists i < n. (p_i = u \wedge p_{i+1} = v)$$

The head of a path is explicitly stated in Rocq, so the path P above would be written as $x_1 :: p$ where p would be the sequence $x_2, \dots, x_n$ (which may be empty) and “ $::$ ” is syntactic sugar for a cons operation, appending x_1 to the beginning of the sequence p . To access the i^{th} index in a path, we use the `nth` function. This takes in a default value in case the i^{th} index does not exist (e.g. the value of i is a greater the length of the path itself). With this context, we present our raw edge membership predicate here.

```
Definition pathp_edge {G : sgraph} (x1 u v : G) (p : seq G) : Prop :=
  exists i, (i < size p) && (nth x1 (x1::p) i == u) && (nth x1 p i == v).
```

The file `spath` also includes basic facts and reasoning about common path operations in relation to path edges. For example, the lemma `is_path_edge` asserts that if edge (u, v) is a path edge, then the edge $u - v$ is in the graph and the lemma `edgep_path_edge` proves that given a two-vertex path $P = x_1, x_2$, the edge (u, v) is an edge in P if and only if $u = x_1$ and $v = x_2$, or symmetrically, $v = x_1$ and $u = x_2$. The longest of these is `cat_path_edge` which allows us to split a path, checking edge membership on the two sub-paths instead. These lemmas solidify the chosen representation and establish a solid

foundation for future applications.

3.4 OMITTED MATERIAL

While we have proved over 100 lemmas and theorems in our code, there are still six proofs which end with `Admitted`. Like the `a sorry` in Lean, the `admitted` command terminates a proof even if there are open goals. In this section, we discuss why each of these were left incomplete and what would be required to finish them.

- *extended_absent*: Suppose C is an edge coloring of a graph G , and suppose we add a new edge to G to get the graph G' . Let C' be the extended edge coloring C for G' . This lemma states that for any vertex $x \in V(G) = V(G')$, if there exists some color c_o in $\mathcal{A}_C(x)$ then the color $\text{Some}(c_o)$ is in $\mathcal{A}_{C'}(x)$. We were somewhat unsatisfied with the way in an extended edge coloring interacted with our other definitions, so we did not yet complete this proof. It likely can be finished by expanding the definitions of absent sets, applying set membership and set difference predicates, and performing casework. Since we were limited in time, confident on the correctness of the statement, and it is not a crucial or striking result, we leave it as future work.
- *imset_rot_del_edge*: Suppose C is an edge coloring of a graph G and R is the edge coloring C after the rotation of some fan in the graph. Then this lemma can be seen as an extension of Lemma 2.10 which shows $C[E(G)] = R[E(G)]$. If e_1 and e_2 are any two edges in some graph G such $C(e_1) = R(e_2)$ (where e_1 and e_2 may be the same edge), it states that $C[E(G) - e_1] = R[E(G) - e_2]$. That is, if the colors are the same in the respective colorings, then we can ignore the edges when considering the color set of the respective colorings and these will still be equivalent color sets. The proof, like most of our proofs with the rotation algorithm, utilizes induction. We have proven the two base cases, which includes proving the corresponding lemma for swapping edge colors, and have made progress on the inductive step. At this point, the proof could become messy as we reason about set membership, rotation, swapping edge colors, and the inductive hypothesis together. There may be a better way to approach this proof or phrase the lemma, and we leave this as future work.
- *rot_proper*: This corresponds to Lemma 2.9 from Chapter 2. That is, it shows the properness of an edge coloring is preserved when a fan is rotated. Again, this proof uses an induction argu-

ment. Now, we would need to incorporate a number of auxiliary lemmas such as `rot_w0_prop`, `rot_notin`, `absent_edge`, and `swap_proper_vertex`. We were unable to finish the proof in Rocq, but these auxiliary lemmas are implemented, and the remainder of the proof should closely follow the written proof in Chapter 2.

- *`in_inverted_absent` and `notin_inverted_absent`*: These two lemmas are statements regarding the absent set of vertices that are in and not in a given Kempe chain, respectively, before and after the inversion occurs with a Kempe swap. These are intuitive statements that follow almost immediately from the definitions of Kempe chains and Kempe swaps. However, the proofs themselves would be on the longer side due to the casework required. Since we were limited on time, confident on the correctness of these statements, and these were not critical results, we left them as future work.
- *`inverted_fan`*: This proposition is meant to capture Proposition 2.19 from Chapter 2 describing a fan before and after we perform a Kempe swap, inverting the colors in a Kempe chain. Applying this proposition is a major step, and most of it is proven in the code; there are already 70 lines of code devoted to proof itself (excluding the statement and assumptions). It is incomplete because we have not yet proven that property 3 of fans holds for the sub-fan under consideration. To do so, we would need to better familiarize ourselves with MathComp’s path type. This includes learning to separate paths into connected sub-paths and splitting the path property into these sub-paths. Additionally, we likely will need the previous two lemmas `in_inverted_absent` and `notin_inverted_absent`. Thus, proving this lemma should be a manageable goal but will take a little bit more time than some of the other admitted statements.

4

Conclusion

In this thesis, we presented an in-depth proof of Vizing’s theorem on the chromatic index of a simple graph. This compilation of proof sketches, published algorithms, and some novel lemmas creates a single source for all the required components in a formal proof of Vizing’s theorem. We implemented the contents of this paper in the proof assistant Rocq. Our main contribution is the definition of an edge coloring and related basic operations within the framework of Rocq’s graph theory library. Once we became familiar with typical Rocq and SSReflect practices as well as the nuances of the graph theory library, the challenge became implementing edge colorings in a way that felt natural and would be the simplest to interact with. With a definition of edge colorings centered around mappings, we formalized

and verified properties about fans, rotation, Kempe chains, and Kempe swaps. Maximal length fans and Kempe chains as alternating paths demanded we familiarize ourselves with Fixpoints and recursion to ensure termination, a strict requirement of Rocq functions. Due to anticipated complications creating maximal Kempe chains, we researched alternative representations and discovered we could sidestep infinite loops entirely with a subgraph representation. We then needed to prove more general lemmas relating connected paths and vertex degree before applying the chains to our final proof. We concluded the project with a verified proof of Vizing's theorem for both Kempe chains as alternating paths and Kempe chains as subgraph components.

4.0.1 FUTURE WORK

While we completed our original goal to implement and verify Vizing's theorem in the proof assistant Rocq, there are many opportunities to extend the project. Firstly, a few admitted lemmas are still scattered within the code as discussed in Section 3.4, so the next immediate goal is to complete the corresponding proofs. Secondly, we believe this project would be a valuable contribution to the graph theory library created by Doczkal and Pous. The next step here is to connect with the library's maintainers. After this, we imagine we will need to edit our library to ensure it matches stylistically, and some lemmas may be made more or less general. We ideally could work in stages, first integrating edge colorings and then progressing towards Kempe chains and fans before eventually contributing to Vizing's theorem itself. Finally, on a related note, there are several formalized lemmas that could be generalized or be starting points for other graph theory results. We have shown that we can extend an edge coloring when a new edge is added to a graph and preserve the properness of the edge coloring. We could support adding any set of non-adjacent edges, in contrast to only a single edge, and still preserve the properness of an edge coloring. Additionally, some aspects of this library, including edge sets at a vertex

as discussed in Section 3.3, could apply to both directed and simple graphs, but we currently only reason over simple graphs. Vizing's theorem itself can be generalized to multigraphs, and proving this would be a worthwhile endeavor. Beyond this, there is a plethora of literature surrounding edge colorings and chromatic index. We hope that future work will continue to improve upon our implementation and one day any edge coloring result will be able to be formally stated in a proof assistant.

References

- [1] F. Guthrie, “Note on the Colouring of Maps,” *Proceedings of the Royal Society of Edinburgh*, vol. 10, pp. 727–728, 1880.
- [2] “London mathematical society,” *The Academy*, vol. 13, no. 320, p. 562, Jun 22 1878, last updated - 2026-03-23. [Online]. Available: <https://search.proquest.com/historical-periodicals/london-mathematical-society/docview/1298598177/se-2>
- [3] R. Fritsch and G. Fritsch, “History,” in *The Four-Color Theorem: History, Topological Foundations, and Idea of Proof*. New York, NY: Springer New York, 1998, pp. 1–41.
- [4] P. Heawood, “Map-Colour Theorem,” *The Quarterly Journal of Pure and Applied Mathematics*, vol. 24, pp. 332–338, 1890.
- [5] Ágnes Cseh, “English translation of “Über graphen und ihre anwendung auf determinantentheorie und mengenlehre” by Dénes König,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.17140>
- [6] V. G. Vizing, “On an estimate of the chromatic class of a p-graph,” *Diskret. Analiz.*, vol. 3, pp. 25–30, 1964.
- [7] K. I. Appel and W. Haken, *Every planar map is four colorable*, 1st ed., ser. Contemporary mathematics (American Mathematical Society) ; v. 98. Providence, Rhode Island: American Mathematical Society, 1989.
- [8] R. J. Wilson, “But is it a proof?” in *Four Colours Suffice: How the Map Problem Was Solved*. London: Allen Lane, 2002, pp. 214–228.
- [9] M. Sozeau, S. Boulrier, Y. Forster, N. Tabareau, and T. Winterhalter, “Coq Coq correct! verification of type checking and erasure for Coq, in Coq,” *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019. [Online]. Available: <https://doi.org/10.1145/3371076>
- [10] G. Gonthier, “Formal proof—the four- color theorem,” 2008. [Online]. Available: <https://api.semanticscholar.org/CorpusID:12620754>

- [11] —, “The Four Color Theorem in Coq.” [Online]. Available: <https://github.com/rocq-community/fourcolor>
- [12] R. P. Gupta, “Studies in the theory of graphs,” Ph.D. dissertation, Indian Statistical Institute, 1967.
- [13] R. L. Brooks, “On colouring the nodes of a network,” *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 37, no. 2, p. 194–197, 1941.
- [14] T. R. Jensen and B. Toft, *Graph Coloring Problems*, ser. Wiley-Interscience Series in Discrete Mathematics and Optimization. New York: John Wiley & Sons, 1995.
- [15] I. Holyer, “The NP-completeness of edge-coloring,” *SIAM Journal on Computing*, vol. 10, no. 4, pp. 718–720, 1981.
- [16] P. Crescenzi and V. Kann, “How to find the best approximation results,” *SIGACT news*, vol. 29, no. 4, pp. 90–97, 1998.
- [17] S. B. Thakare and H. R. Bhapkar, “Graph coloring, types and applications: A survey,” *International Journal of Mathematical Combinatorics*, vol. 1, no. 1, pp. 110–139, 03 2024. [Online]. Available: <https://search.proquest.com/scholarly-journals/graph-coloring-types-applications-survey/docview/3068462330/se-2>
- [18] A. Schrijver, *Combinatorial Optimization: Polyhedra and Efficiency*. Springer, 2003.
- [19] P. J. Cameron, “Minimal edge-colourings of complete graphs,” *Journal of the London Mathematical Society*, vol. s2-11, no. 3, pp. 337–346, 1975.
- [20] A. Hilton, “Embedding incomplete latin rectangles and extending the edge colourings of graphs,” *Nanta Mathematica*, vol. 10, 01 1977.
- [21] J. Edmonds and E. L. Johnson, “Matching, euler tours and the chinese postman,” *Mathematical Programming*, vol. 5, pp. 88–124, 1973.
- [22] S. Fiorini, “A bibliographic survey of edge-colorings,” *Journal of graph theory*, vol. 2, no. 2, pp. 93–106, 1978.
- [23] G. Gonthier, A. Asperti, J. Avigad, Y. Bertot, C. Cohen, F. Garillot, S. Le Roux, A. Mahboubi, R. O’Connor, S. Ould Biha, I. Pasca, L. Rideau, A. Solovyev, E. Tassi, and L. Théry, “A machine-checked proof of the odd order theorem,” in *Interactive Theorem Proving*, S. Blazy,

- C. Paulin-Mohring, and D. Pichardie, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 163–179.
- [24] D. Kästner, X. Leroy, S. Blazy, B. Schommer, M. Pister, and C. Ferdinand, “CompCert - A Formally Verified Optimizing Compiler,” 01 2016.
 - [25] Axiom Math AI. (2026) Axiom. [Online]. Available: <https://axiommath.ai/>
 - [26] Harmonic. (2026) Harmonic: Ai for formal mathematical reasoning. AI system for formal mathematical reasoning and mathematical superintelligence. [Online]. Available: <https://harmonic.fun/>
 - [27] L. Noschinski, “A graph library for isabelle,” *Mathematics in Computer Science*, vol. 9, no. 1, pp. 23–39, 2015. [Online]. Available: <https://doi.org/10.1007/s11786-014-0183-z>
 - [28] T. mathlib Community, “The lean mathematical library,” in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, 2020.
 - [29] The MC Team, “Mathematical components,” Coq library, n.d. [Online]. Available: <https://math-comp.github.io/>
 - [30] C. Doczkal and D. Pous, “Graph theory,” 2018, coq library, GitHub repository. [Online]. Available: <https://github.com/coq-community/graph-theory>
 - [31] B. Bollobás, *Graph Theory: An Introductory Course*. Springer, 1979.
 - [32] J. R. Rao and E. W. Dijkstra, “Designing the proof of Vizing’s theorem,” in *Programming and Mathematical Method*, M. Broy, Ed. Berlin, Heidelberg: Springer, 1992, pp. 17–25.
 - [33] J. Misra and D. Gries, “A constructive proof of Vizing’s theorem,” *Information Processing Letters*, vol. 41, no. 3, pp. 131–133, 1992.
 - [34] R. Green, “Vizing’s Theorem and Edge-Chromatic Graph Theory,” 2015. [Online]. Available: <https://api.semanticscholar.org/CorpusID:36220274>
 - [35] R. Diestel, *Colouring*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2025, pp. 123–154. [Online]. Available: https://doi.org/10.1007/978-3-662-70107-2_5
 - [36] D. Scheide, M. Stiebitz, L. M. Favrholt, and B. Toft, “Graph edge coloring: Vizing’s theorem and goldberg’s conjecture,” in *Graph Edge Coloring*. John Wiley & Sons, Incorporated, 2012.

- [37] J. Breitner, “Proving vizing’s theorem with rodin,” 2011. [Online]. Available: <https://www.joachim-breitner.de/>
- [38] Z. Aarbi, H. He, and M. Gatti, “Formalizing vizing’s theorem,” Math 480: Introduction to Mathematical Formalization, University of Washington, 2025, mentored by Jarod Alper and Vasily Ilin. [Online]. Available: <https://sites.math.washington.edu/ai/spring2025.html>
- [39] A. Bhoja, “A verified implementation of the Misra and Gries edge coloring algorithm,” 2025.
- [40] M. Grinstead, “Formalization of Vizing’s theorem in Rocq,” 2026. [Online]. Available: <https://github.com/milligang/vizing-thesis>
- [41] M. Elkin and L. Barenboim, “Edge-coloring and maximal matching,” in *Distributed Graph Coloring*, ser. Synthesis Lectures on Distributed Computing Theory. Switzerland: Springer International Publishing AG, 2013, pp. 93–102.
- [42] W. A. Howard, “The formulae-as-types notion of construction,” in *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, J. P. Seldin and J. R. Hindley, Eds. Boston, MA: Academic Press, 1980, pp. 479–490.
- [43] The Rocq Development Team, “Calculus of inductive constructions (cic),” in *The Rocq Prover Reference Manual, Version 9.1.0*, 2025, core language chapter. [Online]. Available: <https://rocq-prover.org/doc/V9.1.0/refman/language/cic.html>
- [44] C. Doczkal and D. Pous, “Graph theory in coq: Minors, treewidth, and isomorphisms,” *Journal of Automated Reasoning*, vol. 64, no. 5, pp. 795–825, 2020. [Online]. Available: <https://doi.org/10.1007/s10817-020-09543-2>
- [45] A. Bhoja, “A verified implementation of the Misra and Gries edge coloring algorithm,” 2025. [Online]. Available: <https://github.com/aroheebhoja/vizing>
- [46] The Rocq Development Team, “Well-founded recursion using equations,” in *The Rocq Platform Documentation*, 2025, chapter in online documentation. [Online]. Available: https://rocq-prover.github.io/platform-docs/equations/tutorial_wf_recursion.html



The Code

We display our Rocq code in this Appendix to be viewed alongside the thesis, though it may also be accessed from our GitHub repository [[40](#)].

A.1 BASIC EXAMPLES

```
1 Set Warnings "-notation-overridden, -notation-incompatible-prefix".  
2 (* to ignore warnings due to the import of ssreflect *)  
3 From mathcomp Require Import all_boot.  
4 From Stdlib Require Import RelationClasses.  
5 From GraphTheory Require Import digraph sgraph dom.  
6 Require Import Stdlib.Logic.Classical_Prop.  
7
```

```

8 Set Bullet Behavior "Strict Subproofs".
9
10 Lemma bipSubCom (n : nat) : subgraph ('K_n,1) ('K_n.+1).
11 Proof.
12   apply: sub_Kn.
13   rewrite card_sum !card_ord.
14   by rewrite addn1.
15 Qed.
16
17 (* any singleton dominates K_n *)
18 Lemma domK n (v : 'K_n) :
19   dominating [set v].
20 Proof.
21   apply/forallP => u.
22   rewrite /closed_neigh_set big_set1 /closed_neigh in_setU1 /orb.
23   case: (u == v) /eqP => //.
24   - move/eqP. rewrite in_opn. rewrite /edge_rel/= by rewrite eq_sym.
25 Qed.

```

A.2 AUXILIARY MATERIAL

```

1   From mathcomp Require Import all_boot.
2   From GraphTheory Require Import edone preliminaries digraph sgraph.
3
4   Set Warnings "-notation-overridden, -notation-incompatible-prefix".
5
6   Set Implicit Arguments.
7   Unset Strict Implicit.
8   Unset Printing Implicit Defensive.
9
10  Lemma notin_setD {T : finType} (A B : {set T}) (x : T) :
11    x \in A -> x \notin A :\| B -> x \in B.
12  Proof.
13    rewrite in_setD => ->.
14    by rewrite andbT negbK.
15  Qed.
16
17  Section preliminaries.
18    Variables (T : finType).
19    Implicit Types (A B : pred {set T}) (e : rel T).

```

```

20
21 Lemma bigmax_eq_pointwise (P : pred T) (F G : T -> nat) :
22   {in P, forall x, F x = G x} -> \max_(i | P i) F i = \max_(i | P i) G i.
23 Proof.
24   move => ?. by elim/big_ind2 : _ => // y1 y2 x1 x2 -> ->.
25 Qed.
26
27 Definition restrict2_mem (P : mem_pred {set T}) e :=
28   [rel u v | in_mem [set u; v] P && e u v].
29 Local Notation restrict2 P := (restrict2_mem (mem P)).
30
31 Lemma sub_restrict2 A e :
32   subrel (restrict2 A e) e.
33 Proof. move => x y /=. by case: (e x y); case: (_ \in A). Qed.
34
35 Lemma restrict2_mono A B e :
36   {subset A <= B} -> subrel (restrict2 A e) (restrict2 B e).
37 Proof. move => H x y /= => /andP [HA ->]. by rewrite !H. Qed.
38
39 Lemma restrict2_irrefl A e :
40   irreflexive e -> irreflexive (restrict2 A e).
41 Proof. move => irr_e x /=; by rewrite irr_e. Qed.
42
43 Lemma restrict2_sym A e :
44   symmetric e -> symmetric (restrict2 A e).
45 Proof. move => sym_e x y /=. by rewrite sym_e setUC. Qed.
46
47 Lemma erpath_sub A e x p :
48   path (restrict2 A e) x p -> path e x p.
49 Proof.
50   elim: p x => // = b p IH x. rewrite -!andbA => /and3P[H1 H2 H3].
51   by rewrite H2 (IH b H3).
52 Qed.
53
54 (* restrict in preliminaries is actually a special case of restrict2 *)
55 Lemma restrict_eq_restrict2 (P : pred T) e :
56   restrict P e =2 restrict2 [pred f : {set T} | f \subset P] e.
57 Proof.
58   move => x y /=; rewrite /restrict_mem /restrict2_mem inE.
59   apply/andb_id2r => _.
60   apply/andP/subsetP=> [[+ +] z /set2P [-> //|-> //]| H].

```

```

61   by split; [apply/H/set21 | apply/H/set22].
62   Qed.
63
64   Lemma restrict2E A e:
65     A =i predT -> connect (restrict2 A e) =2 connect e.
66   Proof.
67     move => H x y. rewrite (eq_connect (e' := e)) //.
68     move => {x y} x y /=. by rewrite !H.
69   Qed.
70
71   End preliminaries.
72   Notation restrict2 A := (restrict2_mem (mem A)).
73
74   Section erestrictGraph.
75   Variables (G : sgraph) (A : pred {set G}).
76
77   Lemma symmetric_restrict2_sedge :
78     symmetric (restrict2 A (--)).
79   Proof. apply: restrict2_sym. exact: sg_sym. Qed.
80
81   Lemma erestrict_sym :
82     connect_sym (restrict2 A (--)).
83   Proof. exact/connect_symI/symmetric_restrict2_sedge. Qed.
84
85   Lemma sedge_in_equiv2 :
86     equivalence_rel (connect (restrict2 A (--))).
87   Proof. exact/equivalence_rel_of_sym/symmetric_restrict2_sedge. Qed.
88
89   Definition erestrict : sgraph :=
90     Eval hnf in SGraph (restrict2_sym A (@sg_sym G))
91       (restrict2_irrefl A (@sg_irrefl G)).
92
93   Lemma upathPR (x y : G) :
94     reflect (exists p : seq G, @upath erestrict x y p)
95       (connect (restrict2 A (--)) x y).
96   Proof. exact: (@upathP erestrict). Qed.
97
98   Lemma erestrict_sub : subgraph erestrict G.
99   Proof. by exists id=> // u v /andP[_ ->]. Qed.
100 End erestrictGraph.
101

```

```

102 Section EdgeNeighborhood.
103   Definition edge_neigh (G : sgraph) x := [set [set x; y] | y in N(G;x)].
104
105   Local Notation "E{ x }" := (@edge_neigh _ x) (at level 0, format "E{ x }").
106   Local Notation "E{ G ; x }" := (@edge_neigh G x) (at level 0, format "E{ G ; x }").
107
108   Lemma sub_all_edges {G : sgraph} (v : G) : E{v} \subset E(G).
109   Proof.
110     apply/subsetP => e.
111     rewrite/edge_neigh.
112     move/imsetP => [w Hw ->].
113     apply/edgesP; exists v, w.
114     split => //.
115     by rewrite in_opn in Hw.
116   Qed.
117
118   Lemma edge_neigh_self {G : sgraph} {x y : G} (xy : x -- y) : [set x; y] \in E{x}.
119   Proof.
120     rewrite /edge_neigh; apply/imsetP; exists y;
121     by rewrite -in_opn in xy.
122   Qed.
123
124   Lemma mem_edge_graph {G : sgraph} (x : G) (e : {set G}) :
125     (e \in E{G;x}) = (e \in E(G)) && (x \in e).
126   Proof.
127     rewrite /edge_neigh.
128     apply/imsetP/andP => [[y Hy ->] | [/edgesP [u [v [-> uv]]] Hx]];
129     first by rewrite !inE eqxx in_edges -in_opn.
130     case/set2P : Hx => ?; subst; [exists v | exists u]; try rewrite in_opn; rewrite //;
131     first by rewrite sg_sym.
132     by apply/doubleton_eq_iff; right.
133   Qed.
134
135   Lemma edgesSetP {G : sgraph} (x : G) (e : {set G}) :
136     reflect (exists y, e = [set x; y] /\ x -- y) (e \in E{G;x}).
137   Proof.
138     apply: (iffP imsetP) => [[y] | [y] [E xy]];
139     first by rewrite in_opn; exists y.
140     exists y; by rewrite // in_opn.
141   Qed.
142

```

```

143 Lemma edge_neigh_edge {G : sgraph} (x y : G) (e : {set G}) :
144   ((e \in E{x}) && (e \in E{y}) && (x != y)) <-> (e == [set x; y]) && x -- y.
145 Proof.
146   split=> [/andP[/andP[/imsetP[z1] Hn1 He1 /imsetP[z2] Hn2 He2]] | /andP[/eqP Heq xy]].
147   - rewrite He2 in He1.
148     case: (iffLR (doubleton_eq_iff y z2 x z1) He1);
149     move=> [Hxy Hz]; first by rewrite Hxy eqxx.
150     rewrite in_opn -Hxy in Hn1; rewrite Hz setUC in He2.
151     by move/eqP: He2.
152   - move: (xy)=> /sg_edgeNeq ->.
153     rewrite Heq (edge_neigh_self xy).
154     have -> : [set x; y] = [set y; x] by apply/doubleton_eq_iff; right.
155     by rewrite sg_sym in xy; rewrite (edge_neigh_self xy).
156 Qed.
157
158 Lemma del_edges_edge_neigh (G : sgraph) (A e : {set G}) (x : G) :
159   e \in E{del_edges A;x} = (e \in E{G;x}) && ~~ (e \subset A).
160 Proof.
161   rewrite/edge_neigh.
162   apply/imsetP/andP => [[z + ->] | [/imsetP [z] Hin -> Hns]].
163   - rewrite (del_edges_opn _ x) => /andP[Hin ->].
164     by split; first apply/imsetP; try exists z.
165   - move/andP: (conj Hin Hns). rewrite -del_edges_opn.
166     by exists z.
167 Qed.
168
169 Lemma card_edge_neigh (G : sgraph) (v : G) :
170   #|E{v}| = #|N(v)|.
171 Proof.
172   rewrite /edge_neigh.
173   apply: card_imset => w1 w2.
174   by rewrite doubleton_eq_left.
175 Qed.
176
177 End EdgeNeighborhood.
178 Notation "E{ x }" := (@edge_neigh _ x) (at level 0, format "E{ x }").
179 Notation "E{ G ; x }" := (@edge_neigh G x) (at level 0, format "E{ G ; x }").
180
181 Lemma del_edges1_neq {G : sgraph} (e del_e : {set G}) :
182   e \in E(del_edges del_e) -> e != del_e.
183 Proof.

```

```

184   move=> He.
185   apply/eqP => Heq.
186   rewrite Heq in He.
187   by move: (del_edgesN del_e); rewrite He.
188   Qed.
189
190 Definition max_degree (G : sgraph) : nat := \max_(x in G) #|N(x)|.
191
192 (* When we delete edges, it's easier to reason about E{x} *)
193 Lemma max_deg_edge (G : sgraph) : \max_(x in G) #|N(x)| = \max_(x in G) #|E{x}|.
194 Proof.
195   apply: bigmax_eq_pointwise => v _; by rewrite card_edge_neigh.
196   Qed.
197
198 Lemma del_edges_max_deg (G : sgraph) (A : {set G}):
199   max_degree (del_edges A) <= max_degree G.
200 Proof.
201   rewrite/max_degree 2!max_deg_edge.
202   apply: bigmax_leq_pointwise => x _.
203   apply: subset_leq_card.
204   apply/subsetP => e.
205   by rewrite (@del_edges_edge_neigh G A e) => /andP[-> _].
206   Qed.

```

A.3 SIMPLE PATHS

```

1  From mathcomp Require Import all_boot.
2  From GraphTheory Require Import edone preliminaries digraph sgraph.
3
4  Set Warnings "-notation-overridden, -notation-incompatible-prefix".
5
6  Set Implicit Arguments.
7  Unset Strict Implicit.
8  Unset Printing Implicit Defensive.
9
10 Section PathHelpers.
11   Variables (G : sgraph) (x y z : G) (p : Path x y).
12
13   Lemma inInteriorP :
14     reflect (z \in interior p) ((z != x) && (z != y) && (z \in p)).

```

```

15 Proof. rewrite !inE negb_or; exact: idP. Qed.
16
17 Lemma isplitInternal :
18   irred p -> z \in interior p ->
19   exists (u v : G) (uz : u -- z) (zv : z -- v) (p1 : Path x u) (p2 : Path v y),
20   p = pcat (pcat p1 (edgep uz)) (pcat (edgep zv) p2) /\ u != v.
21 Proof.
22   move=> Ip /inInteriorP /andP[/andP[xNz zNy] z_in_p]; rewrite eq_sym in xNz.
23   case/(isplitP Ip) def_p1 : _ / z_in_p => [pl pr Ipl Ipr Iz].
24   case: (splitR pl xNz) Ipl => u [pl'] [uz] El Ipl.
25   case: (splitL pr zNy) Ipr => v [zv] [pr'] [Er _] Ipr.
26   exists u, v, uz, zv, pl', pr'.
27   split; first by rewrite El Er.
28   (* by contradiction if u = v *)
29   apply/eqP=> eq_uv.
30   have u_in_pl : u \in pl by rewrite El mem_pcat_edgeR path_end.
31   have u_in_pr : u \in pr by rewrite eq_uv Er mem_pcat_edgeL path_begin.
32   have := Iz u u_in_pl u_in_pr.
33   by have /eqP := sg_edgeNeq uz.
34 Qed.
35
36 Lemma deg1_internal : #|N(z)| <= 1 -> irred p -> z \notin interior p.
37 Proof.
38   move=> /leq_gtF deg1 Ip; apply/negP=> z_in_p.
39   have [u [v] [uz] [zv] [p1] [p2] [pc uNv]] := isplitInternal Ip z_in_p.
40   suff : 2 <= #|N(z)|; first by rewrite deg1.
41   rewrite -(ltn_add2r 1) addn1.
42   have -> : 2 = #|[set u; v]| by rewrite cards2 uNv.
43   rewrite addn1 ltnS.
44   apply/subset_leqif_cards.
45   by rewrite subUset 2!subset 2!in_opn sg_sym.
46 Qed.
47
48 End PathHelpers.
49
50 Proposition shared_interior3
51   (G : sgraph)
52   (x y z : G)
53   (p : Path x y)
54   (q : Path x z)
55   :

```



```

56 #|N(x)| = 1 -> #|N(y)| = 1 -> #|N(z)| = 1 ->
57 x != y -> y != z -> x != z ->
58 irred p -> irred q ->
59 exists w : G, 3 <= #|N(w)|.
60 Proof.
61 move=> /eq_leq degx /eq_leq degy /eq_leq degz xNy yNz xNz Ip Iq.
62 have z_nin_p : z \notin p by have := deg1_internal degz Ip;
63 rewrite !inE negb_and negb_or eq_sym xNz eq_sym yNz.
64 have y_nin_q : y \notin q by have := deg1_internal degy Iq;
65 rewrite !inE negb_and negb_or eq_sym xNy yNz.
66 case: (splitL p xNy) => u [xu] [p'] [Epc _].
67 case: (splitL q xNz) => u' [xu'] [q'] [Eqc _].
68 have uEu' : u = u'.
69 {
70   move/card_le1_eqP: degx=> in_nx.
71   have := in_nx u u'.
72   rewrite 2!in_opn=> eq_uu'.
73   by have := eq_uu' xu xu'.
74 }
75 have x_nin_p' : x \notin p' by rewrite Epc irred_edgeL in Ip; case/andP: Ip.
76 have y_nin_q' : y \notin q'
77   by move: y_nin_q; rewrite Eqc mem_pcat_edgeL negb_or=> /andP[_ ->].
78 have /andP[x_nin_q' Iq'] : (x \notin q') && irred q'
79   by rewrite Eqc irred_edgeL in Iq.
80 have u_in_p' : u \in p' by rewrite path_begin.
81 have u_in_q : u \in q by rewrite uEu' Eqc mem_pcat_edgeL path_begin.
82 case: (split_at_last (p := p') u_in_q u_in_p') => v [p1] [p2] [def_p' v_in_q v_last].
83 case/(isplitP Iq) def_q' : _ / v_in_q => [q1 q2 Iq1 Iq2 Iv].
84 exists v.
85 have [v_in_p' v_in_q'] : v \in p' /\ v \in q
86   by rewrite def_p' def_q' 2!mem_pcat 2!path_end.
87 have xNv : x != v by
88   apply/eqP=> xEv; rewrite -xEv in v_in_p'; rewrite v_in_p' in x_nin_p'.
89 have vNy : v != y by
90   apply/eqP=> vEy; rewrite vEy in v_in_q'; rewrite v_in_q' in y_nin_q.
91 have vNz : v != z by
92   apply/eqP=> vEz; rewrite vEz in v_in_p';
93   rewrite Epc mem_pcat_edgeL v_in_p' negb_or andbF in z_nin_p.
94 case: (splitR q1 xNv) => vx [q1'] [vxv] def_q1.
95 case: (splitL p2 vNy) => vy [vyv] [p2'] [def_p2 _].
96 case: (splitL q2 vNz) => vz [vvz] [q2'] [def_q2 _].

```

```

97 have [hwy [hwz hwx]] : vy \in N(v) /\ vz \in N(v) /\ vx \in N(v) by
98   rewrite 3!in_opn vvy vvz sg_sym.
99 have [vx_in_q1 [vy_in_p2 vz_in_q2]] : vx \in q1 /\ vy \in p2 /\ vz \in q2 by
100   rewrite def_q1 def_p2 def_q2 3!mem_pcat 3!path_begin.
101 have vzNvx : vz != vx.
102 {
103   apply/eqP=> /esym vxEvz; rewrite vxEvz in vx_in_q1.
104   have vzEv := Iv vz vx_in_q1 vz_in_q2.
105   by have := (sg_edgeNeq vvz); rewrite vzEv eq_refl.
106 }
107 have vzNvy : vz != vy.
108 {
109   apply/eqP=> /esym vyEvz; rewrite vyEvz in vy_in_p2.
110   have vz_in_q : vz \in q by rewrite def_q' mem_pcat vz_in_q2.
111   have vyEv := v_last vz vz_in_q vy_in_p2.
112   by have := (sg_edgeNeq vvz); rewrite vyEv eq_refl.
113 }
114 have vxNvy : vx != vy.
115 {
116   apply/eqP=> vxEvy; rewrite vxEvy in vx_in_q1.
117   have vy_in_q : vy \in q by rewrite def_q' mem_pcat vx_in_q1.
118   have vyEv := v_last vy vy_in_q vy_in_p2.
119   by have := (sg_edgeNeq vvy); rewrite vyEv eq_refl.
120 }
121 rewrite -(ltn_add2r 1).
122 have -> : 2 + 1 = #|[set vz; vx; vy]|.
123 {
124   have [djzy djxy] :
125     [disjoint [set vz] & [set vy]] /\ [disjoint [set vx] & [set vy]]
126     by do 2 rewrite disjoint1 -in_setC in_setC1.
127   by rewrite (@cardsU _ [set vz; vx] _)
128     setIUl (disjoint_setIO djzy) (disjoint_setIO djxy)
129     setOU cards0 cards1 cards2 vzNvx.
130 }
131 rewrite addn1 ltnS.
132 apply/subset_leqif_cards.
133 by rewrite 2!subUset 3!subset 3!in_opn vvz sg_sym.
134 Qed.
135
136 Definition pathp_edge {G : sgraph} (x u v : G) (s : seq G) : Prop :=
137   exists i, (i < size s) && (nth x (x::s) i == u) && (nth x s i == v).

```

```

138
139 Definition Path_edge {G : sgraph} {x y : G} (p : Path x y) (u v : G) :=
140   pathp_edge x u v (tail p).
141
142 Section PathEdge.
143   Variables (G : sgraph) (x y z u v : G) (p : Path x y).
144
145   Lemma is_path_edge : Path_edge p u v -> u == v.
146   Proof.
147     rewrite/Path_edge/pathp_edge=> [[i] /andP[/andP[Hs /eqP <-] /eqP <-]].
148     move: (pathpW (valP p))=> /pathP Hp.
149     exact: Hp x i Hs.
150   Qed.
151
152   Lemma not_path_edge :
153     ~ u == v -> ~ Path_edge p u v.
154   Proof.
155     apply/contranot; exact: is_path_edge.
156   Qed.
157
158   Lemma vert_not_in_path :
159     (u \notin p) \/\ (v \notin p) -> ~ Path_edge p u v.
160   Proof.
161     case=> Hn [i /andP[/andP[Hs /eqP Hn1] /eqP Hnr]];
162     [have Hin : u \in p | have /(mem_tail x) Hin : v \in tail p].
163     - have /(mem_nth x) : i < size (x :: tail p) by
164       move/(ltn_addr 1): Hs;
165       rewrite [size (tail p) + 1]addnC
166         -[1 + size (tail p)]/(size([::x]) + size (tail p)) -size_cat.
167     by rewrite Hn1 -nodesE.
168     - by rewrite Hin in Hn.
169     - rewrite -Hnr; exact: (mem_nth x Hs).
170     - by rewrite -nodesE in Hin; rewrite Hin in Hn.
171   Qed.
172
173   Lemma edgep_path_edge (xy : x == y) :
174     (Path_edge (edgep xy) u v) <-> ((x == u) && (y == v)).
175   Proof.
176     rewrite/Path_edge/pathp_edge /=;
177     split=> [[i] | Heq]; by [case i | exists 0].
178   Qed.

```

```

179
180 Lemma cat_path_edge (q : Path y z) :
181   Path_edge (pcat p q) u v <=> Path_edge p u v \ / Path_edge q u v.
182 Proof.
183   rewrite/Path_edge -[tail (pcat p q)]/((tail p) ++ (tail q)) /pathp_edge size_cat;
184   split=> [[i] /andP[/andP[Hs +] +] []].
185   - rewrite nth_cat (nth_cat x (x :: tail p) (tail q) i).
186   case Hi: (i < size (tail p)).
187   + have Hlt : size (tail p) < size (x :: tail p) by [].
188     have -> := leq_ltn_trans (ltnW Hi) Hlt=> Hnl Hnr.
189     left; exists i; by rewrite Hi Hnl Hnr.
190   + move/negbT: Hi; rewrite -leqNgt -catls size_cat /= ltnS => Hi Hnl Hnr.
191     right; exists (i - size (tail p)).
192     rewrite -(ltn_subLR (size (tail q)) Hi) in Hs.
193     rewrite Hs (set_nth_default x y Hs) {}Hnr /= andbT.
194     rewrite leq_eqVlt in Hi; move/orP: Hi Hnl=> [/eqP Hi | Hi];
195     [move/esym/eq_leq: (Hi) | move/ltn_geF: (Hi)]=> ->.
196     - have/eqP H0 : (size (tail p) - size (tail p)) == 0 by rewrite subn_eq0.
197     by rewrite -Hi -last_nth path_last /= H0 nth0 => /eqP <-.
198     - rewrite -catls nth_cat -[_ < 1]/(_ == 0) subn1 subn_eq0 leqNgt Hi /=.
199     have Hlt : (i - size (tail p) - 1) < size (tail q)
200       by rewrite subn1; exact: (leq_ltn_trans (leq_pred (i - size (tail p))) Hs).
201     by rewrite (set_nth_default x y Hlt) addnC subnDA.
202   - case; move=> [i] /andP[/andP[Hi Hnl] Hnr];
203     [exists i | exists (size (tail p) + i)];
204     rewrite -catls catA 2!(@nth_cat _ _ (tail q) _) size_cat /=;
205     first by rewrite Hi (ltn_addl _ Hi) (ltn_addr _ Hi).
206     rewrite ltn_add2l addnC ltn_add2r (@ltnNge _ (size (tail p)))
207       leq_addl -[i + size (tail p) - size (tail p)]addnBA // =
208       subnn addn0 (set_nth_default y x Hi) Hi Hnr andTb andbT.
209   case Hs: (i == 0).
210   move/eqP: Hs Hnl=> -> /=.
211   rewrite add0n -(@catls _ x _) nth_cat /=.
212   case Hp: (size (tail p) < 1)=> /eqP <-;
213   last by rewrite subn1 nth_last path_last.
214   rewrite -[_ < 1]/(_ <= 0) in Hp.
215   have Hxy: x = y.
216   {
217     have := leq_trans (idx_mem (path_end p)) Hp;
218     rewrite -[_ <= 0]/(_ == 0) subn0 -(idx_start p);
219     move/eqP/esym=> +. apply (idx_inj (path_begin p)).

```

```

220     }
221     by move/eqP: Hp Hxy; rewrite subn0 => -> ->.
222     - move/eq0_lt0n: Hs Hn1=> Hs.
223       have HiltS : i - 1 < size (tail q)
224         by rewrite -ltn_predL -subn1 in Hs; exact: (ltn_trans Hs Hi).
225       rewrite -cat1S nth_cat ltnS leqNgt Hs /=.
226       by rewrite subnDr (set_nth_default y x HiltS).
227   Qed.
228
229 End PathEdge.
230
231 Lemma idp_path_edge {G : sgraph} (x u v : G) : Path_edge (idp x) u v -> False.
232 Proof. by rewrite/Path_edge/pathp_edge /= => [[]]. Qed.

```

A.4 EDGE COLORINGS

```

1  From HB Require Import structures.
2  From mathcomp Require Import all_boot.
3  From GraphTheory Require Import edone preliminaries digraph sgraph.
4  Require Import aux.
5
6  Set Warnings "-notation-overridden, -notation-incompatible-prefix".
7
8  Set Implicit Arguments.
9  Unset Strict Implicit.
10 Unset Printing Implicit Defensive.
11
12 Section EdgeColoring.
13   (* ---- Edge Coloring Functional Definition ---- *)
14   Variables (G : sgraph) (ColorType : finType).
15   Implicit Type (x : G).
16
17   (* An edge coloring function assigns edges in E(G) to colors *)
18   Definition edgeColoringType : Type := {set G} -> ColorType.
19   Implicit Type (c : edgeColoringType).
20
21   Definition is_proper_edge_coloring c : Prop :=
22     forall (x : G),
23     {in E{x}&, forall (e1 e2 : {set G}), c e1 = c e2 -> e1 = e2}.
24

```

```

25 Definition properEdgeColoringType : Type := { c | is_proper_edge_coloring c }.
26 Implicit Type (pc : properEdgeColoringType).
27
28 Coercion proper_to_edge_coloring
29   pc : edgeColoringType := proj1_sig pc.
30
31 Lemma single_colored_proper pc
32   {x y z : G} (xy : x -- y) (xz : x -- z) (c0 : ColorType)
33   :
34     (sval pc) [set x; y] = c0 ->
35     ((sval pc) [set x; z] = c0) <-> (y = z).
36 Proof.
37   have := (proj2_sig pc); rewrite/is_proper_edge_coloring=> is_pc xy_c0.
38   specialize (is_pc x [set x; y] [set x; z]).
39   split=> [z_eq| <- //].
40   rewrite z_eq in is_pc.
41   by apply/(doubleton_eq_left x)/is_pc; try apply/edge_neigh_self.
42 Qed.
43
44 Definition coloring_image c (E : {set {set G}}) : {set ColorType} := c @: E.
45 Local Notation "c [ E ]" := (coloring_image c E) (at level 50).
46
47 Lemma in_c_edge_neighP c x (c0 : ColorType) :
48   reflect (exists2 y, [set x; y] \in E(G) & c [set x; y] = c0) (c0 \in c[E{x}]).
49 Proof.
50   rewrite/edge_neigh; apply/(iffP idP)=>[/imsetP[e] /imsetP[y]||[y] He Hc].
51   - rewrite in_opn -in_edges=> Hin He Hc; rewrite He in Hc.
52     by exists y.
53   - apply/imsetP; exists [set x; y]=> //=.
54     apply/imsetP; exists y; by rewrite // in_opn -in_edges He.
55 Qed.
56
57 Lemma in_c_all_edgeP c (c0 : ColorType) :
58   reflect (exists2 e, e \in E(G) & c e = c0) (c0 \in c[E(G)]).
59 Proof.
60   apply/(iffP idP)=> [/imsetP[e] He /esym|[e] /edgesP [x] [y] [He Hxy]] Hc;
61   first by exists e.
62   rewrite -in_edges in Hxy; rewrite He in Hc.
63   have: (c0 \in c[E{x}]) by apply/in_c_edge_neighP; exists y.
64   rewrite -2!subset=> Hsub.
65   exact: (subset_trans Hsub (imsetS c (sub_all_edges x))).

```

```

66 Qed.
67
68 Lemma imset_c_del_edge c (e1 e2 : {set G}) :
69   e1 \in E(G) -> e2 \in E(G) ->
70   c e1 = c e2 ->
71   c [E(del_edges e1)] = c [E(del_edges e2)].
72 Proof.
73   rewrite/coloring_image=> e1_in_G e2_in_G eqc12.
74   case: (boolP (e1 == e2))=> [/eqP ->|| neq12].
75   apply/setP=> c0.
76   wlog suff H : e1 e2 e1_in_G e2_in_G eqc12 neq12 /
77   (c0 \in [set c e | e in E(del_edges e1)] ->
78    c0 \in [set c e | e in E(del_edges e2)]).
79   {
80     have H12 := H e1 e2 e1_in_G e2_in_G eqc12 neq12.
81     rewrite eq_sym in neq12.
82     have H21 := H e2 e1 e2_in_G e1_in_G (esym eqc12) neq12.
83     apply/idP/idP; [exact: H12 | exact: H21].
84   }
85   move=> /imsetP [e3 e3_in_E eq03]; apply/imsetP.
86   case: (boolP (e3 == e2))=> [/eqP eq23| neq23].
87   - exists e1; last by rewrite eqc12 eq03 eq23.
88     rewrite mem_del_edges e1_in_G.
89     exact: edges_eqn_sub e1_in_G e2_in_G neq12.
90   - exists e3; last by [].
91     move: e3_in_E; rewrite 2!mem_del_edges=> /andP[e3_in_G _].
92     rewrite e3_in_G.
93     exact: edges_eqn_sub e3_in_G e2_in_G neq23.
94 Qed.
95
96 Lemma leq_col_deg c x : #|c[E{x}]| <= max_degree G.
97 Proof.
98   apply: (leq_trans (leq_imset_card _ _)).
99   rewrite card_edge_neigh /max_degree.
100  exact: leq_bigmax_cond.
101 Qed.
102
103 Lemma eq_deg_pcol pc x : #|pc[E{x}]| = #|E{x}|.
104 Proof.
105   apply: card_in_imset.
106   exact: (proj2_sig pc).

```

```

107 Qed.
108
109 Lemma leq_vertex_graph c x : #|c[E{x}]| <= #|c[E(G)]|.
110 Proof.
111   exact: subset_leq_card (imsetS c (sub_all_edges x)).
112 Qed.
113
114 Lemma leq_maxdeg_pcol pc : max_degree G <= #|pc[E(G)]|.
115 Proof.
116   rewrite /max_degree max_deg_edge.
117   apply/bigmax_leqP=> x _.
118   apply: (leq_trans _ (leq_vertex_graph pc x)).
119   by rewrite (eq_deg_pcol pc x).
120 Qed.
121
122 End EdgeColoring.
123 Notation "c [ E ]" := (coloring_image c E) (at level 50).
124
125 Section ChromIdx.
126   Variables (G : sgraph).
127   Implicit Types (k chi : nat).
128
129   (* A k-edge-coloring is a proper coloring which uses exactly k colors *)
130   Definition kEdgeColoringType k : Type :=
131     { ColorType : finType &
132       { c : properEdgeColoringType G ColorType | #|c[E(G)]| == k } }.
133
134   Coercion k_to_proper_coloring {k} (kc : kEdgeColoringType k) :
135     properEdgeColoringType G (projT1 kc) :=
136     proj1_sig (projT2 kc).
137
138   Definition proper_to_k_coloring
139     {ColorType : finType} (pc : properEdgeColoringType G ColorType)
140   : kEdgeColoringType #|pc[E(G)]| := (existT _ ColorType (exist _ pc (eqxx _))).
141
142   Definition card_k_col {k} (kc : kEdgeColoringType k) :
143     #|kc[E(G)]| = k := eqP (proj2_sig (projT2 kc)).
144
145   (* G is k-colorable if a k-edge-coloring exists. *)
146   Definition k_edge_colorable k : Prop := inhabited (kEdgeColoringType k).
147

```



```

148  (* The chromatic index chi is the smallest k such that G is k-colorable *)
149  Definition is_chromatic_index chi : Prop :=
150    k_edge_colorable chi /\ forall k, k < chi -> ~ k_edge_colorable k.
151
152  (* We can already lower bound the chromatic index *)
153  Theorem chi_lower_bound chi :
154    is_chromatic_index chi ->
155    max_degree G <= chi.
156  Proof.
157    do 3![elim]=> ColorType is_chiEC _.
158    elim: is_chiEC=> ? /eqP <-.
159    by rewrite leq_maxdeg_pcol.
160  Qed.
161
162  (* Any valid k-edge-coloring upper bounds chi *)
163  Lemma chi_upper_bound k chi :
164    is_chromatic_index chi ->
165    k_edge_colorable k ->
166    chi <= k.
167  Proof.
168    move=> [Hchi_color Hchi_min] kc.
169    rewrite leqNgt.
170    apply/negP=> Hlt.
171    have nkc : ~ k_edge_colorable k := Hchi_min _ Hlt.
172    exact: nkc kc.
173  Qed.
174
175  Lemma chi_upper_bound_trans k chi :
176    is_chromatic_index chi ->
177    (exists n, k_edge_colorable n /\ n <= k) ->
178    chi <= k.
179  Proof.
180    move=> ? [n] [n_lt_k].
181    have chi_lt_n : chi <= n by exact/chi_upper_bound.
182    exact (leq_trans chi_lt_n n_lt_k).
183  Qed.
184
185  (* ---- One-to-one Coloring ---- *)
186
187  (* injective coloring: each edge is a color *)
188  Definition injEdgeColoringType : edgeColoringType G {set G} :=

```

```

189     fun e => e.
190
191 Definition proper_inj_coloring : properEdgeColoringType G {set G}.
192 Proof.
193   exists injEdgeColoringType.
194   by move=> _ e1 e2 _ _ eq.
195 Defined.
196
197 Lemma imset_inj : proper_inj_coloring[E(G)] = E(G).
198 Proof.
199   apply/setP=> e.
200   apply/imsetP/idP=> [[e']] He' -> |He|; last by exists e.
201   by rewrite /proper_inj_coloring /injEdgeColoringType.
202 Qed.
203
204 Definition inj_k_coloring : kEdgeColoringType #|E(G)|.
205 Proof.
206   exists {set G}, proper_inj_coloring; by rewrite imset_inj.
207 Defined.
208
209 (* Thus, all graphs have a k-edge-coloring with k = #|E(G)| *)
210 Lemma inj_chrom : k_edge_colorable #|E(G)|.
211 Proof.
212   constructor; exact inj_k_coloring.
213 Qed.
214
215 (* If chi is a chromatic index of G, then chi <= |E(G)| *)
216 Corollary chromatic_index_le_edges chi (is_chi : is_chromatic_index chi) :
217   chi <= #|E(G)|.
218 Proof.
219   exact: (chi_upper_bound is_chi inj_chrom).
220 Qed.
221
222 (* TO THINK: could also prove chromatic index exists and is unique *)
223 End ChromIdx.
224
225 Section AbsentSet.
226 Variables (G : sgraph).
227 Implicit Types (x y : G).
228
229 Definition absent_set {ColorType : finType}

```

```

230 (c : edgeColoringType G ColorType) x :=
231   setD (c[E(G)]) (c[E{x}]).
232
233 Lemma absent_in_imset
234   {ColorType : finType} (c : edgeColoringType G ColorType) (c0 : ColorType) x
235   :
236   c0 \in absent_set c x -> c0 \in c[E(G)].
237 Proof.
238   by move=>/setDP [? _].
239 Qed.
240
241 Lemma absent_edge
242   {ColorType : finType} (c : edgeColoringType G ColorType) (c0 : ColorType) x y
243   :
244   c0 \in absent_set c x -> y \in N(x) -> c0 != c [set x; y].
245 Proof.
246   move=> /setDP[_ /memPnC Hnin] ?.
247   have in_cneigh : c [set x; y] \in c[E{x}].
248   { by apply/imsetP; exists [set x; y]; first by apply/imsetP; exists y. }
249   apply Hnin; exact: in_cneigh.
250 Qed.
251
252 Lemma absent_edge_sym
253   {ColorType : finType} (c : edgeColoringType G ColorType) (c0 : ColorType) x y
254   :
255   c0 \in absent_set c x -> x \in N(y) -> c0 != c [set y; x].
256 Proof.
257   have -> : [set y; x] = [set x; y] by rewrite doubleton_eq_iff; right.
258   rewrite in_opn sg_sym -in_opn.
259   exact: absent_edge.
260 Qed.
261
262 Lemma absent_del_edge {ColorType : finType} (c : edgeColoringType G ColorType) x y z :
263   [set x; y] \in E(G) -> x != z -> y != z ->
264   c [set x; y] \notin c[E(del_edges [set x; y])] ->
265   c [set x; y] \in absent_set c z.
266 Proof.
267   rewrite /absent_set=> xy_in xNz yNz w0p; apply/setDP; split;
268   first by apply/imsetP; exists [set x; y].
269   apply/negP=> /imsetP [e e_at_z cxy].
270   apply: (negP w0p); apply/imsetP; exists e; last by [].

```

```

271     rewrite mem_del_edges.
272     apply/andP; split;
273     first by exact: (subsetP (sub_all_edges z) _ e_at_z).
274     apply/subsetPn.
275     move/edgesSetP: e_at_z => [w [-> _]].
276     exists z; first by rewrite !inE eqxx.
277     by rewrite !inE negb_or eq_sym xNz eq_sym.
278 Qed.
279
280 Proposition exists_absent_color {k : nat} (kc : kEdgeColoringType G k):
281   max_degree G + 1 <= k ->
282   forall x : G, exists c, c \in (absent_set kc x).
283 Proof.
284   rewrite addn1=> ? x; apply/set0Pn.
285   rewrite -card_gt0 cardsDS; last by apply imsetS; apply sub_all_edges.
286   by rewrite subn_gt0 (card_k_col kc) (leq_ltn_trans (leq_col_deg kc x)).
287 Qed.
288
289 End AbsentSet.
290
291 Section ExtendCol.
292   Variables (G : sgraph) (del_e : {set G}) (He : del_e \in E(G)).
293
294   Definition extendedColType
295     {ColorType : finType}
296     (c : edgeColoringType (del_edges del_e) ColorType)
297   : edgeColoringType G (option ColorType) :=
298     fun e => if e == del_e then None else Some (c e).
299
300   Lemma proper_extended_col
301     {ColorType : finType}
302     (pc : properEdgeColoringType (del_edges del_e) ColorType)
303   : is_proper_edge_coloring (extendedColType pc).
304 Proof.
305   move:pc => [c Hp] x f0 f1 Hf0 Hf1.
306   rewrite /extendedColType.
307   case H00: (f0 == del_e); case H10 : (f1 == del_e) => //.
308   - move/eqP: H00=> ->; move/eqP: H10 => -> //.
309   move=> [Heq]; apply (Hp x); last by [];
310   move/negbT: H00=> H00; move/negbT: H10=> H10;
311   move/(subsetP (sub_all_edges x)): (Hf0)=> Hf0G;

```

```

312   move/(subsetP (sub_all_edges x)): (Hf1)=> Hf1G;
313   move: (edges_eqn_sub Hf0G He H00)=> Hsub0;
314   move: (edges_eqn_sub Hf1G He H10)=> Hsub1;
315   by rewrite (@del_edges_edge_neigh G del_e _).
316 Qed.
317
318 Lemma card_extended_col
319   {k : nat}
320   (kc : kEdgeColoringType (del_edges del_e) k)
321 : #|extendedColType kc[E(G)]| = k + 1.
322 Proof.
323   rewrite /extendedColType /coloring_image.
324   rewrite (del_edges1 He) imsetU1 eq_refl.
325   have -> :
326     [set (if e == del_e then None else Some ((sval (sval (projT2 kc))) e))
327       | e in E(del_edges del_e)]
328     = Some @: (kc[E(del_edges del_e)]).
329   {
330     rewrite -imset_comp.
331     apply/setP=> c0.
332     apply/imsetP/imsetP=> [[e He' ->] | [e He' ->]];
333     exists e=> //; rewrite ifF //;
334     apply/negbTE/negP=> /eqP eEd;
335     have := del_edgesN del_e; rewrite -{1}eEd;
336     by rewrite He'.
337   }
338   rewrite cardsU1 card_imset; last by exact: Some_inj.
339   rewrite card_k_col.
340   have -> : None \notin [set Some x | x in kc [E(del_edges del_e)]];
341   { by apply/imsetP => [[x _ /eqP //]]. }
342   by rewrite addnC.
343 Qed.
344
345 (* extendedColType of a k-edge-coloring produces a (k+1)-edge-coloring *)
346 Definition k_extended_col
347   {k : nat}
348   (kc : kEdgeColoringType (del_edges del_e) k)
349 : kEdgeColoringType G (k + 1).
350 Proof.
351   exists (option (projT1 kc)),
352   (exist _ (extendedColType kc) (@proper_extended_col _ kc));

```

```

353   by rewrite (card_extended_col kc).
354   Defined.
355
356   Lemma extended_absent
357     {k : nat}
358     {kc : kEdgeColoringType (del_edges del_e) k}
359     {c0 : projT1 kc}
360     {x : G}
361   : c0 \in absent_set kc x -> Some c0 \in absent_set (k_extended_col kc) x.
362   Proof.
363     rewrite/absent_set=> /setDP[c0_in_kc c0_nin_kc].
364     apply/setDP.
365   Admitted.
366
367   End ExtendCol.
368
369   Section Recolor.
370   Variables (G : sgraph) (ColorType : finType) (c : edgeColoringType G ColorType).
371   Implicit Types (e f : {set G}).
372
373   Definition recolor_edge e c0 : edgeColoringType G ColorType :=
374     fun edge => if edge == e then c0 else c edge.
375
376   Lemma recolor_eq e c0 : (recolor_edge e c0) e = c0.
377   Proof. by rewrite /recolor_edge eqxx. Qed.
378
379   Lemma recolor_neq e f c0 : f != e -> (recolor_edge e c0) f = c f.
380   Proof. by rewrite /recolor_edge=> /negPf ->. Qed.
381
382   Lemma del_edges_imset_recolor e c0 :
383     c[E(del_edges e)] = recolor_edge e c0 [E(del_edges e)].
384   Proof.
385     apply/setP => c1.
386     by apply/imsetP/imsetP; move=> [e2 He2 ->]; rewrite /recolor_edge;
387     exists e2 => //; case: ifP => /eqP Heq //;
388     rewrite Heq in He2; move: (del_edgesN e); rewrite -in_setC => /setCP.
389   Qed.
390
391   Lemma imset_recolor e c0 :
392     c0 \in c[E(G)] ->
393     recolor_edge e c0 [E(G)] \subset c[E(G)].

```

```

394 Proof.
395   rewrite/coloring_image/recolor_edge=> /imsetP [e1] e1_in_G def_c0.
396   apply/subsetP=> c1 /imsetP [e2] e2_in_G.
397   case: ifP=> _ def_c1; apply/imsetP;
398   last by exists e2.
399   by exists e1; rewrite -def_c1 in def_c0.
400 Qed.
401
402 Lemma recolor_proper (x y : G) c0 :
403   is_proper_edge_coloring c ->
404   c0 \in (absent_set c x :&: absent_set c y) ->
405   is_proper_edge_coloring (recolor_edge [set x; y] c0).
406 Proof.
407   rewrite/recolor_edge in_setI=> Hp /andP[Hax Hay].
408   move: Hp=> Hp z e1 e2 Hz1 Hz2.
409   move: (Hz1) (Hz2); rewrite/edge_neigh=> /imsetP [w1] Hw1 Ezw1 /imsetP [w2] Hw2 Ezw2.
410   rewrite Ezw1 Ezw2.
411   case: ifP=> /eqP /doubleton_eq_iff;
412   [case; move=> [H1 H2]|rewrite 4!(rwP eqP) !(rwP andP) (rwP orP) (rwP negP) negb_or];
413   (case: ifP=> /eqP /doubleton_eq_iff;
414   [case; move=> [H3 H4]| rewrite 4!(rwP eqP) 2!(rwP andP) (rwP orP) (rwP negP) negb_or]);
415   rewrite doubleton_eq_iff 5!(rwP eqP) 2!(rwP andP) (rwP orP).
416   - by rewrite H2 H4 3!eq_refl.
417   - by rewrite H2 H4 -H3 H1 2!eq_refl.
418   - rewrite -H1 in Hax=> _ /eqP Hc0.
419     have Hcontra := absent_edge Hax Hw2.
420     by rewrite Hc0 eqxx in Hcontra.
421   - by rewrite 2!eq_refl H2 -H3 H1 -H4 eq_refl.
422   - by rewrite H2 H4 3!eq_refl.
423   - rewrite -H1 in Hay=> _ /eqP Hc0.
424     have Hcontra := absent_edge Hay Hw2.
425     by rewrite Hc0 eqxx in Hcontra.
426   - rewrite -H3 in Hax=> _ /eqP Hc0.
427     have Hcontra := absent_edge Hax Hw1.
428     by rewrite Hc0 eqxx in Hcontra.
429   - rewrite -H3 in Hay=> _ /eqP Hc0.
430     have Hcontra := absent_edge Hay Hw1.
431     by rewrite Hc0 eqxx in Hcontra.
432   - rewrite Ezw1 in Hz1; rewrite Ezw2 in Hz2.
433     rewrite eq_refl=> _ _ /eqP /(Hp z _ _ Hz1 Hz2) /doubleton_eq_left ->.
434     by rewrite eq_refl.

```

```

435 Qed.
436
437 Lemma replace_col e c0 :
438   e \in E(G) ->
439   c0 \in c[E(del_edges e)] ->
440   c e \notin c[E(del_edges e)] ->
441   #|recolor_edge e c0 [E(G)]| = #|c[E(G)]| - 1.
442 Proof.
443   move=> e_in_e.
444   rewrite (del_edges1 e_in_e).
445   move: (del_edges_imset_recolor e c0).
446   rewrite /coloring_image 2!imsetU1 2!cardsU1 recolor_eq => -> -> -> /.
447   by rewrite add0n add1n subn1.
448 Qed.
449
450 Definition swap_edge e f : edgeColoringType G ColorType :=
451   fun edge =>
452     if edge == e then c f
453     else if edge == f then c e
454     else c edge.
455
456 Lemma swap_edge_eq {e f} : e == f -> swap_edge e f =1 c.
457 Proof.
458   move=> /eqP -> e'.
459   rewrite /swap_edge.
460   case Heq: (e' == f) => //.
461   by move /eqP: Heq ->.
462 Qed.
463
464 Lemma imset_swap e f :
465   e \in E(G) ->
466   f \in E(G) ->
467   c[E(G)] = (swap_edge e f)[E(G)].
468 Proof.
469   move=> He Hf; apply /setP=> c0.
470   apply /imsetP/imsetP; move=> [e2 He2 ->]; rewrite /swap_edge;
471   exists (if e2 == e then f else if e2 == f then e else e2) => //;
472   repeat case: ifP => //; repeat move=> /eqP -> //; try rewrite eq_refl //.
473   - do 2 move=> _ -> //.
474   - move=> _ /eqP -> //.
475 Qed.

```



```

476
477 (* Same proof as above*)
478 Lemma imset_swap_vertex e f (x : G) :
479   e \in E{x} ->
480   f \in E{x} ->
481   c[E{x}] = (swap_edge e f)[E{x}].
482 Proof.
483   move=> He0 He1; apply/setP=> c0.
484   apply/imsetP/imsetP; move=> [e2 He2 ->]; rewrite /swap_edge;
485   exists (if e2 == e then f else if e2 == f then e else e2) => //;
486   repeat case: ifP=> //; repeat move=> /eqP -> //; try rewrite eq_refl //.
487   - do 2 move=> _ -> //.
488   - move=> _ /eqP -> //.
489 Qed.
490
491 Lemma imset_swap_nin e f (x : G) :
492   x \notin e -> x \notin f ->
493   c[E{x}] = (swap_edge e f)[E{x}].
494 Proof.
495   move=> x_nin_e x_nin_f; apply/setP=> c0.
496   apply/imsetP/imsetP; move=> [e2 e2_in_G ->]; rewrite /swap_edge;
497   exists e2=> //; move: e2_in_G; rewrite mem_edge_graph=> /andP[_ x_in_e2];
498   by (case: ifP=> [/eqP contra]);
499     [by rewrite contra in x_in_e2; rewrite x_in_e2 in x_nin_e];
500     (case: ifP=> [/eqP contra]);
501     [by rewrite contra in x_in_e2; rewrite x_in_e2 in x_nin_f]].
502 Qed.
503
504 Lemma swap_absent_nin (e f : {set G}) (x : G) :
505   x \notin e -> x \notin f -> e \in E(G) -> f \in E(G) ->
506   absent_set (swap_edge e f) x = absent_set c x.
507 Proof.
508   rewrite/absent_set=> x_nin_e x_nin_f e_in_G f_in_G.
509   by rewrite (imset_swap e_in_G f_in_G) (imset_swap_nin x_nin_e x_nin_f).
510 Qed.
511
512 Lemma swap_proper_vertex (x y z : G) :
513   is_proper_edge_coloring c ->
514   (c [set x; y]) \in absent_set c z ->
515   (c [set x; z]) \in absent_set c y ->
516   y \in N(x) -> z \in N(x) ->

```

```

s17   is_proper_edge_coloring (swap_edge [set x; y] [set x; z]).
s18 Proof.
s19   rewrite /is_proper_edge_coloring /swap_edge
s20     => pc abs_z abs_y y_at_x z_at_x u e1 e2
s21     /edgesSetP [v1] [def_e1 uv1] /edgesSetP [v2] [def_e2 uv2].
s22   rewrite def_e1 def_e2.
s23   case: (boolP (v1 == v2))=> [/eqP ->| v1Nv2]; first by [].
s24   have [yNx zNx] : ((y == x) = false) /\ ((z == x) = false)
s25   by move: y_at_x z_at_x; do 2 rewrite in_opn sg_sym=> /sg_edgeNeq ->.
s26   (* case if (u, v1) = (x, y) *)
s27   case: ifP=> /eqP /doubleton_eq_iff.
s28   - move=> [[uEx v1Ey] | [uEy v1Ex]].
s29     + rewrite uEx in uv1 uv2 def_e1 def_e2 *.
s30       rewrite v1Ey in v1Nv2 uv1 def_e1 *.
s31       case: ifP=> [/eqP /doubleton_eq_left v2Ny | _];
s32       first by rewrite v2Ny eq_refl in v1Nv2.
s33       case: ifP=> /eqP /doubleton_eq_left v2Ez /esym.
s34       * rewrite v2Ez in def_e2 uv2 *.
s35         exact: (pc x [set x; y] [set x; z] (edge_neigh_self uv1) (edge_neigh_self uv2)).
s36       * by rewrite in_opn in z_at_x=>
s37         /(pc x [set x; v2] [set x; z] (edge_neigh_self uv2)
s38         (edge_neigh_self z_at_x)) /doubleton_eq_left.
s39     + rewrite uEy.
s40       case: ifP=> [/eqP /doubleton_eq_iff [[yEx _] | [_ ->]] | _];
s41       try rewrite v1Ex //; first by rewrite yEx eq_refl in yNx.
s42       case: ifP=> [/eqP /doubleton_eq_iff | _].
s43       * rewrite 4!(rwP eqP) 2!(rwP andP) yNx -v1Ex=> /orP.
s44       rewrite /= => /andP[_ /eqP v1Ev2]; by rewrite v1Ev2 eq_refl in v1Nv2.
s45       * rewrite -in_opn uEy in uv2=> cEc.
s46       have := absent_edge abs_y uv2; by rewrite cEc eq_refl.
s47   - case: ifP=> /eqP /doubleton_eq_iff.
s48   + move=> [[uEx v1Ez] | [uEz v1Ex]].
s49     * rewrite uEx v1Ez in uv1 uv2 *.
s50       case: ifP=> [/eqP /doubleton_eq_left v2Ey _ /esym|];
s51       first by rewrite v2Ey in uv2 *;
s52       exact: (pc x [set x; z] [set x; y] (edge_neigh_self uv1) (edge_neigh_self uv2)).
s53       case: ifP=> [/eqP /doubleton_eq_left -> //|_ /eqP /doubleton_eq_left v2Ny _].
s54       by rewrite in_opn in y_at_x
s55         => /(pc x _ _ (edge_neigh_self y_at_x) (edge_neigh_self uv2))
s56         /doubleton_eq_left /esym.
s57     * rewrite uEz in uv2 *.

```

```

558     case: ifP=>[/eqP /doubleton_eq_iff _ _ cEc];
559     first by have := absent_edge_sym abs_z z_at_x; rewrite cEc eq_refl.
560     case: ifP=> [/eqP /doubleton_eq_iff|_ _ _ cEc];
561     last by rewrite -in_opn in uv2; have := absent_edge abs_z uv2;
562         rewrite cEc eq_refl.
563     rewrite 4!(rwP eqP) 2!(rwP andP) zNx -v1Ex eq_refl=> /orP.
564     rewrite /= => /eqP v1Ev2.
565     by rewrite v1Ev2 eq_refl in v1Nv2.
566 + case: ifP=> [/eqP /doubleton_eq_iff [[uEx v2Ey] | [uEy v2Ex]] + + cEc]].
567     -- rewrite uEx v2Ey in uv1 cEc *.
568     rewrite in_opn in z_at_x.
569     have /doubleton_eq_left -> :=
570         pc x _ _ (edge_neigh_self uv1) (edge_neigh_self z_at_x) cEc.
571     by rewrite 4!(rwP eqP) 2!(rwP andP) 2!eq_refl=> /orP.
572     -- rewrite uEy in uv1 cEc *.
573     rewrite -in_opn in uv1.
574     have := absent_edge abs_y uv1.
575     by rewrite cEc eq_refl.
576 * case: ifP=>[/eqP /doubleton_eq_iff [[uEx v2Ez] | [uEz v2Ex]] + + +|_ _ _ _] cEc.
577     -- rewrite uEx v2Ez in uv1 cEc * => _ _ .
578     rewrite in_opn in y_at_x.
579     have /doubleton_eq_left -> :=
580         pc x _ _ (edge_neigh_self uv1) (edge_neigh_self y_at_x) cEc.
581     by rewrite 4!(rwP eqP) 2!(rwP andP) 2!eq_refl=> /orP.
582     -- rewrite uEz in uv1 cEc *.
583     rewrite -in_opn in uv1.
584     have := absent_edge abs_z uv1.
585     by rewrite cEc eq_refl.
586     exact: (pc u _ _ (edge_neigh_self uv1) (edge_neigh_self uv2) cEc).
587 Qed.
588
589 End Recolor.

```

A.5 FANS AND ROTATION

```

1 From HB Require Import structures.
2 From mathcomp Require Import all_boot.
3 From GraphTheory Require Import edone preliminaries digraph sgraph.
4 Require Import aux edge_coloring.
5 From Equations Require Import Equations.

```

```

6
7 Set Warnings "-notation-overridden, -notation-incompatible-prefix".
8
9 Set Implicit Arguments.
10 Unset Strict Implicit.
11 Unset Printing Implicit Defensive.
12
13 Section Fan.
14   Variable (G : sgraph) (ColorType : finType).
15   Implicit Types (c : edgeColoringType G ColorType) (v wk w : G) (e : {set G}) (f : seq G).
16
17   (* 1. For all w in the fan centered at v, w is in the neighborhood of v *)
18   Definition neigh_prop v f := all (fun w => w \in N(v)) f.
19
20   (* 2. if w0 is the first item in fan f centered at v under coloring c,
21      (v, w0) is a distinct color from the rest of the edges in the graph *)
22   Definition w0_prop
23     {ColorType} (c : edgeColoringType G ColorType) e
24   := c e \notin c[E(del_edges e)].
25
26   Lemma w0_prop_extended {e} (c_del : edgeColoringType (del_edges e) ColorType)
27   : w0_prop (extendedColType c_del) e.
28   Proof.
29     rewrite /w0_prop /extendedColType eq_refl.
30     by apply/negP => /imsetP [e' /del_edges1_neq /negbTE ->].
31   Qed.
32
33   Lemma w0_col_extended {e} (c_del : edgeColoringType (del_edges e) ColorType)
34   : (extendedColType c_del) e = None.
35   Proof. by rewrite /extendedColType eq_refl. Qed.
36
37   (* 3. for all w_i, w_{i+1} in the fan f centered at v under coloring c,
38      the color of (v, w_{i+1}) is absent at w_i *)
39   Definition absent_prop c e w :=
40     (c e) \in (absent_set c w).
41
42   Definition fanp c f v w0 wk :=
43     (last wk f == w0) &&
44     uniq (wk::f) &&
45     neigh_prop v (wk::f) &&
46     w0_prop c [set v; w0] &&

```

```

47   path (
48     fun x2 => absent_prop c [set v; x2]
49   ) wk f.
50
51 Lemma fanpW c f v w0 wk :
52   fanp c f v w0 wk -> path (fun x2 => absent_prop c [set v; x2]) wk f.
53 Proof. by case/andP. Qed.
54
55 Lemma fanpW_rev c f v w0 wk :
56   fanp c f v w0 wk ->
57   path (fun x2 x1 => absent_prop c [set v; x1] x2) (last wk f) (rev (belast wk f)).
58 Proof.
59   move=> /fanpW. by rewrite rev_path.
60 Qed.
61
62 Lemma fanp_last c f v w0 wk : fanp c f v w0 wk -> last wk f = w0.
63 Proof. by case/andP=> /andP[/andP[/andP[/eqP-> _] _] _]. Qed.
64
65 Lemma fanp_uniq c f v w0 wk : fanp c f v w0 wk -> uniq (wk::f).
66 Proof. by case/andP=> /andP[/andP[/andP[_ ->] _] _]. Qed.
67
68 Lemma fanp_w0_prop c f v w0 wk : fanp c f v w0 wk -> w0_prop c [set v; w0].
69 Proof. by case/andP=> /andP [_ ->] _]. Qed.
70
71 Lemma fanp_neigh c f v w0 wk : fanp c f v w0 wk -> neigh_prop v (wk::f).
72 Proof. by case/andP => /andP [/andP [_ ->] _] _]. Qed.
73
74 Lemma rev_neigh f v wk : neigh_prop v (wk::f) -> neigh_prop v (rev (wk::f)).
75 Proof. by rewrite /neigh_prop all_rev. Qed.
76
77 Definition valid_fan_vertex {c f v w0 wk} (fan : fanp c f v w0 wk) (w : G) :=
78   (w \in N(v)) && (w \notin wk::f) && absent_prop c [set v; w] wk.
79
80 Lemma fanp_cons {c f v w0 wk} (fan : fanp c f v w0 wk) (w : G) :
81   valid_fan_vertex fan w ->
82   fanp c (wk::f) v w0 w.
83 Proof.
84   by move: fan;
85   rewrite /fanp last_cons /neigh_prop /valid_fan_vertex
86   => /andP[/andP[/andP[/andP[H1 Hu] Hn]] -> Hp] /andP[/andP[Hin Hnin] Ha].
87 Qed.

```

```

88
89 Lemma sub_fanp
90   {c f v w0 wk}
91   {f1 f2 : seq G} {w}
92   (Hcat : (wk::f) = f1 ++ (w :: f2))
93   : fanp c f v w0 wk -> fanp c f2 v w0 w.
94 Proof.
95   rewrite/fanp -(last_cons wk wk) Hcat cat_uniq
96     /neigh_prop all_cat -cat_rcons last_cat last_rcons
97   => /andP[/andP[/andP[/andP[-> /andP[_ /andP[_ ->]]] /andP[_ ->]] ->] +].
98   case: f1 Hcat=> [|wk' f1|.
99     - by rewrite cat0s; case=> <- <-.
100    - rewrite cat_cons; case=> _ ->.
101    by rewrite -cat_rcons cat_path last_rcons=> /andP[_ ->].
102 Qed.
103
104 End Fan.
105
106 Section Pack.
107   Variables (G : sgraph) (ColorType : finType).
108   Implicit Types (c : edgeColoringType G ColorType) (v w : G).
109
110   Section FanDef.
111     Variables (c : edgeColoringType G ColorType) (v w0 w : G).
112
113     Record Fan : predArgType := { fval : seq G; _ : fanp c fval v w0 w }.
114
115     HB.instance Definition _ := [isSub for fval].
116     HB.instance Definition _ := [Countable of Fan by <:].
117
118   End FanDef.
119 End Pack.
120
121 Section FanOps.
122   Variables (G : sgraph) (ColorType : finType).
123   Implicit Types (k : nat) (c : edgeColoringType G ColorType) (fs : seq G).
124
125   Fixpoint rotate c fs (v : G) : edgeColoringType G ColorType :=
126     match fs with
127     | w0 :: ((w1::tl) as ws) =>
128       rotate

```

```

129         (swap_edge c [set v; w0] [set v; w1])
130         WS v
131     | _ => c
132 end.
133
134 Lemma Fan_of_proof
135   {v w0 : G}
136   (c_del : edgeColoringType (del_edges [set v; w0]) ColorType)
137 : v -- w0 -> fanp (extendedColType c_del) [::] v w0 w0.
138 Proof.
139   by rewrite /fanp (w0_prop_extended c_del) -in_opn.
140 Qed.
141
142 Definition Fan_of_del_edges
143   {v w0 : G}
144   (He : v -- w0)
145   (c_del : edgeColoringType (del_edges [set v; w0]) ColorType)
146 := Build_Fan (Fan_of_proof c_del He).
147
148 Lemma k_Fan_of_proof
149   {k} {v w0 : G}
150   (He : [set v; w0] \in E(G))
151   (kc_del : kEdgeColoringType (del_edges [set v; w0]) k)
152 : fanp (k_extended_col He kc_del) [::] v w0 w0.
153 Proof.
154   rewrite /fanp (w0_prop_extended kc_del) //=.
155   by have -> : w0 \in N(v) by move: (He); rewrite in_edges in_opn.
156 Qed.
157
158 (* TO THINK: Is there a way to avoid this (just use fan_of...) *)
159 Definition k_Fan_of_del_edges
160   {k} {v w0 : G}
161   (He : [set v; w0] \in E(G))
162   (kc_del : kEdgeColoringType (del_edges [set v; w0]) k)
163 := Build_Fan (k_Fan_of_proof He kc_del).
164 End FanOps.
165
166 Section Rotation.
167 Variables (G : sgraph) (ColorType : finType) (c : edgeColoringType G ColorType)
168           (v w0 wk : G) (f : Fan c v w0 wk).
169 Implicit Type (w : G).

```

```

170
171 Lemma fanW : path (fun x2 => absent_prop c [set v; x2]) wk (fval f).
172 Proof. move: (valP f); exact: fanpW. Qed.
173
174 Lemma fan_last : last wk (val f) = w0.
175 Proof. move: (valP f); exact: fanp_last. Qed.
176
177 Lemma fan_rev : rev (wk::val f) = w0::(rev (belast wk (fval f))).
178 Proof. rewrite lastI rev_rcons. congr (_ :: _). exact: fan_last. Qed.
179
180 Lemma fanW_rev :
181   path (fun x2 x1 => absent_prop c [set v; x1] x2) w0 (rev (belast wk (fval f))).
182 Proof. move: (valP f)=> /fanpW_rev. by rewrite fan_last. Qed.
183
184 Lemma fan_uniq : uniq (wk::val f).
185 Proof. move: (valP f); exact: fanp_uniq. Qed.
186
187 Lemma fan_neigh : neigh_prop v (wk::val f).
188 Proof. move: (valP f); exact: fanp_neigh. Qed.
189
190 Lemma fan_w0_prop : w0_prop c [set v; w0].
191 Proof. move: (valP f); exact: fanp_w0_prop. Qed.
192
193 Lemma in_neigh w : w \in (wk::val f) -> w \in N(v).
194 Proof.
195   move: fan_neigh; rewrite/neigh_prop=> /allP H. exact: H.
196 Qed.
197
198 Definition fancons {w} (H : valid_fan_vertex (valP f) w) := Build_Fan (fanp_cons H).
199
200 Lemma sub_fan {f1 f2 : seq G} w (Hcat : (wk::val f) = f1 ++ (w :: f2)) : Fan c v w0 w.
201 Proof.
202   move: (valP f)=> Hf. exact: (Build_Fan (sub_fan Hcat Hf)).
203 Qed.
204
205 Definition rotateF : edgeColoringType G ColorType :=
206   rotate c (rev (wk::val f)) v.
207
208 Lemma rot_notin (e : {set G}) :
209   e \in E(G) -> e \notin E{v} -> c e = rotateF e.
210 Proof.

```



```

211   rewrite /rotateF mem_edge_graph=> -> /= e_nin_v; set fs := (rev (wk::val f)).
212   elim: fs c=> [|w1 [|w2 wss] IH] d //.
213   change (d e = rotate (swap_edge d [set v; w1] [set v; w2]) (w2 :: wss) v e).
214   rewrite -(IH (swap_edge d [set v; w1] [set v; w2])).
215   rewrite /swap_edge. move: e_nin_v.
216   case: ifP=> [/eqP -> /negP|]; first by rewrite set21.
217   case: ifP=> [/eqP -> /negP|//]; by rewrite set21.
218 Qed.
219
220 Lemma imset_rot_vertex : c[E{v}] = rotateF[E{v}].
221 Proof.
222   rewrite /rotateF; set fs := (rev (wk::val f)).
223   have Hws : neigh_prop v fs by apply rev_neigh; exact: fan_neigh.
224   elim: fs c Hws=> [|w1 ws IH] d // = /andP [Hw1 Hws].
225   case: ws IH Hws=> [|w2 wss IH] Hws //.
226   rewrite -(IH (swap_edge d [set v; w1] [set v; w2])) //.
227   move/andP: Hws => [Hw2 _].
228   have He1 : [set v; w1] \in E{v} by rewrite/edge_neigh; apply/imsetP; exists w1.
229   have He2 : [set v; w2] \in E{v} by rewrite/edge_neigh; apply/imsetP; exists w2.
230   exact: imset_swap_vertex He1 He2.
231 Qed.
232
233 (* Basically the same as above *)
234 Lemma imset_rot : c[E(G)] = rotateF[E(G)].
235 Proof.
236   rewrite/rotateF; set fs := (rev (wk::val f)).
237   have Hws: neigh_prop v fs by apply rev_neigh; exact: fan_neigh.
238   elim: fs c Hws=> [|w1 [|w2 wss] IH] d // = /andP [Hw1 Hws].
239   rewrite -(IH (swap_edge d [set v; w1] [set v; w2])) //.
240   move/andP: Hws => [Hw2 _].
241   have He1: [set v; w1] \in E(G) by rewrite in_opn -in_edges in Hw1.
242   have He2: [set v; w2] \in E(G) by rewrite in_opn -in_edges in Hw2.
243   exact: imset_swap He1 He2.
244 Qed.
245
246 Lemma imset_rot_del_edge (e0 e1 : {set G}) :
247   e0 \in E(G) -> e1 \in E(G) ->
248   c e0 = rotateF e1 ->
249   c[E(del_edges e0)] = rotateF[E(del_edges e1)].
250 Proof.
251   rewrite /coloring_image=> e0_in_G e1_in_G.

```

```

252   rewrite/rotateF; set fs := (rev (wk::val f)).
253   elim: fs c=> [/=|w1 [/=|w2 wss] IH] d.
254   - exact: imset_c_del_edge.
255   - exact: (IH d).
256   rewrite -[rotate d [:: w1, w2 & wss] v]/(rotate (swap_edge d [set v; w1] [set v; w2])
257       (w2 :: wss) v)=> de0.
258   specialize (IH (swap_edge d [set v; w1] [set v; w2])).
259 Admitted.
260
261 Lemma rotate_last (fs : seq G) (d : edgeColoringType G ColorType) :
262   d [set v; head w0 fs] = rotate d fs v [set v; last w0 fs].
263 Proof.
264   elim: fs d=> [/=|w1 [w2 wss] IH] d'; try by [].
265   change (d' [set v; w1] =
266       rotate (swap_edge d' [set v; w1] [set v; w2]) (w2 :: wss) v [set v; last w2 wss]
267   ).
268   rewrite -(IH (swap_edge d' [set v; w1] [set v; w2])) /=.
269   rewrite /swap_edge;
270   case: ifP=> [/eqP /doubleton_eq_left -> //[]].
271   by rewrite eq_refl.
272 Qed.
273
274 Lemma rot_first_last : c [set v; w0] = rotateF [set v; wk].
275 Proof.
276   rewrite /rotateF.
277   set fs := (rev (wk::val f)).
278   have -> : w0 = (head w0 fs) by rewrite /fs fan_rev.
279   have -> : wk = (last w0 fs) by rewrite /fs rev_cons last_rcons.
280   exact: (rotate_last fs c).
281 Qed.
282
283 Lemma rot_w0_prop : w0_prop rotateF [set v; wk].
284 Proof.
285   have +: w0_prop c [set v; w0] by exact: fan_w0_prop.
286   rewrite/w0_prop.
287   have vw0_in_G : [set v; w0] \in E(G).
288   {
289     rewrite in_edges -in_opn.
290     apply/in_neigh.
291     by have := mem_last wk (\val f); rewrite fan_last.
292   }

```

```

293   have vwk_in_G : [set v; wk] \in E(G).
294   {
295     rewrite in_edges -in_opn.
296     apply/in_neigh.
297     by rewrite mem_head.
298   }
299   have Heq: c [E(del_edges [set v; w0])] = rotateF [E(del_edges [set v; wk])]
300     := imset_rot_del_edge vw0_in_G vwk_in_G rot_first_last.
301   by rewrite rot_first_last Heq.
302   Qed.
303
304   (* TO THINK: Arguably doesn't need to be it's own lemma *)
305   Lemma card_rot :
306     #|c[E(G)]| = #|rotateF[E(G)]|.
307   Proof. by rewrite imset_rot. Qed.
308
309   Lemma rot_absent_center : absent_set c v = absent_set rotateF v.
310   Proof.
311     by rewrite/absent_set imset_rot imset_rot_vertex.
312   Qed.
313
314   Lemma rot_absent_fan w c0 :
315     w \in (wk::val f) ->
316     c0 \in (absent_set c v :&: absent_set c w) ->
317     c0 \in (absent_set rotateF v :&: absent_set rotateF w).
318   Proof.
319     rewrite /absent_set imset_rot imset_rot_vertex=>
320       /in_neigh w_at_v /setIP[/setDP[c0_in_r c0_nin_v] /setDP[_ /negP c0_nin_w]].
321     apply/setIP; split; apply/setDP; split; try by assumption.
322     apply/negP=> /imsetP [e /edgesSetP [y] [def_e wy]] cNr.
323     have /andP[/andP[e_at_w e_at_y] wNy] : (e \in E{G;w}) && (e \in E{G;y}) && (w != y)
324       by apply/edge_neigh_edge; rewrite def_e wy.
325     apply: c0_nin_w.
326     have e_in_E : [set w; y] \in E(G)
327       by move: e_at_w; rewrite mem_edge_graph def_e=> /andP[-> _].
328     apply/in_c_edge_neighP; exists y=> //.
329     case: (boolP (y == v)) => [/eqP yEv | yNv].
330     - exfalso. move: c0_nin_v=> /negP c0_nin_v.
331       apply: c0_nin_v. apply/in_c_edge_neighP; rewrite -yEv.
332       have set_refl : [set w; y] = [set y; w] by apply/doubleton_eq_iff; right.
333       rewrite set_refl in def_e e_in_E.

```

```

334     exists w=> //.
335     by rewrite cNr def_e.
336     rewrite cNr def_e.
337     apply/rot_notin/negP=> // /imsetP [x] x_at_v /doubleton_eq_iff [[wEv yEx] | [_ yEv]].
338     - move: w_at_v. by rewrite in_opn=> /sg_edgeNeq; rewrite wEv eq_refl.
339     by rewrite yEv eq_refl in yNv.
340 Qed.
341
342 Lemma rot_proper :
343   is_proper_edge_coloring c ->
344   is_proper_edge_coloring rotateF.
345 Proof.
346   rewrite/rotateF fan_rev.
347   set fs := rev (belast wk (val f)).
348   have nps: neigh_prop v (w0 :: fs)
349     by rewrite /fs -fan_rev; apply rev_neigh; exact: fan_neigh.
350   have abs_fs: path (fun x2 x1 => absent_prop c [set v; x1] x2) w0 fs :=
351     fanW_rev; rewrite /absent_prop in abs_fs.
352   have w0p: w0_prop c [set v; w0] := fan_w0_prop; rewrite /w0_prop in w0p.
353   elim: fs c w0 abs_fs w0p nps
354     => [|w1 [|w2 wss] IH] d x0 /andP[abs_w01 abs_fs] x0p
355       /andP[x0_at_v /andP[w1_at_v nps]] pc_d.
356   - rewrite/rotate.
357     apply/(swap_proper_vertex pc_d (absent_del_edge _ _ _ _) abs_w01 x0_at_v w1_at_v);
358     rewrite //; first by rewrite in_edges -in_opn.
359     + by move: w1_at_v; rewrite in_opn=> /sg_edgeNeq ->.
360     + (* contradiction if x0 = w1 from abs_w01 *) admit.
361   apply: (IH (swap_edge d [set v; x0] [set v; w1]) w1).
362 Admitted.
363
364 End Rotation.
365
366 Section MaximalFan.
367   Variables (G : sgraph) (ColorType : finType) (c : edgeColoringType G ColorType) (v w0 : G).
368   Implicit Types (wk : G) (ck : ColorType).
369
370   Definition is_fanmax {wk} (f : Fan c v w0 wk) : Prop :=
371     forall w, ~~ valid_fan_vertex (valP f) w.
372
373   Equations fanmax {wk} (f : Fan c v w0 wk)
374     : {w & Fan c v w0 w} by wf #|N(v) : \: [set x in wk :: val f] | lt :=

```

```

375     fanmax f :=
376     match pickP (valid_fan_vertex (valP f)) with
377     | Pick w Pw => fanmax (fancons Pw)
378     | Nopick _ => existT _ wk f
379     end.
380 Next Obligation.
381   apply/ltP/proper_card/properP. rewrite (set_cons w _).
382   split.
383   - exact/setDS/subsetU1.
384   - have /andP[/andP[Hin Hnin] _] := Pw.
385     by exists w; rewrite 2!inE // in_set1 inE eq_refl Hin.
386 Qed.
387
388 Lemma fanmax_is_max {wk} (f : Fan c v w0 wk)
389 : is_fanmax (projT2 (fanmax f)).
390 Proof.
391   rewrite/is_fanmax=> w.
392   funelim (fanmax f).
393   case: pickP=> [wp Pw | Np].
394   - exact (H wp Pw w).
395   - by rewrite (Np w).
396 Qed.
397
398 Lemma fanmax_present
399   {ck} {wk} (f : Fan c v w0 wk)
400   (Hf : is_fanmax f)
401   (Hnab : ck \notin absent_set c v)
402   (Hab : ck \in absent_set c wk)
403   :
404   exists wj,
405   [&& [set v; wj] \in E(G), c [set v; wj] == ck & wj \in (wk :: val f)].
406 Proof.
407   have Hinc: ck \in c[E(G)] by rewrite/absent_set in_setD in Hab; exact: proj2 (andP Hab).
408   have Hatv: ck \in c[E{v}] by exact: notin_setD Hinc Hnab.
409   move/in_c_edge_neighP: Hatv=> [wj] Hine Hc.
410   exists wj; rewrite/is_fanmax in Hf; move: (Hf wj); rewrite/valid_fan_vertex.
411   have -> : wj \in N(v) by rewrite in_opn -in_edges.
412   by rewrite /absent_prop Hine Hc Hab eq_refl /= andbT negbK.
413 Qed.
414
415 End MaximalFan.

```

A.6 KEMPE CHAINS AND KEMPE SWAPS

```

1  From HB Require Import structures.
2  From mathcomp Require Import all_boot.
3  From GraphTheory Require Import edone preliminaries digraph sgraph.
4  Require Import aux_spath edge_coloring fans.
5  From Equations Require Import Equations.
6
7  Set Warnings "-notation-overridden, -notation-incompatible-prefix".
8
9  Set Implicit Arguments.
10 Unset Strict Implicit.
11 Unset Printing Implicit Defensive.
12
13 Section KempeGraph.
14   Variables (G : sgraph) (ColorType : finType)
15             (c : edgeColoringType G ColorType) (ca cb : ColorType).
16   Implicit Types (x y : G) (e : {set G}).
17
18   (* A Kempe graph will consist of all ca-cb kempe chains in the graph *)
19   Definition kempe_rel :=
20     [rel x y : G | x -- y && ((c [set x; y] == ca) || (c [set x; y] == cb))].
21
22   Lemma kempe_sym : symmetric kempe_rel.
23   Proof. by move => x y /<=>; by rewrite sgP setUC. Qed.
24
25   Lemma kempe_irrefl : irreflexive kempe_rel.
26   Proof. by move => x /<=>; rewrite sgP. Qed.
27
28   Definition kempe_graph := @SGraph G kempe_rel kempe_sym kempe_irrefl.
29
30   Lemma mem_kempe e : e \in E(kempe_graph) = (e \in E(G)) && ((c e == ca) || (c e == cb)).
31   Proof.
32     apply/edgesP/andP => [[x] [y] [-> /andP[xy xyc]][]]; first by rewrite in_edges.
33     by move/edgesP => [x] [y] [-> xy xyNA]; exists x,y; split => //; apply/andP.
34   Qed.
35
36   Lemma kempe_opn x z :
37     z \in N(kempe_graph;x) = (z \in N(G;x)) && ((c [set x; z] == ca) || (c [set x; z] == cb))
38   Proof. by rewrite !inE -in_edges mem_kempe in_edges. Qed.
39

```

```

40 Lemma kempe_sub : E(kempe_graph) \subset E(G).
41 Proof. by apply/subsetP=> e; rewrite mem_kempe => /andP[]. Qed.
42
43 Lemma kempe_edgeN e : (c e != ca) -> (c e != cb) -> e \notin E(kempe_graph).
44 Proof. by rewrite mem_kempe negb_and negb_or=> -> ->. Qed.
45
46 (* A kempe chain is a single component of a kempe graph containing a given vertex *)
47 Definition kempe_chain x := @component_of kempe_graph x.
48
49 Lemma edge_in_component (H : sgraph) (u v : H) :
50   [set u; v] \in E(H) -> v \in component_of u.
51 Proof.
52   rewrite in_edges=> /edgeP p.
53   by apply/components_pblockP; exists p.
54 Qed.
55
56 End KempeGraph.
57
58 Section InvertGraph.
59   Variables (G : sgraph) (ColorType : finType)
60             (c : edgeColoringType G ColorType) (ca cb : ColorType).
61
62   Definition invertedKempeGraph (e : {set G}) :=
63     if (e \in E(kempe_graph c ca cb)) then
64       if (c e == ca) then cb else ca
65     else c e.
66
67   Lemma mem_invert_ab :
68     (ca \in c[E(G)] <-> cb \in invertedKempeGraph[E(G)]) /\
69     (cb \in c[E(G)] <-> ca \in invertedKempeGraph[E(G)]).
70   Proof.
71     rewrite/invertedKempeGraph.
72     split;
73     (split=> [/in_c_all_edgeP [e] in_e ce_abP|/in_c_all_edgeP [e]]);
74     [
75       (* forward direction *)
76       move/eqP: (ce_abP)=> ce_ab;
77       apply/in_c_all_edgeP; exists e; first by [];
78       have ->: e \in E(kempe_graph c ca cb) by rewrite mem_kempe in_e ce_ab
79     ]
80     (* reverse direction *)

```

```

81     rewrite mem_kempe=> in_e; rewrite in_e /=;
82     case ce_a: (c e == ca)=> /=; move/eqP: ce_a=> ce_a;
83     case ce_b: (c e == cb)=> /=; move/eqP: ce_b=> ce_b;
84     move=> contra; apply/in_c_all_edgeP; exists e=> //
85   ]);
86   first by rewrite ce_ab.
87   - rewrite contra in ce_a; contradiction.
88   - by case ce_a: (c e == ca)=> /=; move/eqP: ce_a=> ce_a; rewrite ce_abP in ce_a.
89   - by rewrite contra.
90   Qed.
91
92 End InvertGraph.
93
94 Section InvertChain.
95   Variables (G : sgraph) (ColorType : finType) (c : edgeColoringType G ColorType)
96             (ca cb : ColorType) (x : G) (chain : {set kempe_graph c ca cb}) (u : G).
97   Hypothesis chain_x : chain = kempe_chain c ca cb x.
98
99   (* technically, e \in subset kc could be enough, but this will be easier to reason with *)
100   Definition invertedChain (e : {set G}) :=
101     if ((e \in E(kempe_graph c ca cb)) && (e \subset chain)) then
102       if (c e == ca) then cb else ca
103     else c e.
104
105   Lemma imset_invert_sub :
106     ca \in c[E(G)] ->
107     cb \in c[E(G)] ->
108     invertedChain[E(G)] \subset c[E(G)].
109   Proof.
110     rewrite /invertedChain
111       => /in_c_all_edgeP [ea] ea_in_e ea_ca /in_c_all_edgeP [eb] eb_in_e eb_cb.
112     apply/subsetP=> c0 /in_c_all_edgeP [e] e_in_e <-.
113     apply/in_c_all_edgeP.
114     case e_in_chain: ((e \in E(kempe_graph c ca cb)) && (e \subset chain));
115     last by exists e; rewrite // e_in_chain.
116     move: e_in_chain; rewrite mem_kempe=> /andP[/andP[_ /orP[/eqP ->|/eqP ->]] e_in_chain];
117     [rewrite eq_refl | case ca_cb : (cb == ca); last by exists ea]; by exists eb.
118   Qed.
119
120   Lemma not_kempe_edge e :
121     (e \notin E(kempe_graph c ca cb)) \/\ ~ (e \subset chain) ->

```



```

122   c e = invertedChain e.
123 Proof.
124   by case=> []/negP not_in; rewrite /invertedChain (negbTE not_in).
125 Qed.
126
127 Lemma is_kempe_edge e :
128   (e \in E(kempe_graph c ca cb)) ->
129   (e \subset chain) ->
130   (c e = ca <-> invertedChain e = cb) /\
131   (c e = cb <-> invertedChain e = ca).
132 Proof.
133   rewrite /invertedChain mem_kempe=> /andP[-> /orP[e_c | e_c]] ->;
134   split; split; move/eqP: (e_c)=> ->; rewrite eq_refl //<= orbT /=;
135   first by move=> ->; rewrite eq_refl.
136   all: case eq_ab: (cb == ca); by move/eqP: eq_ab.
137 Qed.
138
139 Lemma in_inverted_absent :
140   u \in chain ->
141   (ca \in absent_set c u <-> cb \in absent_set invertedChain u) /\
142   (cb \in absent_set c u <-> ca \in absent_set invertedChain u).
143 Proof.
144 Admitted.
145
146 Lemma notin_inverted_absent :
147   u \notin chain ->
148   c[E(G)] = invertedChain[E(G)] ->
149   absent_set c u = absent_set invertedChain u.
150 Proof.
151 Admitted.
152
153 Lemma w0_inverted (w0 : G) :
154   c [set u; w0] != ca -> c [set u; w0] != cb ->
155   w0_prop c [set u; w0] -> w0_prop invertedChain [set u; w0].
156 Proof.
157   rewrite /w0_prop /coloring_image=> /negbTE cNa /negbTE cNb /negP c_nin_E.
158   apply/negP=> /imsetP [e] e_in_E eq_i.
159   apply: c_nin_E; apply/imsetP.
160   exists e=> //.
161   have cEi : c [set u; w0] = invertedChain [set u; w0].
162   {

```

```

163     apply/not_kempe_edge; left.
164     apply/negP=> +.
165     by rewrite (@mem_kempe G _ c ca cb [set u; w0]) cNa cNb /= andbF.
166   }
167   move: (eq_i).
168   rewrite -cEi /invertedChain.
169   case: (boolP ((e \in E(kempe_graph c ca cb)) && (e \subset chain)))=> //.
170   case: ifP=> _ _ contra; by move: cNa cNb; rewrite contra eq_refl.
171   Qed.
172
173   End InvertChain.
174
175   Section KempeProper.
176   Variables (G : sgraph) (ColorType : finType) (pc : properEdgeColoringType G ColorType)
177             (ca cb : ColorType) (x : G) (chain : {set kempe_graph pc ca cb}).
178   Implicit Types (u : G).
179   Hypothesis chain_x : chain = kempe_chain pc ca cb x.
180
181   Lemma injective_in_kempe u :
182     {in E{kempe_graph pc ca cb;u} &, injective (proj1_sig pc)}.
183   Proof.
184     move=> e1 e2; move: ((proj2_sig pc) u e1 e2).
185     rewrite 4!mem_edge_graph 2!(@mem_kempe G _ pc ca cb _)
186       => Hc /andP[/andP[? ?] ?] /andP[/andP[? ?] ?].
187     exact: Hc.
188   Qed.
189
190   Lemma max_deg_kempe u :
191     #|N(kempe_graph pc ca cb;u)| <= 2.
192   Proof.
193     rewrite -card_edge_neigh.
194     rewrite -(card_in_imset (@injective_in_kempe u)).
195     apply: (@leq_trans #|[set ca; cb]| _ _); last first.
196     - rewrite cards2; by case: (ca == cb).
197     apply/subset_leq_card/subsetP=> col /imsetP [e + ->].
198     by rewrite !inE mem_edge_graph (@mem_kempe G _ pc ca cb _)=> /andP[/andP[_ ->] _].
199   Qed.
200
201   Lemma min_deg_kempe u v :
202     u \in chain -> v \in chain -> u != v ->
203     0 < #|N(kempe_graph pc ca cb;u)| /\ 0 < #|N(kempe_graph pc ca cb;v)|.

```

```

204 Proof.
205   rewrite chain_x /kempe_chain=> /same_component comp_u v_in uNv.
206   have u_connect : connected (@component_of (kempe_graph pc ca cb) u) :=
207     @connected_component_of (kempe_graph pc ca cb) u.
208   have u_in_u : u \in @component_of (kempe_graph pc ca cb) u :=
209     @in_component_of (kempe_graph pc ca cb) u.
210   have v_in_u : v \in @component_of (kempe_graph pc ca cb) u by rewrite -comp_u in v_in.
211   case: (path_in_connected u_connect u_in_u v_in_u)=> p Ip _.
212   split; apply/card_gt0P.
213   - case: (splitL p uNv)=> w [+] _.
214     by exists w; rewrite in_opn.
215   - case: (splitR p uNv) => w [] [wv] _.
216     by exists w; rewrite in_opn sg_sym.
217 Qed.
218
219 Lemma inverted_proper :
220   is_proper_edge_coloring (invertedChain chain).
221 Proof.
222   have := proj2_sig pc.
223   rewrite /is_proper_edge_coloring /invertedChain => is_pc u e1 e2 e1_in_e e2_in_e.
224   specialize (is_pc u e1 e2 e1_in_e e2_in_e);
225   case e1_in_K: (e1 \in E(kempe_graph pc ca cb)); case e1_in_C: (e1 \subset chain);
226   case e2_in_K: (e2 \in E(kempe_graph pc ca cb)); case e2_in_C: (e2 \subset chain)=> /;
227   try by exact: is_pc.
228   - move: e1_in_K e2_in_K; rewrite 2!mem_kempe.
229     case: ifP=> /eqP c_e1 /andP[e1_in_G /= /eqP ca_cb_e1]; [move: c_e1 | move: ca_cb_e1];
230     case: ifP=> /eqP c_e2 + /andP[e2_in_G /= /eqP ca_cb_e2] ca_cb;
231     rewrite -?ca_cb -?ca_cb_e2 -?c_e2; exact: is_pc.
232   - have contra : e2 \subset chain.
233     {
234       move: e1_in_e e2_in_e e1_in_K e2_in_K e1_in_C e2_in_C
235       => /edgesSetP [y1] [uy1 _] /edgesSetP [y2] [uy2 _].
236       rewrite uy1 uy2 2!subUset 3!subset chain_x /kempe_chain
237       => /edge_in_component _ /edge_in_component y2_in_u /andP[u_in_x _] _.
238       have <- := (@same_component (kempe_graph pc ca cb) u x u_in_x).
239       by rewrite (@in_component_of (kempe_graph pc ca cb) u) y2_in_u.
240     }
241   by rewrite contra in e2_in_C.
242 1,2:
243   move: e2_in_e e2_in_K;
244   rewrite mem_kempe mem_edge_graph=> /andP[-> _] /negbT;

```

```

245     rewrite /= negb_or=> /andP[ca_e2 cb_e2];
246     case: ifP=> _ contra; move: ca_e2 cb_e2; by rewrite contra eq_refl.
247 - have contra : e1 \subset chain.
248   {
249     move: e1_in_e e2_in_e e1_in_K e2_in_K e1_in_C e2_in_C
250     => /edgesSetP [y1] [uy1 _] /edgesSetP [y2] [uy2 _].
251     rewrite uy1 uy2 2!subUset 3!subset chain_x /kempe_chain
252     => /edge_in_component y1_in_u /edge_in_component _ _ /andP[u_in_x _].
253     have <- := (@same_component (kempe_graph pc ca cb) u x u_in_x).
254     by rewrite (@in_component_of (kempe_graph pc ca cb) u) y1_in_u.
255   }
256   by rewrite contra in e1_in_C.
257 all:
258   move: e1_in_e e1_in_K;
259   rewrite mem_kempe mem_edge_graph=> /andP[-> _] /negbT;
260   rewrite /= negb_or=> /andP[ca_e1 cb_e1];
261   case: ifP=> _ contra; move: ca_e1 cb_e1; by rewrite contra eq_refl.
262 Qed.
263
264 Definition chain_endpt u :=
265   (ca \in absent_set pc u) \/\ (cb \in absent_set pc u).
266
267 Lemma chain_endptP u :
268   (ca \in absent_set pc u) \/\ (cb \in absent_set pc u) -> chain_endpt u.
269 Proof. by []. Qed.
270
271 Lemma deg_chain_endpt u :
272   chain_endpt u -> #|N(kempe_graph pc ca cb;u)| <= 1.
273 Proof.
274   rewrite -card_edge_neigh /chain_endpt -(card_in_imset (@injective_in_kempe u));
275   case=> /absent_edge abs; [set cset := [set cb] | set cset := [set ca]];
276   (apply: (@leq_trans #|cset| _ _); last by rewrite cards1);
277   apply/subset_leq_card/subsetP=> col /imsetP [e + ->];
278   rewrite mem_edge_graph (@mem_kempe G _ pc ca cb _)
279     => /andP[/andP[/edgesP [y] [z] [def_e yz] /orP[+|//]]];
280   move: (yz); rewrite sg_sym -in_opn=> zy; rewrite -in_opn in yz;
281   rewrite def_e=> /eqP ce /set2P[eq_u | eq_u];
282   rewrite -eq_u in yz zy;
283   try by rewrite inE -ce.
284   - have := abs z yz; by rewrite -ce eq_u eq_refl.
285   - have := abs y zy; by rewrite -ce eq_u setUC eq_refl.

```

```

286   - have := abs z yz; by rewrite -ce eq_u eq_refl.
287   - have := abs y zy; by rewrite -ce eq_u setUC eq_refl.
288 Qed.
289
290 Proposition chain_two_endpts (u v w : G) :
291   u != v -> v != w -> u != w ->
292   chain_endpt u /\ chain_endpt v /\ chain_endpt w ->
293   (u \notin chain) \/ (v \notin chain) \/ (w \notin chain).
294 Proof.
295   case: (boolP (u \in chain)) => [u_in|u_nin]; last by left.
296   case: (boolP (v \in chain)) => [v_in|v_nin]; last by right; left.
297   case: (boolP (w \in chain)) => [w_in|w_nin]; last by right; right.
298   move=> uNv vNw uNw [/deg_chain_endpt + [/deg_chain_endpt + /deg_chain_endpt +]].
299   rewrite leq_eqVlt (@leq_eqVlt #|N(kempe_graph pc ca cb;v)| _)
300     (@leq_eqVlt #|N(kempe_graph pc ca cb;w)| _)
301     ltnNge (@ltnNge #|N(kempe_graph pc ca cb;v)| _)
302     (@ltnNge #|N(kempe_graph pc ca cb;w)| _).
303   have [[-> ->] [_ ->]] :=
304     conj (min_deg_kempe u_in v_in uNv) (min_deg_kempe u_in w_in uNw).
305   move: (u_in) (v_in) (w_in); rewrite chain_x /kempe_chain 3!orbF
306     => /same_component comp_u + /eqP degu /eqP degv /eqP degw.
307   have u_in_u : u \in (@component_of (kempe_graph pc ca cb)) u
308     by exact: (@in_component_of (kempe_graph pc ca cb) u).
309   have u_connect : connected (@component_of (kempe_graph pc ca cb) u)
310     by exact: connected_component_of.
311   rewrite -comp_u=> v_in_u w_in_u.
312   have [p Ip p_in_u] := path_in_connected u_connect u_in_u v_in_u.
313   have [q Iq q_in_u] := path_in_connected u_connect u_in_u w_in_u.
314   have [y /ltn_geF] :=
315     @shared_interior3 (kempe_graph pc ca cb) _ _ _ _ _ degu degv degw uNv vNw uNw Ip Iq.
316   by have -> := max_deg_kempe y.
317 Qed.
318
319 (*
320   Technically, the following is also true less hypotheses.
321   But since we only use this in the proof of vizing's theorem,
322   we make this more specific (to not repeat our work from there)
323   *)
324
325 Proposition inverted_fan
326   {w0 wi wj w : G} {f1 f2 : seq G}

```

```

327 (imset_pc_ic : pc [E(G)] = (invertedChain chain) [E(G)])
328 (fan : Fan pc x w0 w)
329 (fsplit : w :: \val fan = f1 ++ [:: wj, wi & f2])
330 (jN0 : wj != w0)
331 (pc_xa_cb : cb \in absent_set pc x)
332 (pc_wia_ca : ca \in absent_set pc wi)
333 (pc_wa_ca : ca \in absent_set pc w)
334 (pc_xwj_ca : (sval pc) [set x; wj] = ca)
335 :
336   exists (w' : G) (ifan : Fan (invertedChain chain) x w0 w'),
337   (ca \in absent_set (invertedChain chain) w').
338 Proof.
339   rewrite -[wj :: wi :: f2]cats1s catA in fsplit.
340   have wj_at_x : wj \in N(x).
341   {
342     apply/(@in_neigh _ _ _ _ _ fan); rewrite fsplit.
343     by rewrite mem_cat cats1 mem_rcons in_cons eq_refl.
344   }
345   have w0_at_x : w0 \in N(x).
346   {
347     apply/(@in_neigh _ _ _ _ _ fan).
348     by have := mem_last w (\val fan); rewrite fan_last.
349   }
350   have w0_ifan : w0_prop (invertedChain chain) [set x; w0].
351   {
352     apply/(w0_inverted _ _ _ (fan_w0_prop fan)).
353     - rewrite -pc_xwj_ca. move: (wj_at_x).
354       suff : (sval pc [set x; w0]) \in absent_set pc wj;
355       first by exact: absent_edge_sym.
356       rewrite eq_sym in jN0.
357       apply/(absent_del_edge _ _ (jN0)); first by rewrite in_edges -in_opn.
358       + move: wj_at_x; by rewrite in_opn=> /sg_edgeNeq /negbT.
359       exact: (fan_w0_prop fan).
360     - rewrite eq_sym. apply/(absent_edge pc_xa_cb w0_at_x).
361   }
362   case: (boolP (wi \in chain))=> wi_in_x.
363   - have /andP[xNwi /andP[wiNw xNw]] : [&& x != wi, wi != w & x != w].
364     {
365       have w_in_f : w \in w::val fan by rewrite in_cons eq_refl.
366       have eq_fmax : val fan = behead ((f1 ++ [:: wj]) ++ wi :: f2) by rewrite -fsplit.
367       have wi_in_fmax : wi \in val fan.

```

```

368 {
369   rewrite eq_fmax; case: f1 fsplit eq_fmax=> [|hd tl] _;
370   first by rewrite cat0s cat1s /behead in_cons eq_refl.
371   by rewrite -catA cat_cons /behead catA mem_cat in_cons eq_refl.
372 }
373 have wi_in_f : wi \in w::val fan by rewrite in_cons wi_in_fmax.
374 have := fan_uniq fan; rewrite cons_uniq=> /andP[w_nin_fmax _].
375 case: (boolP (x != wi))=> [xNwi|/negbNE /eqP xEwi];
376 [rewrite andTb; case: (boolP (wi != w))=> [wiNw|/negbNE /eqP wiEw];
377   [rewrite andTb; case: (boolP (x != w))=> [xNw|/negbNE /eqP xEw]=> //;
378     have := in_neigh w_in_f;
379     rewrite in_opn xEw=> /sg_edgeNeq /eqP; contradiction
380     | rewrite andFb; rewrite wiEw in wi_in_fmax; by rewrite (wi_in_fmax) in w_nin_fmax]
381 |
382   rewrite andFb; have := in_neigh wi_in_f;
383   rewrite in_opn xEwi=> /sg_edgeNeq /eqP; contradiction
384 ].
385 }
386 have x_in_x := (@in_component_of (kempe_graph pc ca cb) x).
387 have w_nin_x : w \notin chain by
388   case
389     (@chain_two_endpts x wi w xNwi wiNw xNw
390      (conj (chain_endptP (or_intror pc_xa_cb))
391       (conj (chain_endptP (or_introl pc_wia_ca))
392        (chain_endptP (or_introl pc_wa_ca))))
393     ));
394 [rewrite chain_x /kempe_chain x_in_x | case; [rewrite wi_in_x]]].
395
396 have ifan : fanp (invertedChain chain) (\val fan) x w0 w.
397 {
398   rewrite /fanp fan_last eq_refl fan_uniq fan_neigh w0_ifan //=.
399   admit.
400 }
401 exists w, (Build_Fan ifan).
402 by rewrite (notin_inverted_absent chain_x w_nin_x imset_pc_ic) in pc_wa_ca.
403 - rewrite (notin_inverted_absent chain_x wi_in_x imset_pc_ic) in pc_wia_ca.
404 have fsmaller := sub_fan fsplit.
405 have ifan : fanp (invertedChain chain) (\val fsmaller) x w0 wi.
406 {
407   rewrite /fanp fan_last eq_refl fan_uniq fan_neigh w0_ifan //=.
408   admit.

```

```

409     }
410     by exists wi, (Build_Fan ifan).
411   Admitted.
412
413 End KempeProper.

```

A.7 ALTERNATING PATHS

```

1  From HB Require Import structures.
2  From mathcomp Require Import all_boot.
3  From GraphTheory Require Import edone preliminaries digraph sgraph.
4  Require Import aux spath edge_coloring fans.
5  From Equations Require Import Equations.
6
7  Set Warnings "-notation-overridden, -notation-incompatible-prefix".
8
9  Set Implicit Arguments.
10 Unset Strict Implicit.
11 Unset Printing Implicit Defensive.
12
13 Section AltPathOps.
14   Variables (G : sgraph) (ColorType : finType).
15   Implicit Types (c : edgeColoringType G ColorType) (ca cb : ColorType) (s : seq G).
16
17   Fixpoint alternates c ca cb s : bool :=
18     match s with
19     | x :: ((y :: tl) as s') =>
20       (c [set x; y] == ca) && alternates c cb ca s'
21     | _ => true
22     end.
23
24   Fixpoint next_col c ca cb s : ColorType :=
25     match s with
26     | _ :: s' => next_col c cb ca s'
27     | _ => cb
28     end.
29
30   Fixpoint alternates_invert c ca cb s : edgeColoringType G ColorType :=
31     match s with
32     | x :: ((y::tl) as s') =>

```



```

33     alternates_invert
34         (recolor_edge c [set x; y] ca)
35     cb ca s'
36 | _ => c
37 end.
38 End AltPathOps.
39
40 Section AltPath.
41 Variables (G : sgraph) (ColorType : finType) (c : edgeColoringType G ColorType) (x y z : G)
42 Implicit Types (ca cb : ColorType) (s : seq G) (p : Path x y) (zx: z -- x) (yz: y -- z).
43
44 Lemma alternates_rcons ca cb yz p :
45     alternates c ca cb (nodes (pcat p (edgep yz))) =
46     alternates c ca cb (nodes p) && (c [set y; z] == next_col c ca cb (nodes p)).
47 Proof.
48   elim: p yz.
49   Admitted.
50
51 Lemma alternate_cons ca cb zx p :
52     alternates c ca cb (nodes (pcat (edgep zx) p)) =
53     (c [set z; x] == ca) && alternates c cb ca (nodes p).
54 Proof.
55   rewrite nodes_pcat !nodesE.
56   by case: (val p).
57 Qed.
58
59 Lemma alternates_ca_cb ca cb p :
60     ((alternates c ca cb (nodes p)) && (alternates c cb ca (nodes p))) ->
61     ((ca == cb) || (x == y)).
62 Proof.
63   Admitted.
64   (* move: x; elim p=> [//| y p'] IH.
65     rewrite -{3}cat1s cat_nilp=> x.
66     rewrite 2!alternate_cons=> /andP[/andP[/eqP Hca Hab] /andP[/eqP Hcb Hac]].
67     by have ->: (ca == cb) by rewrite -Hca Hcb.
68   Qed. *)
69
70 Definition altpath ca cb {u v : G} (p0 : Path u v) :=
71     alternates c ca cb (nodes p0) && (irred p0).
72
73 Lemma altpathE ca cb {u v : G} (p0 : Path u v) :

```

```

74   altpath ca cb p0 = alternates c ca cb (nodes p0) && (irred p0).
75   Proof. by []. Qed.
76
77   Definition altpath_next_col {ca cb p} (ap : altpath ca cb p) : ColorType :=
78     next_col c ca cb (nodes p).
79
80   Lemma altpath_idp ca cb : altpath ca cb (idp x).
81   Proof. by rewrite altpathE nodesE irred_idp. Qed.
82
83   Lemma altpath_edge ca cb xy : (@altpath ca cb x y (edgep xy)) = (c [set x; y] == ca).
84   Proof. by rewrite altpathE nodesE irred_edge /=. Qed.
85
86   Lemma altpath_edgeL ca cb zx p :
87     altpath ca cb (pcat (edgep zx) p) =
88     (z \notin p) && (c [set z; x] == ca) && (altpath cb ca p).
89   Proof.
90     case: p=> p pth_p.
91     rewrite !altpathE irred_edgeL mem_path !nodesE /=.
92     by rewrite !andbA (andbAC _ _ (z \notin x :: p)) (andbC _ (z \notin x :: p)).
93   Qed.
94
95   Lemma altpath_edgeR ca cb p yz :
96     altpath ca cb (pcat p (edgep yz)) =
97     (z \notin p) && (c [set y; z] == next_col c ca cb (nodes p)) && altpath ca cb p.
98   Proof.
99     by rewrite !altpathE irred_edgeR alternates_rcons !andbA
100       (andbC (alternates c ca cb (nodes p)) _)
101       (andbAC _ _ (z \notin p)) (andbC _ (z \notin p)).
102   Qed.
103
104   Definition altpath_endpt {ca cb p} (ap : altpath ca cb p) (u : G) :=
105     (ca \in absent_set c u) \/\ (cb \in absent_set c u).
106
107   Lemma altpath_endptP {ca cb p} (ap : altpath ca cb p) (u : G) :
108     (ca \in absent_set c u) \/\ (cb \in absent_set c u) -> altpath_endpt ap u.
109   Proof. by []. Qed.
110
111   Lemma altpath_two_endpts {ca cb p} (ap : altpath ca cb p) (u v : G) :
112     altpath_endpt ap z /\ altpath_endpt ap u /\ altpath_endpt ap v ->
113     (z \notin p) \/\ (u \notin p) \/\ (v \notin p).
114   Proof.

```

```

115   Admitted.
116
117 End AltPath.
118
119 Lemma altpath_mem
120   {G : sgraph} {ColorType : finType}
121   (c : edgeColoringType G ColorType)
122   (ca cb : ColorType)
123   (x y u v : G)
124   (p : Path x y) :
125   altpath c ca cb p ->
126   Path_edge p u v ->
127   (c [set u; v] == ca) || (c [set u; v] == cb).
128 Proof.
129   elim: p ca cb=> [_ _ _ /idp_path_edge //|x0 z0 p xz0 IH] ca cb.
130   rewrite altpath_edgeL=> /andP[/andP[Hnin Hc0] Hap] /cat_path_edge.
131   case=> [/edgep_path_edge /andP[/eqP <- /eqP <-]|He]; first by rewrite Hc0.
132   by move/orP: (IH cb ca Hap He)=> [->|->].
133 Qed.
134
135 Section AltMax.
136   Variables (G : sgraph) (ColorType : finType) (pc : properEdgeColoringType G ColorType)
137             (ca cb : ColorType) (x : G).
138   Implicit Types (y : G).
139   Hypothesis start_abs : cb \in absent_set pc x.
140
141   Definition valid_altpath_vertex {y} {p : Path x y} (ap : altpath pc ca cb p) (z : G) :=
142     (z \in N(y)) && ((proper_to_edge_coloring pc) [set y; z] == altpath_next_col ap).
143
144   Lemma valid_altpath_edge
145     {y z} {p : Path x y}
146     {ap : altpath pc ca cb p}
147     : valid_altpath_vertex ap z -> y -- z.
148   Proof.
149     by rewrite/valid_altpath_vertex in_opn=> /andP[-> _].
150   Qed.
151
152   Lemma altpath_rcons
153     {y} {p : Path x y}
154     (ap : altpath pc ca cb p)
155     (z : G)

```

```

156     (Pz : valid_altpath_vertex ap z)
157 : altpath pc ca cb (pcat p (edgep (valid_altpath_edge Pz))).
158 Proof.
159   rewrite altpath_edgeR. move: Pz=> /andP[Hn Hc].
160 Admitted.
161
162 Definition is_apmax {y} {p : Path x y} (ap : altpath pc ca cb p) : Prop :=
163   altpath_next_col ap \in absent_set pc y.
164
165 Equations apmax {y} {p : Path x y} (ap : altpath pc ca cb p)
166 : {z & {q : Path x z & altpath pc ca cb q}} by wf #|[set: G] :\: ([set v in p])| lt :=
167   apmax ap :=
168     match pickP (valid_altpath_vertex ap) with
169     | Pick z Pz => apmax (altpath_rcons Pz)
170     | Nopick _ => existT _ y (existT _ p ap)
171   end.
172 Next Obligation.
173   apply/ltP/proper_card/properP.
174   (* rewrite mem_pcat_edgeR.
175     split.
176     - apply/setDS. *)
177 Admitted.
178
179 Lemma apmax_is_max {y} {p : Path x y} (ap : altpath pc ca cb p)
180 : is_apmax (projT2 (projT2 (apmax ap))).
181 Proof.
182 Admitted.
183
184 Lemma apmax_pcat {y} {p : Path x y} (ap : altpath pc ca cb p) :
185   exists q, projT1 (projT2 (apmax ap)) = pcat p q.
186 Proof.
187 Admitted.
188
189 End AltMax.
190
191 Section Invert.
192 Variables (G : sgraph) (ColorType : finType) (c : edgeColoringType G ColorType).
193 Implicit Types (ca cb : ColorType) (x y : G).
194
195 Definition invert
196   {ca cb x y}

```

```

197   {p : Path x y}
198   (ap : altpath c ca cb p)
199 : edgeColoringType G ColorType :=
200   alternates_invert c ca cb (nodes p).
201
202 Lemma invert_proper
203   {ca cb x y}
204   {p : Path x y}
205   (ap : altpath c ca cb p)
206 :
207   is_proper_edge_coloring c ->
208   is_proper_edge_coloring (invert ap).
209 Proof.
210 Admitted.
211
212 Lemma imset_invert
213   {ca cb x y}
214   {p : Path x y}
215   (ap : altpath c ca cb p)
216 : ca \in c[E(G)] -> (invert ap) [E(G)] \subset c [E(G)].
217 Proof.
218   rewrite/invert.
219   elim: p ca cb c ap => [|u v q uv IH] ca cb d;
220   [|rewrite nodes_pcat /edgep];
221   rewrite !nodesE // -[val (edgep uv)]/[:: v] /behead altpath_edgeL.
222   (* manually rewrite, otherwise unfolds too far *)
223   rewrite -[alternates_invert d ca cb ([:: u; v] ++ \val q)]
224           /(alternates_invert (recolor_edge d [set u; v] ca) cb ca (v::val q)).
225   elim: q IH=> [/= _|w z q' wz _] IH; first by exact: imset_recolor.
226   move=>/andP[/andP[Hnu tmp0] Hap]; move: (Hap).
227   rewrite -nodesE altpath_edgeL=>/andP[/andP[tmp1 /eqP Hwzb] Hap'].
228   have Hcb : cb \in (recolor_edge d [set u; w] ca)[E(G)].
229   {
230     have Hnwz : w != z by move/sg_edgeNeq: (wz); rewrite eq_sym=> /negbT.
231     have : [set w; z] != [set u; w].
232     {
233       apply/eqP; rewrite doubleton_eq_iff; case.
234       - by move=> [_ Hwz]; rewrite Hwz eq_refl in Hnwz.
235       - (* use Hnu and mem lemmas to say u != w *)
236         admit.
237     }

```

```

238   (* have : recolor_neq ([set w; z] != [set u; w]).
239   have : cb \in d [E(G)] by apply/imsetP; exists [set w; z];
240   move: (wz); rewrite // = in_edges. *)
241   admit.
242 }
243 move: (IH cb ca (recolor_edge d [set u; w] ca)).
244 Admitted.
245
246 Lemma card_invert
247   {ca cb x y}
248   {p : Path x y}
249   (ap : altpath c ca cb p)
250 :
251   ca \in c[E(G)] -> #|(invert ap) [E(G)]| <= #|c [E(G)]|.
252 Proof.
253 Admitted.
254
255 End Invert.
256
257 Section InvertProp.
258 Variables (G : sgraph) (ColorType : finType) (pc : properEdgeColoringType G ColorType)
259           (ca cb : ColorType) (x y : G) (p : Path x y) (ap : altpath pc ca cb p).
260 Implicit Types (u v : G).
261 Hypothesis Ha : is_apmax ap.
262
263 Definition not_mem_inverted : Prop :=
264   forall u v, ~ Path_edge p u v ->
265   (proper_to_edge_coloring pc) [set u; v] = (invert ap) [set u; v].
266
267 Definition mem_inverted : Prop :=
268   forall u v, Path_edge p u v ->
269   (((proper_to_edge_coloring pc) [set u; v] = ca) <-> ((invert ap) [set u; v] = cb)) /\
270   (((proper_to_edge_coloring pc) [set u; v] = cb) <-> ((invert ap) [set u; v] = ca)).
271
272 Lemma invert_is_inverted : not_mem_inverted /\ mem_inverted.
273 Proof.
274   rewrite/not_mem_inverted/mem_inverted; split=> u v Hp.
275 Admitted.
276
277 Lemma invert_absent_not_mem {u} (Hu : u \notin p) :
278   (invert ap)[E(G)] = pc[E(G)] -> absent_set pc u = absent_set (invert ap) u.

```

```

279 Proof.
280   have : pc [E{G;u}] = invert ap [E{G;u}].
281   {
282     apply/setP=> c0.
283     apply/imsetP/imsetP; move=> [e Hin ->]; exists e=> //;
284     move: Hin; rewrite/edge_neigh=> /imsetP [v] Hn ->;
285     have Hp := (vert_not_in_path (or_introl Hu));
286     move: invert_is_inverted=> [+ _];
287     rewrite/not_mem_inverted=> Hnm;
288     by have := Hnm u v (Hp v).
289   }
290   by rewrite/absent_set=> -> ->.
291 Qed.
292
293 Lemma invert_absent_ca {u}
294   (Hu : u \in p)
295   (Habu : cb \in absent_set pc u)
296 : ca \in absent_set (invert ap) u.
297 Proof.
298 Admitted.
299
300 Lemma invert_absent_cb {u}
301   (Hz : u \in p)
302   (Habu : ca \in absent_set pc u)
303 : cb \in absent_set (invert ap) u.
304 Proof.
305 Admitted.
306
307 Lemma invert_fan_nodes {u v : G} (f1 : Fan pc x v u) (f2 : Fan (invert ap) x v u):
308   (u :: val f1) = (u :: val f2).
309 Proof.
310 Admitted.
311
312 Lemma invert_fan {u v : G} (f : Fan pc x v u) :
313   cb \in absent_set pc x -> Fan (invert ap) x v u.
314 Proof.
315 Admitted.
316
317 End InvertProp.

```

A.8 VIZING'S THEOREM

```

1  From HB Require Import structures.
2  From mathcomp Require Import all_boot.
3  From GraphTheory Require Import edone preliminaries digraph sgraph.
4  Require Import aux spath edge_coloring fans alternate_path kempe.
5  From Equations Require Import Equations.
6
7  Set Warnings "-notation-overridden, -notation-incompatible-prefix".
8
9  Set Implicit Arguments.
10 Unset Strict Implicit.
11 Unset Printing Implicit Defensive.
12
13 Lemma smaller_coloring
14   {G : sgraph} {v w0 wj : G} {k}
15   {c : kEdgeColoringType G k}
16   (f : Fan c v w0 wj) (cj : projT1 c) :
17   k = max_degree G + 1 + 1 ->
18   cj \in (absent_set c v :&: absent_set c wj) ->
19   k_edge_colorable G (max_degree G + 1).
20 Proof.
21   move=> Hk Hcvw.
22   have Hneigh : wj \in N(v) := (in_neigh (mem_head wj (val f))).
23   have Hvw : [set v; wj] \in E(G).
24   { by move: Hneigh; rewrite in_opn in_edges. }
25   pose c' := rotateF f.
26   have Hprop' : is_proper_edge_coloring c' :=
27     rot_proper (proj2_sig (k_to_proper_coloring c)).
28   have Hin' : cj \in c'[E(del_edges [set v; wj])].
29   {
30     rewrite in_setI in Hcvw; move/andP: (Hcvw)=> [Hcv _]; move: (Hcv).
31     rewrite /absent_set (imset_rot f) (imset_rot_vertex f) /coloring_image/c'
32       => /setDP[/imsetP [ej Hej] Hcj _].
33     rewrite (del_edges1 Hvw) in_setU1 in Hej; rewrite (rot_absent_center f) in Hcv.
34     have Hneq: ej != [set v; wj] by move: (absent_edge Hcv Hneigh); rewrite Hcj;
35     apply contra_neq=> ->.
36     rewrite (negbTE Hneq) orFb in Hej.
37     by apply/imsetP; exists ej.
38   }
39   have Hnotin': c'[set v; wj] \notin c'[E(del_edges [set v; wj])] by exact: rot_w0_prop.

```



```

40 pose c' := recolor_edge c' [set v; wj] cj.
41 have Hprop' := recolor_proper Hprop' (rot_absent_fan (mem_head wj (val f)) Hcvw).
42 move: (replace_col Hvw Hin' Hnotin').
43 rewrite -card_rot (card_k_col c).
44 have ->: k - 1 = max_degree G + 1 by rewrite Hk addn1 subn1.
45 move=> Hcard'.
46 by constructor; exists (projT1 c), (exist _ c' Hprop'); rewrite Hcard'.
47 Qed.
48
49 Lemma base_case (G : sgraph) (no_edges : E(G) = set0) :
50   k_edge_colorable G 0 /\ 0 <= max_degree G + 1.
51 Proof.
52   split; last by [].
53   move/eqP: no_edges; rewrite -cards_eq0=> /eqP <-.
54   exact/inj_chrom.
55 Qed.
56
57 Theorem Vizings_altpath (G : sgraph) (chi : nat):
58   is_chromatic_index G chi ->
59   max_degree G <= chi <= max_degree G + 1.
60 Proof.
61   move=> is_chi;
62   rewrite chi_lower_bound //=.
63   apply (chi_upper_bound_trans is_chi)=> {is_chi} {chi}.
64   (* Induction on # of edges *)
65   elim/(size_ind (fun G => #|E(G)|)) : G=> G IH.
66   case: (set_0Vmem E(G))=> [/base_case|e Ein0]; first by exists 0.
67   have [v [w0] [Edef0 _]] :=
68     edgesP _ Ein0; rewrite {}Edef0 in Ein0; set G' := del_edges [set v; w0].
69   have leq_gg': max_degree G' + 1 <= max_degree G + 1
70     by rewrite leq_add2r; exact: del_edges_max_deg.
71   have{}/IH [k' [[kc'] leq_kg']] : #|E(G')| < #|E(G)|
72     by apply: proper_card; exact: del_edges_proper Ein0 _.
73   have: k' <= max_degree G + 1 by exact: (leq_trans leq_kg' leq_gg').
74   rewrite leq_eqVlt=> /orP[/eqP eq_kg[] {leq_kg'}]; first last.
75   - (* if k' < max_degree G + 1, then we are done *)
76     pose kc := k_extended_col Ein0 kc'.
77     exists (k' + 1); by split; [|rewrite addn1|.
78   (* now, k' = max_degree G + 1 *)
79   rewrite {}eq_kg in kc'; pose kc := k_extended_col Ein0 kc'.
80   (* create a maximal fan from w0 to w *)

```

```

81 pose f0: Fan kc v w0 w0 := k_Fan_of_del_edges Ein0 kc'.
82 case Hfmax: (fanmax f0) => [w fmax].
83 have is_fmax: is_fanmax fmax by move: (fanmax_is_max f0); rewrite {}Hfmax /=.
84 (* there exists some color c0 absent at w *)
85 move: (exists_absent_color kc' leq_gg' w) => [c0] Habw0' {leq_gg'}.
86 have{Habw0'} Habw0 := extended_absent Ein0 Habw0'.
87 case: (boolP (Some c0 \in absent_set kc v)) => [Habv0 | Hnabv0].
88 - (* if c0 is absent at v, we can replace extra color with c0 *)
89   have Hcap: (Some c0 \in absent_set kc v :&: absent_set kc w)
90     by apply/setIP/(conj Habv0 Habw0).
91   by exists (max_degree G + 1); move: (smaller_coloring fmax erefl Hcap).
92 (* Otherwise, we will need to create a fan and rotate *)
93 move: (exists_absent_color kc (leq_addr _ (max_degree G + 1)) v) => [c1] Habv1.
94 (* There also exists an edge v--wj colored c0, where wj != w0 is in the fan *)
95 have := (fanmax_present is_fmax Hnabv0 Habw0)
96   => [[wj] /andP[Einj /andP[/eqP Hkcj Hfanj]]] {is_fmax Hnabv0}.
97 have Evj : v -- wj by rewrite in_edges in Einj.
98 (* split fan at wj as f1 and (wj::f2) *)
99 case/splitPr fsplit: (w::val fmax)/Hfanj => [f1 f2 _].
100 case: f2 fsplit=> [|wi f2'] fsplit.
101 - (* contradiction if f2 is empty *)
102   have Hneqj0 : wj != w0.
103   {
104     apply: (@contra_neq _ _ ((sval (k_to_proper_coloring kc)) [set v; wj])
105       ((sval (k_to_proper_coloring kc)) [set v; w0]) _ _)> [-> //|];
106     have /eqP -> :
107       (sval (k_to_proper_coloring kc)) [set v; w0] == None
108       by move/eqP: (w0_col_extended kc').
109     by rewrite Hkcj.
110   }
111   by rewrite -(fan_last fmax) -(last_cons w w) fsplit cats1 last_rcons eq_refl in Hneqj0.
112 (* so f2 is non-empty, i.e. wi != w0 *)
113 have Habwi0 : Some c0 \in absent_set kc wi.
114 {
115   move: fsplit.
116   by case: f1=> [|wk f1']; [rewrite cat0s|];
117   case=> Hw Hfval; have := fanW fmax;
118   rewrite Hfval /absent_prop; [rewrite Hw | rewrite -cat_rcons cat_path last_rcons];
119   rewrite /path Hkcj=> /andP[_ +] //; move=> /andP[-> _].
120 }
121 rewrite -[wj :: wi :: f2']cats1 catA in fsplit.

```

```

122 have fsmallest := sub_fan fsplit.
123 (* Construct c0 c1 Kempe Chain starting with v--wj *)
124 move/eqP: (Hkcj); rewrite -(altpath_edge kc _ c1)=> ap0.
125 case Hapmax: (apmax Habv1 ap0) => [z [pth apm]].
126 have Hpv : v \in pth by exact: path_begin.
127 have := apmax_pcat Habv1 ap0; rewrite Hapmax /= => [[q] Hq].
128 have HaisMax : is_apmax apm by move: (apmax_is_max Habv1 ap0); rewrite {}Hapmax /=.
129 have /(imset_invert apm) /eqVproper : Some c0 \in kc[E(G)]
130   by apply/in_c_all_edgeP; exists [set v; wj].
131 pose Hkcp : properEdgeColoringType G (projT1 kc) :=
132   (exist _ (invert apm) (@invert_proper _ _ _ _ _ _ apm
133     (proj2_sig (k_to_proper_coloring kc)))).
134 case=> Hsi; first last.
135 - exists #|Hkcp [E(G)]|.
136   split; first by constructor; exact (proper_to_k_coloring Hkcp).
137   move/proper_card: Hsi.
138   rewrite card_k_col -[#|Hkcp [E(G)]|]/(#|invertapm [E(G)]|)=> Hsi.
139   by rewrite -(leq_add2r 1); rewrite -addn1 in Hsi.
140 have card_ic : #|Hkcp [E(G)]| = (max_degree G + 1 + 1) by rewrite Hsi card_k_col.
141 pose ikc : kEdgeColoringType G (max_degree G + 1 + 1) :=
142   existT _ (projT1 kc) (exist _ Hkcp card_ic).
143 (* v is an endpoint because c1 was absent here *)
144 have Hcj : (invert apm) [set v; wj] = c1.
145 {
146   have Hp : Path_edge pth v wj
147     by rewrite Hq; apply cat_path_edge; left; apply edgep_path_edge.
148   move: (proj2 (invert_is_inverted HaisMax) v wj Hp)=> [/iffLR H _]; exact: H Hkcj.
149 }
150 have Hkci_fmax : Fan ikc v w0 w := invert_fan HaisMax fmax Habv1.
151 have Habv0 : Some c0 \in absent_set ikc v := invert_absent_ca HaisMax Hpv Habv1.
152 case: (boolP (wi \in pth))=> Hpwi; exists (max_degree G + 1).
153 - (* wi is in the path of apm *)
154   have Hpw : w \notin pth by
155     case
156       (altpath_two_endpts
157         (conj (altpath_endptP apm (or_intror Habv1))
158           (conj (altpath_endptP apm (or_introl Habwi0))
159             (altpath_endptP apm (or_introl Habw0))))
160       ));
161   [rewrite Hpv | case; [rewrite Hpwi|]].
162   rewrite (invert_absent_not_mem HaisMax Hpw Hsi) in Habw0.

```

```

163   have Hcap : (Some c0 \in absent_set ikc v :&: absent_set ikc w)
164     by apply/setIP/(conj Habv0 Habw0).
165   by have := (smaller_coloring Hkci_fmax erefl Hcap).
166   - (* wi is not in the alternating path *)
167     rewrite (invert_absent_not_mem HaisMax Hpwi Hsi) in Habwi0.
168     rewrite {}(invert_fan_nodes HaisMax fmax Hkci_fmax) in fsplit.
169     have Hkci_fsmallest : Fan ikc v w0 wi := sub_fan fsplit.
170     have Hcap : (Some c0 \in absent_set ikc v :&: absent_set ikc wi)
171       by apply/setIP/(conj Habv0 Habwi0).
172     by have := (smaller_coloring Hkci_fsmallest erefl Hcap).
173   Qed.
174
175 Theorem Vizings_kempe (G : sgraph) (chi : nat):
176   is_chromatic_index G chi ->
177   max_degree G <= chi <= max_degree G + 1.
178 Proof.
179   move=> is_chi;
180   rewrite chi_lower_bound //=.
181   apply (chi_upper_bound_trans is_chi)=> {is_chi} {chi}.
182   (* Induction on # of edges *)
183   elim/(size_ind (fun G => #|E(G)|)) : G=> G IH.
184   case: (set_0Vmem E(G))=> [/base_case|[e Ein0]]; first by exists 0.
185   have [v [w0] [Edef0 _]] :=
186     edgesP _ Ein0; rewrite {}Edef0 in Ein0; set G' := del_edges [set v; w0].
187   have leq_gg' : max_degree G' + 1 <= max_degree G + 1
188     by rewrite leq_add2r; exact: del_edges_max_deg.
189   have{}/IH [k' [[kc'] leq_kg']] : #|E(G')| < #|E(G)|
190     by apply: proper_card; exact: del_edges_proper Ein0 _.
191   have: k' <= max_degree G + 1 by exact: (leq_trans leq_kg' leq_gg').
192   rewrite leq_eqVlt=> /orP[/eqP eq_kg[] {leq_kg'}]; first last.
193   - (* if k' < max_degree G + 1, then we are done *)
194     pose kc := k_extended_col Ein0 kc'.
195     exists (k' + 1); by split; [|rewrite addn1|.
196     (* now, k' = max_degree G + 1 *)
197     rewrite {}eq_kg in kc'; pose kc := k_extended_col Ein0 kc'.
198     (* create a maximal fan from w0 to w *)
199     pose f0: Fan kc v w0 w0 := k_Fan_of_del_edges Ein0 kc'.
200     case Hfmax: (fanmax f0) => [w fmax].
201     have is_fmax: is_fanmax fmax by move: (fanmax_is_max f0); rewrite {}Hfmax /=.
202     (* there exists some color ck absent at w *)
203     move: (exists_absent_color kc' leq_gg' w)=> [ck] Habw0' {leq_gg'}.

```

```

204 have{Habw0'} Habw0 : Some ck \in absent_set kc w := extended_absent Ein0 Habw0'.
205 case: (boolP (Some ck \in absent_set kc v))=> [Habv0 | Hnabv0].
206 - (* if ck is absent at v, we can replace extra color with ck *)
207   have Hcap: (Some ck \in absent_set kc v :&: absent_set kc w)
208     by apply/setIP/(conj Habv0 Habw0).
209   by exists (max_degree G + 1); move: (smaller_coloring fmax erefl Hcap).
210   (* Otherwise, we will need to create a fan and rotate *)
211   move: (exists_absent_color kc (leq_addr _ (max_degree G + 1)) v) => [cv] Habv1.
212   (* There also exists an edge v--wj colored ck, where wj != w0 is in the fan *)
213   have := (fanmax_present is_fmax Hnabv0 Habw0)
214     => [[wj] /andP[Einj /andP[/eqP Hkcj Hfanj]]] {is_fmax Hnabv0}.
215   have Evj : v -- wj by rewrite in_edges in Einj.
216   (* split fan at wj as f1 and (wj::f2) *)
217   case/splitPr fsplit: (w::fval fmax)/Hfanj=> [f1 f2 _].
218   have Hneqj0 : wj != w0.
219   {
220     apply: (@contra_neq _ _ ((proper_to_edge_coloring (k_to_proper_coloring kc)) [set v; wj]))
221       ((proper_to_edge_coloring (k_to_proper_coloring kc)) [set v; w0]) _ => [-> //]];
222     have /eqP -> :
223       (proper_to_edge_coloring (k_to_proper_coloring kc))
224       [set v; w0] == None by move/eqP: (w0_col_extended kc').
225     by rewrite Hkcj.
226   }
227   case: f2 fsplit=> [|wi f2'] fsplit.
228   - (* contradiction if f2 is empty *)
229     by rewrite -(fan_last fmax) -(last_cons w w) fsplit cats1 last_rcons eq_refl in Hneqj0.
230   (* so f2 is non-empty, i.e. wi != w0 *)
231   have kc_wia_ck : Some ck \in absent_set kc wi.
232   {
233     move: fsplit.
234     by case: f1=> [|wk f1']; [rewrite cat0s];
235     case=> Hw Hfval; have := fanW fmax;
236     rewrite Hfval /absent_prop; [rewrite Hw | rewrite -cat_rcons cat_path last_rcons];
237     rewrite /path Hkcj=> /andP[_ +] //; move=> /andP[-> _].
238   }
239
240   (* Construct ck cv Kempe Chain containing v *)
241   pose chain := kempe_chain kc (Some ck) cv v.
242   have chain_refl : chain = kempe_chain kc (Some ck) cv v by [].
243   set ic := invertedChain (kempe_chain kc (Some ck) cv v).
244   set ipc : properEdgeColoringType G (projT1 kc) :=

```

```

245     (exist _ (ic) (inverted_proper chain_refl)).
246 have /eqVproper [/esym imset_kc_ic | imset_kc_ic] : ic[E(G)] \subset kc[E(G)] :=
247   imset_invert_sub chain (absent_in_imset kc_wia_ck) (absent_in_imset Habv1); first last.
248 - exists #|ipc [E(G)]|.
249   split; first by constructor; exact (proper_to_k_coloring ipc).
250   move/proper_card: imset_kc_ic.
251   rewrite card_k_col -[#|ipc [E(G)]|]/[#|ic [E(G)]|]=> card_ic.
252   by rewrite -(leq_add2r 1); rewrite -addn1 in card_ic.
253 have card_ic : #|ic[E(G)]| == (max_degree G + 1 + 1) by rewrite -imset_kc_ic card_k_col.
254 pose ikc : kEdgeColoringType G (max_degree G + 1 + 1) :=
255   existT _ (projT1 kc) (exist _ ipc card_ic).
256 have v_in_v := (@in_component_of (kempe_graph kc (Some ck) cv) v).
257 have ic_va_ck : Some ck \in absent_set ikc v.
258 {
259   have [_ H] := in_inverted_absent chain_refl v_in_v.
260   by apply H.
261 }
262 have [u [ikc_fan ic_ua_ck]] : (exists _ : _, exists _ : Fan ikc v w0 _, _)
263   := inverted_fan chain_refl imset_kc_ic fsplit Hneqj0 Habv1 kc_wia_ck Habw0 Hkcj.
264 have Hcap : (Some ck \in absent_set ikc v :&: absent_set ikc u)
265   by apply/setIP/(conj ic_va_ck ic_ua_ck).
266 by exists (max_degree G + 1); have := smaller_coloring ikc_fan erefl Hcap.
267 Qed.

```